

C++17 数据结构与算法：原理精要、工程实现与教学指南

全书总目录

第一部分：绪论

- 第 1 章 复杂度分析与 ADT

第二部分：线性结构

- 第 2 章 线性表
- 第 3 章 栈与队列

第三部分：串

- 第 4 章 字符串匹配 (KMP)

第四部分：树与二叉树

- 第 5 章 二叉树基础与遍历
- 第 6 章 二叉搜索树
- 第 7 章 堆与优先队列
- 第 8 章 平衡二叉树
- 第 9 章 多路搜索树
- 第 10 章 多维搜索树 (KD 树)

第五部分：图

- 第 11 章 图论算法

第六部分：查找

- 第 12 章 哈希表
- 第 13 章 并查集

第七部分：排序

- 第 14 章 排序算法

第八部分：高级字符串

- 第 15 章 后缀树与后缀数组

附录

- 附录 A: 数据结构选择速查
- 附录 B: 常见面试题与练习
- 附录 C: 常见陷阱与注意事项

所属模块：第一部分：绪论
对应实现：complexity_adt.cpp

本章导航

- 1.1 算法评价的本质：为什么需要复杂度分析？
- 1.2 三大渐进记号的严格定义与几何直观
- 1.3 常见复杂度阶级与物理时空感受
- 1.4 代码复杂度分析实战法则
- 1.5 分治递归的利器：主定理（Master Theorem）与递归树直观
- 1.6 均摊分析（Amortized Analysis）三大法宝
- 1.7 抽象数据类型（ADT）与现代 C++ 契约设计
- 本章小结
- 思考题与课后自测

第 1 章 复杂度分析与 ADT

难度：★☆☆☆☆ | 代码：complexity_adt.cpp

前置知识：基本的 C++ 语法、函数调用与循环结构。本章为全书的理论基石，定义了全书用于度量算法优劣的统一语言。

学习目标：1. 建立对“增长率”而非“绝对运行秒数”的直观敏感度；2. 掌握 O 、 Ω 、 Θ 三大渐进记号的数学定义、极限判别法与适用语境；3. 熟练运用主定理（Master Theorem）和递归树求解分治递归式；4. 理解均摊分析（聚合分析、记账法、势能法）与“平均情况”的本质区别；5. 掌握抽象数据类型（ADT）的核心理念：以“接口与实现分离”构建高内聚低耦合的代码。

1.1 算法评价的本质：为什么需要复杂度分析？

许多初学者在刚接触编程时，往往习惯使用 `std::chrono` 记录代码运行前后的时间戳，用“耗费了多少毫秒”来衡量一个算法的快慢。然而，**绝对运行时间是极不可靠的**：- 它受制于硬件配置（酷睿 i9 与 树莓派的执行速度判若

云泥)； - 它受制于编译器优化等级（-O0 与 -O3 差异可达数倍至数十倍）； - 它受制于输入规模与测试数据的偶然性（对 10 个数字排序和对 1000 万个数字排序无法直接比较）。

为了摆脱硬件与实现细节的纠缠，计算机科学家图灵奖得主 Donald Knuth 等人确立了**渐进分析 (Asymptotic Analysis)** 框架： - **核心关注点**：当输入规模 ($n \rightarrow \infty$) 时，算法运行时间或空间占用的**增长率 (Growth Rate)**。 - **基本假设 (RAM 模型)**：每条基本指令（加减乘除、指针解引用、数组寻址、赋值）均在常数时间 ($O(1)$) 内完成。

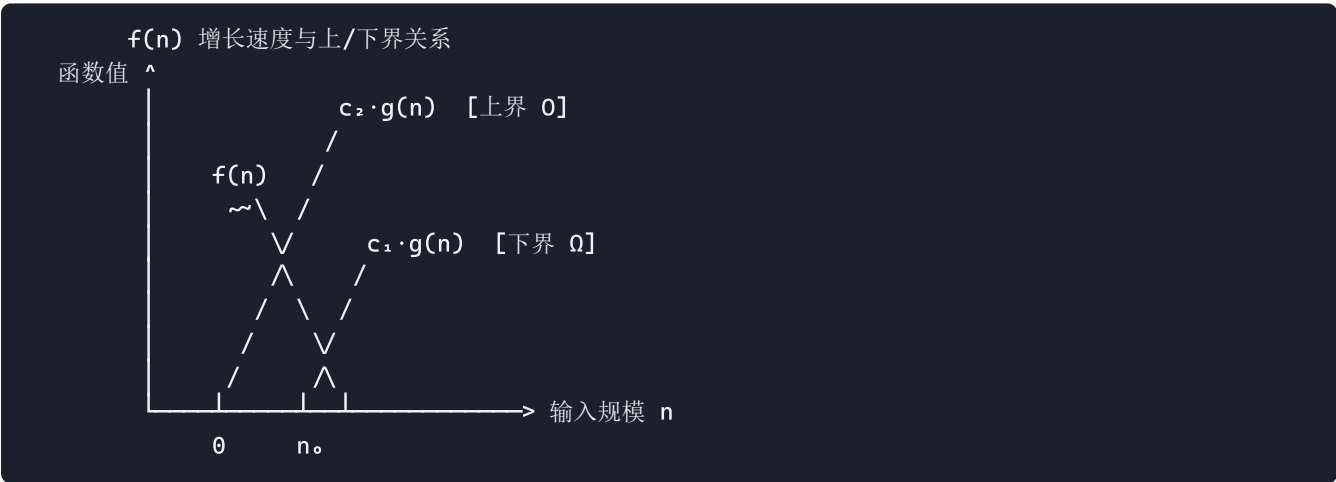
```
算法 A:  $T(n) = 100n$ 
算法 B:  $T(n) = 0.01n^2$ 

当  $n = 10$  时:    $T_A = 1000$ ,    $T_B = 1$            (算法 B 胜出)
当  $n = 1000$  时:  $T_A = 100,000$ ,  $T_B = 10,000$       (算法 B 依然较快)
当  $n = 100,000$  时:  $T_A = 10^7$ ,    $T_B = 10^8$       (算法 A 彻底反超!)
当  $n = 10^7$  时:    $T_A = 10^9$ ,    $T_B = 10^{12}$      (算法 B 已经崩溃, 相差千倍)
```

结论：高阶项永远主导低阶项。随着数据规模扩大，高阶算法的劣势无法通过升级硬件来弥补。这就是我们为什么追求更低复杂度算法的根本原因。

1.2 三大渐进记号的严格定义与几何直观

在经典教材（如 CLRS《算法导论》、邓俊辉《数据结构》）中，渐进记号有着严格的集合与极限定义。



1.2.1 渐进上界: O (Big-Oh)

- **数学定义**:
$$O(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0, \text{使得 } \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n) \}$$
- **含义**：存在一个常数 (c) 和阈值 (n_0)，当 (n) 超过 (n_0) 后，($f(n)$) 永远不会超过 ($c \cdot g(n)$)。它刻画了算法性能的**最坏情况上界**。
- **极限法判定**：若 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = L < \infty$ ，则 $f(n) = O(g(n))$ 。

1.2.2 渐进下界: Ω (Big-Omega)

- **数学定义**:
$$\Omega(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0, \text{使得 } \forall n \geq n_0, 0 \leq c \cdot g(n) \leq f(n) \}$$

- **含义：**算法无论在何种情况下，其耗时增长率**至少**和 $(g(n))$ 一样快。它常用于证明某个问题的**理论计算下界**（例如：任何基于比较的排序问题其时间下界为 $(\Omega(n \log n))$ ）。

1.2.3 渐进紧确界： (Θ) (Big-Theta)

- **数学定义：**
 $(\Theta(g(n))) = \{ f(n) \mid \exists c_1, c_2 > 0, n_0 > 0, \text{使得 } \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \}$
- **充要条件：** $(f(n) = \Theta(g(n)) \text{ iff } f(n) = O(g(n)) \text{ 且 } f(n) = \Omega(g(n)))$ 。
- **含义：** $(f(n))$ 与 $(g(n))$ 同阶增长，上下界被严格夹逼。

工程提示与误区澄清：

口语中工程师常说的“这个算法复杂度是 $(O(n \log n))$ ”，如果它在最好、最坏和平均情况下都是这个量级（如归并排序），学术上更严谨的表述其实是 $(\Theta(n \log n))$ 。但根据业界惯例，一般用 (O) 来泛指最坏情况运行时间。

1.3 常见复杂度阶级与物理时空感受

下表以现代主流计算机（假设每秒执行 (10^8) 次基本操作，约 100 MIPS）为基准，直观感受不同量级在不同规模下的表现：

记号	名称	$(n=100)$ 计算步数	$(n=10^6)$ 耗时预估	典型场景 / 算法
$(O(1))$	常数阶	1	$(\approx 10 \text{ ns})$	数组寻址、哈希表查找、栈 push/pop
$(O(\log n))$	对数阶	(≈ 7)	$(\approx 0.2 \mu\text{s})$	二分查找、平衡树 (AVL/红黑树) 查找
$(O(n))$	线性阶	100	$(\approx 10 \text{ ms})$	线性遍历扫描、计数排序、单链表遍历
$(O(n \log n))$	线性对数阶	(≈ 700)	$(\approx 0.2 \text{ s})$	快速排序、归并排序、堆排序、倍增算法
$(O(n^2))$	平方阶	(10^4)	$(\approx 2.7 \text{ 小时})$	冒泡/选择/插入排序、双重循环枚举
$(O(n^3))$	立方阶	(10^6)	$(\approx 31.7 \text{ 年})$	Floyd-Warshall 全源最短路、矩阵乘法朴素算法
$(O(2^n))$	指数阶	(1.27×10^{30})	宇宙毁灭亦无法算完	旅行商问题 (TSP) 暴力回溯、子集枚举
$(O(n!))$	阶乘阶	(9.33×10^{157})	不可承受之重	全排列暴力生成

关键经验法则（竞赛与系统设计通用）： - 若 $(n \leq 10^8)$ ：可设计 $O(n)$ 算法； - 若 $(n \leq 10^5 \sim 10^6)$ ：必须控制在 $O(n \log n)$ ； - 若 $(n \leq 1000 \sim 5000)$ ：允许 $O(n^2)$ ； - 若 $(n \leq 20 \sim 30)$ ：通常预示着要使用状压 DP 或回溯搜索 ($O(2^n)$ 或 $O(n \cdot 2^n)$)。

1.4 代码复杂度分析实战法则

分析一段代码的时间复杂度并不需要复杂的微积分，掌握以下组合规则即可化繁为简：

1. 加法法则（顺序执行取大者）：

$$(T(n) = T_1(n) + T_2(n) = O(f(n)) + O(g(n)) = O(\max(f(n), g(n))))$$

例：先对数组排序 ($O(n \log n)$)，再遍历打印一遍 ($O(n)$)，总体耗时依然是 $O(n \log n)$ 。

2. 乘法法则（嵌套循环取乘积）：

$$(T(n) = T_1(n) \times T_2(n) = O(f(n)) \times O(g(n)) = O(f(n) \cdot g(n)))$$

例：外层循环遍历 (n) 次，内层调用二分查找 ($O(\log n)$)，总体为 $O(n \log n)$ 。

3. 对数步长循环：

```
cpp for (int i = 1; i < n; i *= 2) { // 每次 i 翻倍，循环次数满足  $2^k \geq n \Rightarrow k = \lceil \log_2 n \rceil$  } // 复杂度  $O(\log n)$ 
```

4. 双指针 / 滑动窗口技巧（均摊分析的雏形）：

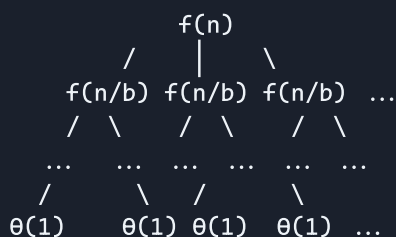
```
cpp int left = 0; for (int right = 0; right < n; ++right) { while (condition && left <= right) { left++; // 尽管有两层循环，但 left 在整个算法过程中最多递增 n 次 } } // 总体依然是  $O(n)$  而非  $O(n^2)$ 
```

1.5 分治递归的利器：主定理（Master Theorem）与递归树直观

分治算法将规模为 (n) 的大问题拆分为 (a) 个规模为 (n/b) 的子问题，拆分与合并的耗时为 $(f(n))$ 。其通用的递归关系式为： $[T(n) = a \cdot T(\frac{n}{b}) + f(n)]$ 其中 $(a \geq 1, b > 1)$ ， $(f(n))$ 为渐进正函数。

1.5.1 递归树的直观博弈

在递归树模型中： - 树的深度为 $(\log_b n)$ ； - 第 (i) 层节点数为 (a^i) ； - 叶子节点总数为 $(a^{\log_b n} = n^{\log_b a})$ ； - 博弈的核心：叶子节点的总工作量 $(\Theta(n^{\log_b a}))$ 与 根节点拆分合并代价 $(f(n))$ 谁占主导地位？



1.5.2 三种情况判定表

情况	条件 (本质)	结论	典型算法实例
Case 1: 树叶主导	$(f(n) = O(n^{\log_b a - \epsilon})), (\epsilon > 0)$ (底层叶子开销远大于顶层)	$(T(n) = \Theta(n^{\log_b a}))$	Strassen 矩阵乘法: $(T(n) = 7T(n/2) + O(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81}))$
Case 2: 各层均等	$(f(n) = \Theta(n^{\log_b a}))$ (每层工作量同阶)	$(T(n) = \Theta(n^{\log_b a} \log n))$	归并排序: $(T(n) = 2T(n/2) + O(n) \Rightarrow a=2, b=2, n^{\log_2 2} = n^1 \Rightarrow \Theta(n \log n))$ 二分查找: $(T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n))$
Case 3: 树根主导	$(f(n) = \Omega(n^{\log_b a + \epsilon}))$ 且满足正则条件 $(a f(n/b) \leq c f(n) (c < 1))$	$(T(n) = \Theta(f(n)))$	某些特殊树型查找: $(T(n) = 2T(n/2) + O(n^2) \Rightarrow \Theta(n^2))$

1.6 均摊分析 (Amortized Analysis) 三大法宝

在工程中，常常遇到这样一种数据结构：**大多数操作极其便宜（如 $O(1)$ ），但偶尔某次操作会非常昂贵（如 $O(n)$ ）**。如果仅仅用最坏情况 $O(n)$ 来概括每一次操作，会极其悲观且失真。

核心原则：均摊分析保证的是任意连续 (m) 次操作序列的最坏总代价不超过 $(m \cdot T_{\text{amortized}})$ 。它不是统计学上的“平均情况”（不需要假设概率分布），而是拥有数学确定性的最坏界限！

方法一：聚合分析 (Aggregate Analysis)

直接求出包含 (n) 次操作的整个序列的总开销 $(T_{\text{total}}(n))$ ，然后均摊为 $(T_{\text{total}}(n) / n)$ 。 - **动态数组**

扩容案例：

设初始容量为 1，每次满时容量翻倍 $(1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow (2^k))$ 。

连续插入 (n) 个元素，只有当 $(i = 1, 2, 4, 8, \dots)$ 时才触发复制。

所有搬移操作的总次数为： $(1 + 2 + 4 + \dots + 2^{\lfloor \log_2 n \rfloor}) < 2n$ 。

加上每个元素本身写入的 (n) 次开销，总成本 $(< 3n)$ 。

故每次 **push_back** 的均摊时间为 $(3n / n = O(1))$ 。

方法二：记账法 (Accounting Method)

为每种操作制定一个“虚拟收费标准（均摊代价）”。当实际成本低于收费时，将盈余作为“信用（Credit）”存入数据结构的节点中；当发生昂贵操作时，提取已存入的信用支付开销。**关键是任何时刻信用总额不能为负。** - **动态数组记账模型：**

对每次普通的插入，我们收取 **3 元**：1. 第 1 元支付本次将元素放入数组的即时成本；2. 第 2 元存为自身未来的信用（当它将来被搬移到新数组时作为搬移费）；3. 第 3 元存为对应前序某个已经在数组中但尚未有搬移费的元素

的信用。

当数组满需要搬移时，所有元素的搬移费早已全额付清，无须额外开销。均摊代价稳稳维持在 $O(1)$ 。

方法三：势能法 (Potential Method)

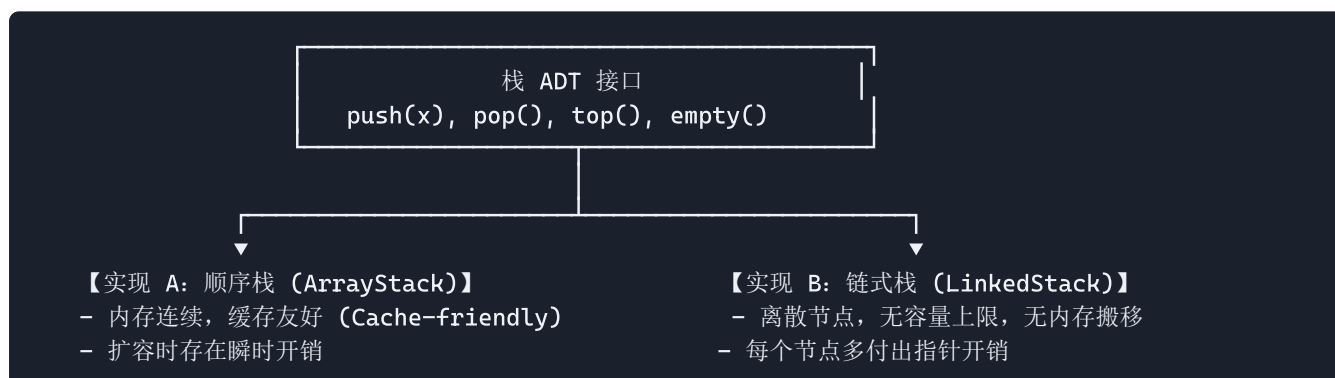
将数据结构当前状态映射为一个非负的“势能”数值 ($\Phi(D_i)$)，初始状态 ($\Phi(D_0) = 0$)。

定义第 i 次操作的均摊代价为实际代价加势能增量： $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ 。只要保证 ($\Phi(D_n) \geq \Phi(D_0)$)，则总实际代价一定被总均摊代价所限定。势能法在高级数据结构（如斐波那契堆、Splay 树的均摊复杂度证明）中是无可替代的利器。

1.7 抽象数据类型 (ADT) 与现代 C++ 契约设计

1.7.1 什么是 ADT?

抽象数据类型 (Abstract Data Type, ADT) 是纯粹从逻辑层面对数据模型以及施加于其上的操作的数学描述。它遵循“做什么 (What)”与“怎么做 (How)”彻底解耦的原则。



1.7.2 现代 C++17 中的 ADT 工程实践

在现代 C++ 开发中，ADT 理念贯彻于：1. **面向接口编程**：通过纯虚基类（抽象接口）或 C++20 Concepts 约束类型；2. **RAII 资源管理**：在构造函数中申请资源，在析构函数中释放资源，保证异常安全；3. **值语义与移动语义**：善用移动构造 `T(T&&)` 与移动赋值 `operator=(T&&)`，让复杂的内部数据结构具备高效转移的能力。

本章小结

- 渐进性是第一原则**：算法的好坏由输入规模扩大时的增长趋势决定，而非机器上的瞬时读秒。
- 大 O 并不等同于最坏情况**： O 在数学上仅代表上界；但在实际工程沟通中，我们约定俗成用最坏情况的大 O 作为最可靠的性能底线。
- 主定理是一面照妖镜**：掌握递归树中“树根拆分”与“树叶合并”的博弈，就能瞬间看穿绝大部分分治算法的渐进复杂度。
- 均摊不等于平均**：均摊分析是确定性的最坏界限保证，为工业界动态扩容容器（如 `std::vector`）的常数级尾插提供了坚不可摧的数学证明。
- ADT 是软件架构之魂**：坚持逻辑契约与物理实现分离，是用 C++ 编写高可靠、可测试、易扩展工程代码的核心思想。

思考题与课后自测

1. **对数底数问题**：为什么在算法分析中，我们通常直接写 $O(\log n)$ ，而不写底数（如 $O(\log_2 n)$ 或 $O(\log_{10} n)$ ）？请从换底公式角度给出严格证明。
2. **主定理实战应用**：
 3. 求 $T(n) = 3T(n/4) + n \log n$ 的渐进复杂度；
 4. 求 $T(n) = 2T(n/2) + n / \log n$ 能否直接应用主定理？若不能，请用递归树方法估计其界。
5. **缩容策略的均摊陷阱**：

某工程师设计了一个动态数组：当 `size == capacity` 时扩容为 2 倍；当 `size == capacity / 2` 时立即缩容为一半。请构造一个最坏操作序列，证明该策略下 `push_back` 和 `pop_back` 交替执行时的均摊复杂度会退化为 $O(n)$ ，并提出正确的缩容防抖设计。

<- 第 1 章 复杂度分析与 ADT | 课程目录 | 第 3 章 栈与队列 ->

所属模块：第二部分：线性结构
对应实现：Linked list.cpp

本章导航

- 2.1 线性表的本质与存储哲学：顺序 vs 链式
 - 2.1.1 顺序表（数组实现）与容量扩充策略
 - 2.1.2 现代硬件视角：CPU 缓存局部性（Cache Locality）的决定性影响
- 2.2 单向链表（Singly Linked List）与操作手算演练
 - 2.2.1 节点定义与核心操作复杂度
 - 2.2.2 经典三指针原地反转算法详解（Dry-Run 演练）
- 2.3 双向链表（Doubly Linked List）与空间换时间
- 2.4 循环链表与约瑟夫环（Josephus Problem）数学推导
- 2.5 链表四大工程杀手锏
 - 2.5.1 哨兵节点（Dummy Node）：彻底消灭头尾特判
 - 2.5.2 快慢指针（Floyd 判环与环入口严格数学证明）
 - 2.5.3 寻找链表中点与归并排序应用
 - 2.5.4 倒数第 k 个节点的双指针滑动窗口
- 本章小结
- 思考题与经典面试题

第 2 章 线性表

难度: ★☆☆☆☆ | 代码: [Linked list.cpp](#)

前置知识: 第 1 章 复杂度分析 (理解 $O(1)$ 与 $O(n)$ 的本质区别, 以及均摊分析思想)。

学习目标: 1. 透彻理解顺序存储 (物理连续) 与链式存储 (物理离散) 的设计权衡; 2. 掌握单链表、双向链表、循环链表的基本增删改查及内存安全管理; 3. 熟练运用“三指针原地翻转”、“哨兵节点”、“快慢指针”解决高频链表问题; 4. 掌握 Floyd 判环算法及环入口定位的严格代数证明。

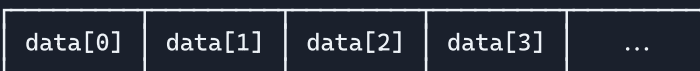
2.1 线性表的本质与存储哲学：顺序 vs 链式

线性表 (Linear List) 是最基础、最直观的逻辑结构：存在唯一的首元素和尾元素，除首尾外，每个元素有且仅有一个直接前驱 (Predecessor) 和一个直接后继 (Successor)。

然而，同一逻辑结构在物理内存中有两种截然相反的实现哲学：

【顺序表 (Sequential List)】

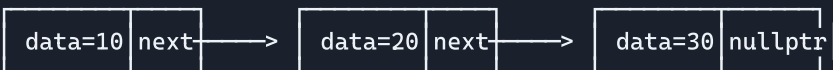
内存布局：紧密相连的一整块连续物理地址



寻址公式： $\text{Loc}(i) = \text{BaseAddress} + i * \text{sizeof}(\text{ElemType})$ $\longrightarrow O(1)$ 随机访问

【链表 (Linked List)】

内存布局：散落在内存堆各处的独立节点，通过指针串联



寻址方式：必须从 head 开始沿 next 指针逐个跳跃 $\longrightarrow O(n)$ 顺序访问

2.1.1 顺序表 (数组实现) 与容量扩充策略

顺序表的核心结构包含三个要素：

```
template <typename T>
class SeqList {
    T* data;           // 指向堆内存中连续数组的指针
    size_t length;     // 当前实际存储的元素个数 (逻辑长度)
    size_t capacity;   // 当前分配的物理内存容量 (物理容量)
};
```

为什么扩容因子 (Growth Factor) 通常选 1.5 或 2? - 若每次仅增加固定大小 (如每次 +10)，插入 (n) 个元素将触发 $O(n)$ 次内存重分配与数据搬移，总代价退化为 $O(n^2)$ ，均摊单次插入为 $O(n)$ 。- 若采用**几何级数扩容 (如 2 倍)**，根据第 1 章的均摊分析，均摊插入代价为 $O(1)$ 。- **工程细节**：MSVC STL 采用 1.5 倍扩容，GCC libstdc++ 采用 2 倍扩容。1.5 倍的优势在于允许之前释放的内存块在后续扩容中被重新复用，减少内存碎片。

2.1.2 现代硬件视角：CPU 缓存局部性（Cache Locality）的决定性影响

在 1980 年代，内存速度与 CPU 主频相当，链表因其“插入删除不搬移数据”而广受推崇。但今天，CPU 访问寄存器/L1 Cache (~1ns) 与访问主存 (~50-100ns) 存在百倍延迟鸿沟！

- **顺序表**：物理空间连续，CPU 抓取一个元素时，硬件预取器（Prefetcher）会自动将相邻整行（Cache Line，通常 64 字节）载入高速缓存，**空间局部性极高**。
- **链表**：节点由 `new` 动态分配，内存地址随机分布在堆中，每次访问 `curr→next` 极易引发 **Cache Miss（缓存未命中）**，导致流水线停顿等待内存读取。

工业界实战铁律：

除非有极其高频的已知位置插入/删除且绝对无法承受搬移开销，否则在大多数现代业务场景中，**优先选用 `std::vector` 而非 `std::list`**。即使是中途插入删除，在元素尺寸较小时，`std::vector` 的内存搬移速度往往依然快于链表指针寻址的开销！

2.2 单向链表（Singly Linked List）与操作手算演练

2.2.1 节点定义与核心操作复杂度

```
struct ListNode {
    int data;
    ListNode* next;
    ListNode(int val) : data(val), next(nullptr) {}
};
```

操作	时间复杂度	关键注意点
头部插入	$O(1)$	<code>newNode→next = head; head = newNode;</code> 顺序不可颠倒
尾部插入	$O(n)$	需遍历至尾节点；若维护了 <code>tail</code> 指针则优化为 $O(1)$
按值查找	$O(n)$	顺序比对，无法二分（不支持随机下标寻址）
删除给定节点	$O(n)$	单链表需从头遍历找到其 直接前驱节点 才能断开链接

2.2.2 经典三指针原地反转算法详解（Dry-Run 演练）

反转链表是所有数据结构面试的试金石。最优雅且空间为 $O(1)$ 的方法是**迭代三指针法**：`prev`、`curr`、`next`。

算法核心不变量： - `prev` 始终指向已经成功反转好的链表新头； - `curr` 是当前正准备反转指向的节点； - `next` 用于暂存 `curr` 原有的后继，防止断链后“丢失后续世界”。

```

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr != nullptr) {
        ListNode* next = curr->next; // 1. 暂存后继
        curr->next = prev;           // 2. 翻转当前节点的指针
        prev = curr;                 // 3. prev 前进
        curr = next;                 // 4. curr 前进
    }
    return prev; // prev 成为新头
}

```

分步跟踪演练 (Dry-Run Trace)

假设输入链表为: **1 → 2 → 3 → nullptr**

【初始状态】

prev = nullptr, curr = [1]
 [1] → [2] → [3] → nullptr

【第 1 轮循环】

1. next 暂存 [2]
 2. curr->next = prev ⇒ [1] → nullptr
 3. prev = curr ⇒ prev = [1]
 4. curr = next ⇒ curr = [2]
 状态: nullptr ← [1] (prev) [2] (curr) → [3] → nullptr

【第 2 轮循环】

1. next 暂存 [3]
 2. curr->next = prev ⇒ [2] → [1]
 3. prev = curr ⇒ prev = [2]
 4. curr = next ⇒ curr = [3]
 状态: nullptr ← [1] ← [2] (prev) [3] (curr) → nullptr

【第 3 轮循环】

1. next 暂存 nullptr
 2. curr->next = prev ⇒ [3] → [2]
 3. prev = curr ⇒ prev = [3]
 4. curr = next ⇒ curr = nullptr (循环终止)
 状态: nullptr ← [1] ← [2] ← [3] (prev)

【最终返回】

返回 prev ([3]), 反转彻底完成!

2.3 双向链表 (Doubly Linked List) 与空间换时间

在单链表中, 如果要删除一个节点 **p**, 必须知道它的前驱。若只持有 **p** 本身的指针, 必须花费 $O(n)$ 从头遍历。

双向链表通过每个节点增加一个 **prev** 指针, 实现了前驱与后继的对称访问。

```

nullptr ← [prev|10|next] ⇔ [prev|20|next] ⇔ [prev|30|next] → nullptr

```

(O(1)) 删除指定节点:

```
void removeNode(DNode* node) {
    if (!node) return;
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
    delete node;
}
```

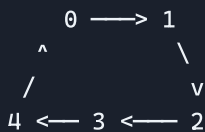
代价: 每个节点多耗费一个指针空间 (在 64 位系统上增加 8 字节开销), 典型的**以空间换时间**策略。广泛应用于操作系统的双向循环任务队列、LRU 缓存机制等。

2.4 循环链表与约瑟夫环 (Josephus Problem) 数学推导

循环链表 (Circular Linked List) 的尾节点不指向 `nullptr`, 而是重新指向 `head`。

经典案例: 约瑟夫问题 (Josephus Problem)

(n) 个人围成一圈 (编号 0 到 (n-1)), 从第 0 个人开始报数, 数到第 (m) 的人出局, 下一个人重新从 1 报数, 求最后幸存者的编号。



1. **模拟法 (循环链表)**: 时间复杂度 $O(n \cdot m)$, 空间复杂度 $O(n)$ 。
2. **数学递推法 (严格推导)**:
设 $J(n, m)$ 为 (n) 个人报数到 (m) 出局的最终胜出者编号。
3. 第一轮出局的人编号为 $((m - 1) \bmod n)$;
4. 剩下 $(n - 1)$ 个人从第 $(m \bmod n)$ 个人开始重新形成新一轮编号;
5. 新旧编号存在严格的映射位移关系: $J(n, m) = (J(n - 1, m) + m) \bmod n$ 基础条件: $J(1, m) = 0$ 。
由此, 可在 $O(n)$ 时间、 $O(1)$ 辅助空间内直接算得幸存者!

2.5 链表四大工程杀手锏

2.5.1 哨兵节点 (Dummy Node): 彻底消灭头尾特判

在操作链表时, 最容易出现的 Bug 就是对“空链表”或“在头部插入/删除”进行繁琐的 `if (head == nullptr)` 特判。

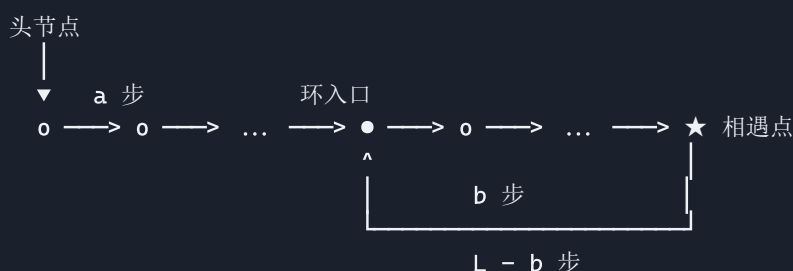
哨兵节点技巧: 在堆上或栈上分配一个虚拟头节点 `dummy`, 令 `dummy.next = head`。此时, 真正的头节点也有了统一的直接前驱 `dummy`!

```
// 示例：删除链表中所有值为 val 的节点（若无哨兵，头节点恰为 val 时处理极易出错）
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* curr = &dummy;

    while (curr->next != nullptr) {
        if (curr->next->data == val) {
            ListNode* del = curr->next;
            curr->next = del->next;
            delete del;
        } else {
            curr = curr->next;
        }
    }
    return dummy.next;
}
```

2.5.2 快慢指针（Floyd 判环与环入口严格数学证明）

算法流程：- 定义快慢两指针：**slow** 每次走 1 步，**fast** 每次走 2 步；- 若链表无环，**fast** 必先到达 **nullptr**；- 若链表有环，**fast** 与 **slow** 必然在环内相遇。



环入口定位的严格代数推导：

- 设链表头到环入口的距离为 (a)；
- 设环入口顺时针到相遇点的距离为 (b)；
- 设环的总周长为 (L)。
- 当两指针在 ★ 相遇时：
- **slow** 走过的路程：($S = a + b$)；
- **fast** 走过的路程：($F = a + b + k \cdot L$) ($k \geq 1$)，表示快指针在环内多转了 (k) 圈；
- 因为 **fast** 的速度是 **slow** 的 2 倍，故路程关系满足：[$F = 2S \implies a + b + k \cdot L = 2(a + b) \implies a = k \cdot L - b = (k - 1)L + (L - b)$]
- 物理含义：
从链表头走 (a) 步到达环入口的距离，**恰好等于** 从相遇点沿环继续走 ((L - b)) 步再加上若干整圈的距离！
- 操作结论：
两指针相遇后，将 **slow** 重新指向链表头，**fast** 停留在相遇点；此后两者均每次走 1 步。当它们再次相遇时，**所处位置必定是环的入口节点！**

2.5.3 寻找链表中点与归并排序应用

- `slow` 每次走 1 步, `fast` 每次走 2 步;
- 当 `fast->next == nullptr` (奇数长度) 或 `fast->next->next == nullptr` (偶数长度) 时, `slow` 恰好停留在中点。
- 这是在 $O(n \log n)$ 时间内对单链表进行**归并排序 (Merge Sort)** 的关键子过程。

本章小结

1. **结构决定行为**: 顺序表长于随机检索 ($O(1)$), 劣于动态插入; 链表长于已知位置的增删 ($O(1)$), 劣于查找 ($O(n)$)。
2. **硬件改变世界**: 现代 CPU 缓存机制让顺序表 (`std::vector`) 在绝大多数普通应用场景下综合性能大幅胜过链表。
3. **指针操作的黄金守则**: 改链之前, **先存后继**, 再接新链; 释放内存防止野指针与内存泄漏。
4. **哨兵与双指针**: 哨兵节点解决的是**代码边界的繁琐特判**; 快慢双指针解决的是**无额外空间下的拓扑特征探测 (环、中点、倒数第 k)**。

思考题与经典面试题

1. **K 个一组翻转链表**: 给定一个单链表, 每 (k) 个节点一组进行翻转, 如果最后剩余节点不足 (k) 个则保持原样。请使用迭代法实现, 并分析其时间与空间复杂度。
2. **双指针相遇圈数证明**: 在 Floyd 判环算法中, 证明当 `slow` 进入环后, 在它走完第一圈 ($\leq L$) 步之内, `fast` 必定会追上它。
3. **$O(1)$ 时间删除无先驱节点**: 如果只给你一个指向链表中间某个节点 `p` 的指针 (非尾节点), 且不知道头节点指针, 如何以 $O(1)$ 时间复杂度“删除”该节点? 这种技巧有什么潜在隐患?

<- 第 2 章 线性表 | 课程目录 | 第 4 章 字符串匹配 (KMP) ->

所属模块: 第二部分: 线性结构
对应实现: linear_structures.cpp

本章导航

- 3.1 操作受限的线性表: 为什么需要自我设限?
- 3.2 栈 (Stack) — 后进先出 (LIFO) 原理与实现
 - 3.2.1 顺序栈与链式栈的实现对比
 - 3.2.2 经典应用一: 括号匹配与调用栈 (Call Stack)
 - 3.2.3 经典应用二: 表达式转换 (中缀转后缀) 与逆波兰求值引擎
 - 3.2.4 变体设计: $O(1)$ 时间的最小栈 (MinStack) 两种方案

- 3.3 队列 (Queue) — 先进先出 (FIFO) 原理与实现
- 3.3.1 假溢出问题与循环队列 (Circular Queue) 的诞生
- 3.3.2 循环队列判空判满的三种经典设计方案
- 3.4 双栈模拟队列与均摊 $O(1)$ 势能证明
- 3.5 进阶利器：单调栈与单调队列实战模型
- 3.5.1 单调栈 (Monotonic Stack)：解决“下一个更大元素”
- 3.5.2 单调队列 (Monotonic Queue)：滑动窗口最值问题
- 本章小结
- 思考题与经典面试题

第 3 章 栈与队列

难度：★☆☆☆☆ | 代码：linear_structures.cpp

前置知识：第 1 章 均摊分析，第 2 章 线性表的顺序与链式存储。

学习目标：1. 理解栈 (LIFO) 与队列 (FIFO) 的受限逻辑契约及其在计算机系统核心地位；2. 掌握循环队列解决“假溢出”的数学原理及三种判空判满设计；3. 掌握逆波兰表达式转换与计算的完整状态机；4. 熟练运用单调栈与单调队列模型解决区间最值与极值覆盖问题。

3.1 操作受限的线性表：为什么需要自我设限？

初学者常常产生疑问：“既然动态数组和双向链表可以在任意位置插入和删除，为什么还要故意阉割功能，搞出只能在端点操作的栈和队列？”

在软件工程中，“能力的约束”往往意味着“确定性与安全性的提升”：

- 1. **防止误操作**：如果允许在调用栈中间随意插入或删除帧，整个程序的控制流将陷入不可预知的崩溃；
- 2. **算法表达更纯粹**：将操作限制为 `push/pop` 或 `enqueue/dequeue`，能够将问题的逻辑状态与底层复杂的内存搬移彻底隔离开来；
- 3. **硬件与系统友善**：CPU 内部的指令寄存器 (ESP/RSP) 天然以栈的形式工作，操作系统的进程调度与网络数据包缓冲天然以队列的形式流转。

3.2 栈 (Stack) — 后进先出 (LIFO) 原理与实现

栈 (Stack) 是限定仅在表的一端 (栈顶, Top) 进行插入和删除的线性表。另一端固定不动，称为栈底 (Bottom)。

```
栈顶 (Top)  → [ 30 ]  ▲ 只能在此端 push / pop
                [ 20 ]  |
栈底 (Bottom) → [ 10 ]  ▼
```

3.2.1 顺序栈与链式栈的实现对比

- 顺序栈 (Array Stack) : 以连续数组为底层, 维护 `int top` 索引 (初始为 -1) 。
- `push(x)` : `data[++top] = x;` ((O(1)))
- `pop()` : `--top;` ((O(1)))
- 优势: 极度缓存友好, 常数因子极小。
- 链式栈 (Linked Stack) : 以单链表为底层, 链表头作为栈顶。
- 为什么不能以链表尾作为栈顶? 因为单链表在尾部删除时需要 O(n) 寻找前驱, 除非使用双向链表。

3.2.2 经典应用一: 括号匹配与调用栈 (Call Stack)

编译器进行语法分析时, 必须检查各类括号 (`()`, `[]`, `{}`) 是否合法嵌套。 - 核心逻辑: 遇到左括号入栈; 遇到右括号, 检查栈顶是否为其对应的左括号。若匹配则弹出, 若不匹配或栈空则语法错误; 遍历结束后栈必须为空。

3.2.3 经典应用二: 表达式转换 (中缀转后缀) 与逆波兰求值引擎

人习惯阅读中缀表达式 (如 `3 + 4 * 2 / (1 - 5)`) , 但计算机难以直接处理括号嵌套与算符优先级。逆波兰表达式 (后缀表达式, Reverse Polish Notation, RPN) 将操作符置于操作数之后 (`3 4 2 * 1 5 - / +`) , 彻底消除了括号, 只需一个栈即可线性求值!

调度场算法 (Shunting-Yard Algorithm, Dijkstra 提出)

输入字符流 —> [遇到数字] —————> 直接输出到后缀队列

————> [遇到 '('] —————> 压入运算符栈

————> [遇到 ')'] —————> 弹出算符并输出, 直到遇到 '(' (弹出并丢弃 '(')

————> [遇到运算符 op] —————> 只要栈顶算符优先级 >= op, 就持续弹出并输出; 然后 op 入

后缀表达式求值状态机 (Dry-Run Trace)

计算 "3 4 2 * 1 5 - / +":

读取符号	操作类型	栈内元素状态	说明
3	数字入栈	[3]	
4	数字入栈	[3, 4]	
2	数字入栈	[3, 4, 2]	
*	弹出 2, 4 计算 (4 \times 2 = 8)	[3, 8]	先弹出的为右操作数!
1	数字入栈	[3, 8, 1]	
5	数字入栈	[3, 8, 1, 5]	
-	弹出 5, 1 计算 (1 - 5 = -4)	[3, 8, -4]	
/	弹出 -4, 8 计算 (8 / (-4) = -2)	[3, -2]	
+	弹出 -2, 3 计算 (3 + (-2) = 1)	[1]	最终计算结果为 1

3.2.4 变体设计：O(1) 时间的最小栈 (MinStack) 两种方案

设计一个栈，支持 `push`、`pop`、`top`，并能在 $O(1)$ 时间内检索到栈中的最小元素 `getMin()`。

- **方案一：双栈同步法（经典直观）**

维护主栈 `dataStack` 和辅助栈 `minStack`。每次 `push(x)` 时，`minStack` 压入 `min(x, minStack.top())`。两栈高度时刻相同，空间额外占用 $O(n)$ 。

- **方案二：差值存储法（进阶空间优化）**

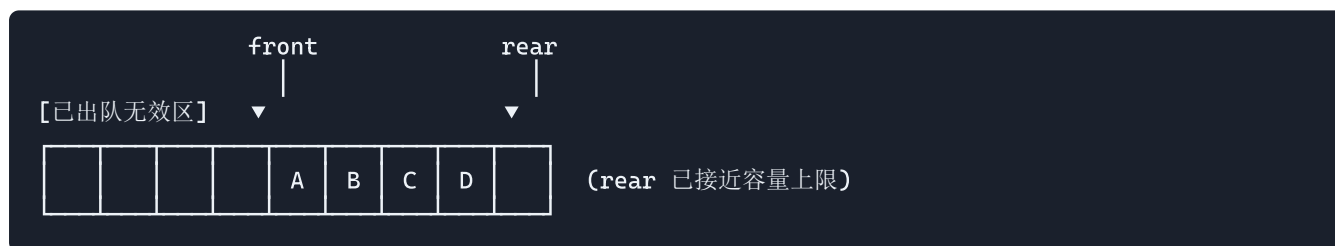
主栈不存原值，而是存入元素与当前最小值的差值：`diff = val - currentMin`。当 `diff < 0` 时，说明当前值刷新了最小值。此法在某些场景下可将空间开销压缩到极致，但需要考虑整型溢出问题。

3.3 队列 (Queue) — 先进先出 (FIFO) 原理与实现

队列 (Queue) 仅允许在表的一端插入（入队，Enqueue），在另一端删除（出队，Dequeue）。

3.3.1 假溢出问题与循环队列 (Circular Queue) 的诞生

若使用普通顺序数组实现队列，设置 `front` 指向队首，`rear` 指向队尾。随着不断出队和入队，`front` 和 `rear` 整体向数组高端漂移：



当 `rear == capacity` 时，即使数组前端有大量空闲位置，也无法再入队新元素，这种现象称为**假溢出 (False Overflow)**。

为了解决假溢出，数学上通过**取模运算 (Modulo Operation)** 将物理线性数组首尾相连，逻辑上视为一个环形缓冲区：

```
rear = (rear + 1) % capacity;
front = (front + 1) % capacity;
```

3.3.2 循环队列判空判满的三种经典设计方案

当 `front == rear` 时，队列到底是**空**还是**满**？这是数据结构考研与工业设计的核心经典命题。

【方案 1：牺牲一个存储单元（最常用标准方案）】

- 队空条件：front == rear
- 队满条件：(rear + 1) % capacity == front
- 队列实际可容纳最大元素数：capacity - 1

【方案 2：维护 size 计数器】

- 结构体增加字段：size_t size = 0;
- 队空条件：size == 0
- 队满条件：size == capacity
- 空间完全利用，无需牺牲存储单元。

【方案 3：维护 tag / last_op 标记位】

- 结构体增加布尔标记：bool tag; (最后一次操作：0 为出队，1 为入队)
- 当 front == rear 时：
 - 若 tag == 0，说明刚发生过出队，此时为【队空】；
 - 若 tag == 1，说明刚发生过入队，此时为【队满】。

3.4 双栈模拟队列与均摊 $O(1)$ 势能证明

如何用两个后进先出的栈（stackIn 与 stackOut）拼装出一个先进先出的队列？

入队元素：1, 2, 3
[stackIn] $\xrightarrow{\text{-(倒入)}}$ [stackOut] $\rightarrow$ 出队

3	1
2	2
1	3

操作法则： - 入队 (push)：直接压入 stackIn，时间复杂度 $O(1)$ ； - 出队 (pop/front)：- 若 stackOut 不为空，直接从 stackOut 弹出； - 若 stackOut 为空，将 stackIn 的所有元素一次性全部倒入 stackOut，再从 stackOut 弹出。

均摊复杂度分析： 每个元素在其生命周期内：1. 压入 stackIn 一次（开销 1）；2. 从 stackIn 弹出并压入 stackOut 一次（开销 2）；3. 从 stackOut 弹出一次（开销 1）。

任意元素最多被移动 4 次基本操作。连续 (n) 次操作的最坏总时间为 $O(n)$ ，均摊单次操作时间严格为 $O(1)$ ！

3.5 进阶利器：单调栈与单调队列实战模型

3.5.1 单调栈 (Monotonic Stack)：解决“下一个更大元素”

定义： 栈内元素自底向上严格保持单调递增或递减。

应用场景： 在 $O(n)$ 线性时间内找到每个元素左边/右边第一个比它大/小的元素。

```
// 模板：寻找数组中每个元素右侧第一个比它大的元素（若无则返回 -1）
std::vector<int> nextGreaterElement(const std::vector<int>& nums) {
    int n = nums.size();
    std::vector<int> result(n, -1);
    std::stack<int> st; // 存储元素下标，栈内对应的值保持单调递减

    for (int i = 0; i < n; ++i) {
        while (!st.empty() && nums[i] > nums[st.top()]) {
            int prevIdx = st.top();
            st.pop();
            result[prevIdx] = nums[i]; // nums[i] 即为 nums[prevIdx] 右侧首个更大值
        }
        st.push(i);
    }
    return result;
}
```

3.5.2 单调队列（Monotonic Queue）：滑动窗口最值问题

问题：大小为 (k) 的窗口在长度为 (n) 的数组上滑动，实时输出当前窗口内的最大值。若每次在窗口内遍历，复杂度为 (O(nk))；利用**双端队列**（`std::deque`）构建**单调递减队列**，可优化至 (O(n))。

核心不变量： - 队列中仅保存可能成为未来窗口最大值的候选元素下标； - 队列内元素值严格单调递减（队头永远是当前窗口最大值）； - 新元素入队前，从队尾剔除所有小于等于它的元素（“如果一个选手比你年轻还比你强，你就可以退役了”）。

本章小结

1. **受限带来确定性：**栈与队列舍弃了随机访问的自由，换取了严格受控的 LIFO/FIFO 语义，是程序控制流与任务调度的心脏。
2. **循环取模化腐朽为神奇：**循环队列巧妙化解了假溢出，三种判空判满方案是计算机体系中缓冲区设计的经典范式。
3. **调度场算法的精髓：**用算符栈管理优先级，将人类习惯的括号嵌套结构扁平化为计算机直接线序吞吐的逆波兰式。
4. **单调性消除无效比较：**单调栈与单调队列通过及时剔除“不可能再成为最优解的冗余元素”，将高阶比较问题线性化降维至 (O(n))。

思考题与经典面试题

1. **柱状图中最大的矩形：**给定 (n) 个非负整数表示柱状图中各个柱子的高度，每个柱子宽度为 1，求能在柱状图中勾勒出的矩形的最大面积。请使用单调栈给出 (O(n)) 解法并详细追踪状态。
2. **双端队列（Deque）的底层架构：**C++ 标准库中的 `std::deque` 既不是单纯的连续数组，也不是双向链表，而是采用“中央映射控制表 + 固定大小缓冲区（Block）”的分段连续架构。请分析这种设计在内存管理与下标随机访问上的权衡考量。

3. **单调队列的过期清除时机**：在滑动窗口最值问题中，如何准确判断并从单调队列头部剔除已经滑出窗口范围的过期元素？

<- 第 3 章 栈与队列 | 课程目录 | 第 5 章 二叉树基础与遍历 ->

所属模块：第三部分：串

对应实现：string_match.cpp

本章导航

- 4.1 字符串匹配问题的本质与暴力匹配的缺陷
- 4.2 KMP 算法的哲学飞跃：为什么主串指针绝不回退？
- 4.3 前缀函数 (Prefix Function) 与最长相等真前后缀
 - 4.3.1 数学定义与几何对称直观
 - 4.3.2 两种主流下标体系对比 (算法导论 π 数组 vs 严蔚敏 next 数组)
- 4.4 线性时间递推求解前缀函数 ($O(m)$) 与均摊分析
- 4.5 KMP 匹配主循环与不变量证明 ($O(n)$)
- 4.6 完整分步手算追踪 (Step-by-step Dry Run)
- 4.7 失配优化：nextval 数组与消除连环失配
- 4.8 工业级单模式匹配算法演进 (Boyer-Moore 与 Sunday)
- 本章小结
- 思考题与经典大厂面试题

第 4 章 字符串匹配 (KMP)

难度：★★★★☆ | 代码：string_match.cpp

前置知识：第 1 章 均摊分析 (双指针回退的均摊常数级证明)，第 2 章 线性存储。

学习目标：1. 理解朴素字符串匹配在重复模式下退化为 $O(n \cdot m)$ 的深层根源；2. 领悟 KMP 算法“利用模式串自相似性，免除主串指针回溯”的核心思想；3. 掌握前缀函数 (π 数组 / next 数组) 的严格数学定义及线性递推构造法；4. 掌握 nextval 优化针对连续重复字符失配的修正机制；5. 能独立手算任何模式串的 next 与 nextval 数组。

4.1 字符串匹配问题的本质与暴力匹配的缺陷

问题定义：给定一个长度为 (n) 的主文本串 (T) 和一个长度为 (m) 的模式串 (P) ($(m \leq n)$)，在 (T) 中找出 (P) 首次或所有出现的起始下标。

朴素暴力匹配 (Brute-Force, BF)

最直观的解法是双重循环对齐：

```
int bruteForce(const string& T, const string& P) {
    int n = T.size(), m = P.size();
    for (int i = 0; i ≤ n - m; ++i) {
        int j = 0;
        while (j < m && T[i + j] == P[j]) {
            ++j;
        }
        if (j == m) return i; // 找到匹配
    }
    return -1;
}
```

```
主串 T:  A A A A A A A A B
模式串 P: A A A B
第 0 轮:  A A A [A] ≠ B (比较 4 次失配)
第 1 轮:    A A [A] A ≠ B (主串指针 i 回退到 1, 再次比对)
第 2 轮:      A [A] A A ≠ B
...
```

- **最坏时间复杂度：** $(O((n - m + 1) \cdot m) = O(n \cdot m))$ 。在自然语言中字符集大，退化不严重；但在 DNA 序列分析（仅有 A, C, G, T 四种字符）或长重复文本中，暴力匹配将极度缓慢！
- **缺陷本质：**第 0 轮已经知道了 $(T[0..2] == \text{"AAA"})$ ，但在第 1 轮中，算法竟然把已经探测过的信息全部抛弃，让主串指针硬生生倒退回第 1 个位置重新比较！

4.2 KMP 算法的哲学飞跃：为什么主串指针绝不回退？

1977 年，Knuth、Morris 和 Pratt 三位大师联合发表了 KMP 算法。其核心洞见只有一句话：

“主串指针 (i) 自始至终只进不退，所有回退与位移全由模式串指针 (j) 根据预处理信息独自承担！”

为什么可以不回退？

假设当前主串在 $(T[i])$ 与模式串 $(P[j])$ 发生失配：- 这意味着：在此之前，模式串的前 (j) 个字符 $(P[0..j-1])$ **已经与主串对应区间 $(T[i-j .. i-1])$ 取得了完全一致的匹配！** - 此时，主串区间的内容已经被模式串的前缀“完全透视”。 - 如果模式串自身有一段“前缀”和“后缀”长得一模一样（自相似性），我们只需将模式串向右滑动，让它的**前缀**直接对齐刚才已经匹配成功的**后缀**，主串指针 (i) 原地不动，继续比较下一个字符即可！

主串 T: ... [a b a b] c ...

模式串 P: [a b a b] a ...

(在 'c' 与 'a' 处失配, j = 4)

前缀

后缀

(最长相等前后缀为 "ab", 长度 k = 2)

滑动对齐: ... a b [a b] c ...

[a b] a ...

(模式串向右滑动, j 直接跳转到 2 继续比对 'c' 与 'a')

4.3 前缀函数 (Prefix Function) 与最长相等真前后缀

4.3.1 数学定义与几何对称直观

定义模式串 (P) 的子串 (P[0..i]) 的前缀函数 (pi[i]) 为:
子串 (P[0..i]) 的最长相等真前缀 (Proper Prefix) 与真后缀 (Proper Suffix) 的长度。 - “真”意味着: 前缀或后缀不能等于子串本身 (长度严格 (< i + 1)) 。

例如 P = "a b a b a"

i = 0: "a"

真前缀: ∅,

真后缀: ∅

⇒ π[0] = 0

i = 1: "a b"

真前缀: "a",

真后缀: "b"

⇒ π[1] = 0

i = 2: "a b a"

真前缀: "a", "ab",

真后缀: "a", "ba"

⇒ "a"=="a" ⇒ π[2] = 1

i = 3: "a b a b"

真前缀: ...

真后缀: ...

⇒ "ab"=="ab" ⇒ π[3] = 2

i = 4: "a b a b a"

真前缀: ...

真后缀: ...

⇒ "aba"=="aba" ⇒ π[4] = 3

4.3.2 两种主流下标体系对比 (算法导论 π 数组 vs 严蔚敏 next 数组)

国内很多考研和经典教材 (严蔚敏版) 习惯使用 next 数组, 而国际竞赛与标准算法 (CLRS) 使用 pi 数组。初学者极易混淆, 二者的映射规则如下:

模式串索引 (i)	0	1	2	3	4	5	6
字符 (P[i])	a	b	a	b	a	a	b
算法导论 / 力扣 (pi[i])	0	0	1	2	3	1	2
严蔚敏 0-indexed next[j]	-1	0	0	1	2	3	1
严蔚敏 1-indexed next[j]	0	1	1	2	3	4	2

本质映射:

严蔚敏版的 next[j] 表示“当在位置 (j) 失配时, (j) 应该跳转到哪个下标继续比较”。它相当于把 pi 数组整体右移一位, 最左端补 -1! 两者在数学实质上完全等价。

4.4 线性时间递推求解前缀函数 ($O(m)$) 与均摊分析

若对每个前缀暴力求解最长相等前后缀，整体需要 ($O(m^2)$)。KMP 的神来之笔在于：用模式串自身与自身匹配，动态规划地在线性 ($O(m)$) 内求出所有 ($\pi[i]$)。

递推推导：

假设已知 ($\pi[0..i-1]$)，现在要求 ($\pi[i]$)：- 令候选长度 ($j = \pi[i-1]$)，它代表了子串 ($P[0..i-1]$) 的最长相等前后缀；- 若 ($P[i] == P[j]$)，则新字符正好将相等前后缀延长了 1 位： $\pi[i] = j + 1$ ；- 若 ($P[i] \neq P[j]$)，说明长度为 ($j+1$) 的匹配宣告破裂。此时我们不能从 0 重新开始，而是退求其次，寻找“次长”的相等前后缀！次长的长度是多少？正是 ($\pi[j-1]$)！我们令 ($j = \pi[j-1]$)，继续检查 ($P[i]$) 是否等于 ($P[j]$)，直到相等或 ($j = 0$)。

```
std::vector<int> computePrefixFunction(const std::string& P) {
    int m = P.size();
    std::vector<int> pi(m, 0);
    int j = 0; // 当前最长匹配前缀的长度

    for (int i = 1; i < m; ++i) {
        // 当字符不匹配且还能向前缩短前缀时，回溯 j
        while (j > 0 && P[i] != P[j]) {
            j = pi[j - 1];
        }
        // 如果当前字符匹配成功，前缀长度加 1
        if (P[i] == P[j]) {
            ++j;
        }
        pi[i] = j;
    }
    return pi;
}
```

为什么复杂度是 ($O(m)$) 而非 ($O(m^2)$)？- 在 `for` 循环中，`j` 每次最多增加 1（通过 `++j`）；- `while` 循环中的 `j = pi[j - 1]` 每次至少让 `j` 减小 1；- 因为 `j` 始终非负，所以 `j` 减少的总次数不可能超过增加的总次数（最多 ($m-1$) 次）！- 故 `while` 内部操作的总体累计次数 ($\leq m-1$)，均摊到每轮循环为 ($O(1)$)，总时间严格为 ($O(m)$)！

4.5 KMP 匹配主循环与不变量证明 ($O(n)$)

有了 (π) 数组后，主串匹配的过程与构造前缀函数如出一辙：

```

int kmpSearch(const std::string& T, const std::string& P) {
    int n = T.size(), m = P.size();
    if (m == 0) return 0;
    std::vector<int> pi = computePrefixFunction(P);

    int j = 0; // 模式串当前匹配到的位置
    for (int i = 0; i < n; ++i) {
        while (j > 0 && T[i] != P[j]) {
            j = pi[j - 1]; // 主串 i 不变，模式串 j 回跳
        }
        if (T[i] == P[j]) {
            ++j;
        }
        if (j == m) {
            return i - m + 1; // 匹配成功，返回在主串中的起始下标
        }
    }
    return -1; // 未找到
}

```

4.6 完整分步手算追踪 (Step-by-step Dry Run)

示例： - 主串 ($T = \text{"ABABDABACDABABCABAB"}$) - 模式串 ($P = \text{"ABABCABAB"}$) - 经预处理，(P) 的 (π) 数组为: `[0, 0, 1, 2, 0, 1, 2, 3, 4]`

【第一阶段：前 4 个字符顺利匹配】

$i=0..3$: $T[0..3] == \text{"ABAB"}$, $P[0..3] == \text{"ABAB"}$, j 从 0 增长至 4

【第二阶段：遭遇失配与回跳】

$i = 4$: $T[4] = \text{'D'}$, $P[4] = \text{'C'}$ ——> 发生失配!

此时 $j = 4$, 查表 $j = \pi[4 - 1] = \pi[3] = 2$

模式串右移, j 变为 2。

再次比对: $T[4] (\text{'D'})$ 与 $P[2] (\text{'A'})$ ——> 依然失配!

查表 $j = \pi[2 - 1] = \pi[1] = 0$

模式串再次右移, j 变为 0。

再次比对: $T[4] (\text{'D'})$ 与 $P[0] (\text{'A'})$ ——> 依然失配!

j 已经为 0 且不匹配, **while** 退出, 本次循环结束。主串指针 i 自增进入 5。

【第三阶段：后续重新推进直至完全匹配】

在 $i = 10$ 之后匹配重新建立, 最终在 $i = 18$ 时 $j == 9 == m$, 成功定位起始位置 10!

4.7 失配优化: nextval 数组与消除连环失配

尽管经典 KMP 已经非常优秀, 但在面对**连续重复字符**时, 依然存在不必要的冗余跳转。

连环失配的尴尬：

假设 $P = \text{"AAAA"}$ ，计算其常规 `next` 数组（严蔚敏版）：- `next = [-1, 0, 1, 2]` - 若在 $P[3] = \text{'A'}$ 处与主串字符（比如 `'B'`）失配：1. 根据 `next[3] = 2`，跳转到 $P[2]$ 去比较；2. 但 $P[2]$ 也是 `'A'`！既然刚才已经知道 `'B' \neq \text{'A'}`，这次再比必定依然失败！3. 接着跳到 $P[1]$ ，还是 `'A'`，再次失败；4. 最后跳到 $P[0]$ ，依然是 `'A'`，再次失败。

`nextval` 修正算法：

若退回的目标位置字符与当前失配字符**完全相同**，则当前位置的跳转目标应当**继承**目标位置的 `nextval`：
$$\text{nextval}[i] = \begin{cases} \text{nextval}[\text{next}[i]], & \text{若 } P[i] == P[\text{next}[i]] \\ P[i], & \text{若 } P[i] \neq P[\text{next}[i]] \end{cases}$$

经过优化后， $P = \text{"AAAA"}$ 的 `nextval` 直接变为 `[-1, -1, -1, -1]`，失配时一步到位，彻底根除连环冗余！

4.8 工业级单模式匹配算法演进（Boyer-Moore 与 Sunday）

在标准库和工业界文本检索工具（如 GNU `grep`）中，最常用的往往不是 KMP，而是**从后向前匹配的 Boyer-Moore (BM) 算法**或**Sunday 算法**：
- **坏字符规则 (Bad Character)**：当末尾字符失配时，根据主串字符在模式串中的最后出现位置，一次性跳过数个乃至整整 m 个字符！
- **好后缀规则 (Good Suffix)**：与 KMP 的好前缀对称。
- **实际性能**：在长模式串和大规模文本搜索中，BM 算法平均时间复杂度可达 $O(n/m)$ （亚线性！），远超 KMP 的 $O(n)$ 。

本章小结

1. **KMP 的精髓在于利用已有的对称性**：主串指针不回退是效率的根本保障，模式串的前缀自相似性是信息复用的源泉。
2. **前缀函数是动态规划的杰作**：利用已求解的 $\pi[j-1]$ 替代穷举，实现了 $O(m)$ 时间的自匹配构造。
3. **均摊分析保证线性上界**：模式串指针的增加与减少次数严格平衡，杜绝了暴力算法的回溯爆炸。
4. **`nextval` 是极致的工程剪枝**：通过识别值等价，避免了连续相同字符上的连环无意义比较。

思考题与经典大厂面试题

1. **周期性字符串检测**：给定一个非空的字符串 s ，检查它是否可以通过由它的一个子串重复多次构成。请证明：若 s 具有长度为 k 的最小重复周期，则其前缀函数必满足 $\pi[n-1] > 0$ 且 $(n \bmod (n - \pi[n-1])) == 0$ 。
2. **手算演练**：对于模式串 $P = \text{"abacaba"}$ ，请手工写出其 π 数组、严蔚敏版 `next` 数组和 `nextval` 数组。
3. **最长回文前缀**：利用 KMP 的前缀函数，如何在 $O(n)$ 时间内找到字符串 s 的最长回文前缀？（提示：构造辅助串 $(s + \text{"\#"} + \text{reverse}(s))$ ）。

所属模块：第四部分：树与二叉树

对应实现：binary_tree.cpp

本章导航

- 5.1 从线性到分层：树结构的诞生与核心术语
- 5.2 二叉树的五大经典数学性质与严格证明
- 5.3 三大特殊二叉树形态辨析：满、完全、完美
- 5.4 二叉树的物理存储：链式 vs 数组索引映射
- 5.5 深度优先遍历 (DFS)：前序、中序、后序的递归本质
- 5.6 非递归遍历的显式栈统一模板与工程实现
- 5.6.1 前序与中序的单栈迭代模板
- 5.6.2 后序遍历单栈迭代法 (lastVisited 状态机)
- 5.7 广度优先遍历 (BFS)：层序遍历与快照分层
- 5.8 经典问题：二叉树的唯一重构算法与反例分析
- 本章小结
- 思考题与经典大厂面试题

第 5 章 二叉树基础与遍历

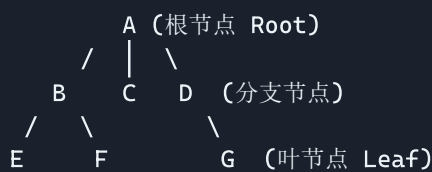
难度：★★☆☆☆ | 代码：binary_tree.cpp

前置知识：第 2 章 链表指针操作，第 3 章 栈 (DFS) 与队列 (BFS)。

学习目标： 1. 建立层次化拓扑结构思维，准确掌握树与二叉树的标准术语； 2. 掌握二叉树五大数学性质的推导（重点： $n_0 = n_2 + 1$ 的边数平衡法）； 3. 深刻理解递归遍历背后的系统调用栈行为； 4. 熟练手写前序、中序、后序的非递归（显式栈）与层序遍历代码； 5. 掌握通过遍历序列唯一重构二叉树的充要条件。

5.1 从线性到分层：树结构的诞生与核心术语

线性表表达的是“一对一”的前驱后继关系；然而自然界与计算机系统中充斥着大量的“一对多”层次关系（如文件目录系统、DOM 树、组织架构、决策分类树）。**树 (Tree)** 应运而生。



核心术语定义：

- **节点的度 (Degree)**：节点拥有的子树个数（如 (A) 的度为 3，(B) 的度为 2，(E) 的度为 0）；
- **树的度**：树中所有节点度的最大值（上图树的度为 3）；
- **叶节点 (Leaf) / 终端节点**：度为 0 的节点；
- **节点的层次 (Level)**：根为第 1 层，其子为第 2 层，以此类推；
- **节点的高度 (Height)**：从该节点到最远叶子节点的最长路径边数（叶节点高度为 0）；
- **节点的深度 (Depth)**：从根节点到该节点的唯一路径边数（根节点深度为 0）；
- **二叉树 (Binary Tree)**：每个节点至多有两棵子树（度 ≤ 2 ），且子树有严格的左右之分，次序不能颠倒。

5.2 二叉树的五大经典数学性质与严格证明

以下性质是二叉树理论的基石，也是算法分析与各类考试的必考考点：

性质 1：层节点数上界

在二叉树的第 (i) 层上至多有 (2^{i-1}) 个节点 $((i \geq 1))$ 。

证明：通过数学归纳法。第 1 层只有根节点 $((2^0 = 1))$ ；设第 $(i-1)$ 层最多有 (2^{i-2}) 个节点，因每个节点最多 2 个孩子，第 (i) 层最多有 $(2 \times 2^{i-2} = 2^{i-1})$ 个节点。

性质 2：树总节点数上界

深度为 (k) 的二叉树至多有 $(2^k - 1)$ 个节点 $((k \geq 1))$ 。

证明：利用等比数列求和：
$$\sum_{i=1}^k 2^{i-1} = \frac{1 \cdot (1 - 2^k)}{1 - 2} = 2^k - 1。$$

性质 3：叶子节点与度为 2 节点的关系（核心定理）

对任何非空二叉树，若叶子节点数为 (n_0) ，度为 2 的节点数为 (n_2) ，则必有： $(n_0 = n_2 + 1)$ 。

【边数平衡证明法（双向数边）】

设二叉树总节点数为 n ，度为 1 的节点数为 n_1 ，总边数为 E 。

1. 从下往上看：除根节点外，每个节点上方均有一条入边连接其父节点：

$$E = n - 1 = (n_0 + n_1 + n_2) - 1$$

2. 从上往下看：度为 2 的节点发出 2 条出边，度为 1 的发出 1 条，叶子发出 0 条：

$$E = 2 * n_2 + 1 * n_1 + 0 * n_0 = 2n_2 + n_1$$

3. 联立两式：

$$(n_0 + n_1 + n_2) - 1 = 2n_2 + n_1$$

两边消去 n_1 ，移项得：

$$n_0 - 1 = n_2 \implies n_0 = n_2 + 1 \quad \blacksquare \text{ (Q.E.D.)}$$

性质 4：完全二叉树的深度

具有 (n) 个节点的完全二叉树的深度为 $(\lfloor \log_2 n \rfloor + 1)$ 。

性质 5：完全二叉树的数组下标父子关系 (0-based)

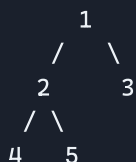
对于从 0 开始连续编号的 (n) 个节点：- 若 $(2i + 1 < n)$ ，则节点 (i) 的左孩子为 $(2i + 1)$ ；否则无左孩子；- 若 $(2i + 2 < n)$ ，则节点 (i) 的右孩子为 $(2i + 2)$ ；否则无右孩子；- 若 $(i > 0)$ ，则节点 (i) 的双亲节点为 $(\lfloor (i - 1) / 2 \rfloor)$ 。

5.3 三大特殊二叉树形态辨析：满、完全、完美

这三个概念在各大教材中极易混淆，其严格工业级界定如下：

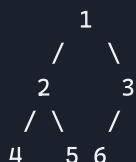
【满二叉树 (Full Binary Tree)】

每个节点度为 0 或 2 (无单分支)



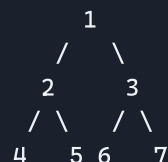
【完全二叉树 (Complete)】

除最底层外全满，底层节点紧靠左



【完美二叉树 (Perfect)】

所有分支节点均有两子，叶子全同层



5.4 二叉树的物理存储：链式 vs 数组索引映射

1. 链式二叉树 (二叉链表)

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
```

在通用场景下最为常用。若需要频繁寻找父节点，还可增加 `TreeNode* parent` 构成三叉链表。

2. 顺序存储（数组连续映射）

仅在**完全二叉树**（如堆、优先队列、线段树）中推荐使用。若用于普通二叉树，必须在缺失节点处填充占位符，面对倾斜树时会造成毁灭性的内存浪费（如深度为 k 的右单斜树仅有 k 个节点，却需分配 $(2^k - 1)$ 个空间！）。

5.5 深度优先遍历（DFS）：前序、中序、后序的递归本质

三种 DFS 遍历在代码结构上**完全一致**，唯一的差别在于**处理（访问）根节点的时机**：

```
void traverse(TreeNode* root) {
    if (!root) return;
    // 1. 在此访问 root→val ==> 前序遍历 (Preorder: Root → Left → Right)
    traverse(root→left);
    // 2. 在此访问 root→val ==> 中序遍历 (Inorder: Left → Root → Right)
    traverse(root→right);
    // 3. 在此访问 root→val ==> 后序遍历 (Postorder: Left → Right → Root)
}
```



前序遍历（NLR）：4 → 2 → 1 → 3 → 6 → 5 → 7（自顶向下，适合树的序列化与深拷贝）
中序遍历（LNR）：1 → 2 → 3 → 4 → 5 → 6 → 7（在 BST 中必然产生严格升序序列！）
后序遍历（LRN）：1 → 3 → 2 → 5 → 7 → 6 → 4（自底向上，适合由叶及根的计算与内存析构释放）

5.6 非递归遍历的显式栈统一模板与工程实现

递归虽美，但当树高达数万层时会引发操作系统栈溢出（Stack Overflow）。使用显式 `std::stack` 可将递归扁平化。

5.6.1 前序与中序的单栈迭代模板

前序与中序有着极其优美的一体化模板——“一路向左入栈，无路可走弹出转向右”：

```

// 中序遍历的统一非递归实现
std::vector<int> inorderTraversal(TreeNode* root) {
    std::vector<int> res;
    std::stack<TreeNode*> st;
    TreeNode* curr = root;

    while (curr != nullptr || !st.empty()) {
        // 1. 一路向左深入，沿途节点全部压栈
        while (curr != nullptr) {
            st.push(curr);
            curr = curr->left;
        }
        // 2. 走到最左端，弹出栈顶
        curr = st.top();
        st.pop();
        res.push_back(curr->val); // 访问节点（若为前序，则在第 1 步压栈前访问即可）

        // 3. 转向右子树
        curr = curr->right;
    }
    return res;
}

```

5.6.2 后序遍历单栈迭代法 (lastVisited 状态机)

后序遍历要求左、右子树都访问完毕后才能访问根节点。当从栈顶看到一个节点时，我们无法立即判断其右子树是否已经被访问过。

解决方案：维护一个 `lastVisited` 指针，记录上一个被弹出的节点。若 `curr->right == nullptr || curr->right == lastVisited`，说明右子树已处理完毕，当前节点方可出栈！

```

std::vector<int> postorderTraversal(TreeNode* root) {
    std::vector<int> res;
    std::stack<TreeNode*> st;
    TreeNode* curr = root;
    TreeNode* lastVisited = nullptr;

    while (curr != nullptr || !st.empty()) {
        while (curr != nullptr) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top();
        // 若右子树存在且尚未被访问过，则转向右子树
        if (curr->right != nullptr && curr->right != lastVisited) {
            curr = curr->right;
        } else {
            // 右子树为空或已被访问过，安全访问当前节点并弹出
            res.push_back(curr->val);
            lastVisited = curr;
            st.pop();
            curr = nullptr; // 必须置空，防止下轮重新向左下压入
        }
    }
    return res;
}

```

5.7 广度优先遍历（BFS）：层序遍历与快照分层

层序遍历借助队列实现逐层从左到右扫描。

“快照（Snapshot）分层技巧”：在处理每一层前，先记录 `int levelSize = q.size()`，一次性处理恰好 `levelSize` 个节点，这样可以严格区分层与层的边界。

```

std::vector<std::vector<int>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>> levels;
    if (!root) return levels;
    std::queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        int levelSize = q.size(); // 核心：快照当前层节点数
        std::vector<int> currentLevel;
        for (int i = 0; i < levelSize; ++i) {
            TreeNode* node = q.front();
            q.pop();
            currentLevel.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        levels.push_back(currentLevel);
    }
    return levels;
}

```

5.8 经典问题：二叉树的唯一重构算法与反例分析

定理：1. 前序 + 中序 $\implies$ 唯一确定 一棵二叉树； 2. 后序 + 中序 $\implies$ 唯一确定 一棵二叉树； 3. 前序 + 后序 $\implies$ 不能唯一确定 二叉树！

为什么前序+后序无法确定？

前序：[1, 2]
后序：[2, 1]

形态 A: 1 形态 B: 1
 / \
 2 2

在前序中 1 在 2 前，后序中 2 在 1 前。由于缺少中序遍历作为左右子树的“分割边界”，我们无法判断节点 2 究竟是 1 的左孩子还是右孩子！

本章小结

1. **二叉树的不变量**：($n_0 = n_2 + 1$) 是二叉树最重要的拓扑特征定理，其证明深刻揭示了入度与出度的图论守恒。
2. **DFS 的时机决定语义**：前序重在“分发与自顶向下”，中序在 BST 中保证“单调升序”，后序重在“汇聚与自底向上释放”。
3. **显式栈与状态追踪**：非递归后序遍历引入 `lastVisited` 是用状态机管理回溯边界的经典实践。
4. **层序快照**：`levelSize = q.size()` 巧妙将连续流式的队列拆解为清晰的二维层次切片。

思考题与经典大厂面试题

1. **Morris 遍历（神级算法）**：如何在 $O(n)$ 时间复杂度、且仅占用 $O(1)$ 辅助空间（不使用栈，不破坏树结构）的前提下完成二叉树的中序遍历？（提示：利用叶子节点的空闲右指针建立线索）。
2. **二叉树最近公共祖先（LCA）**：给定一棵二叉树和其中的两个节点 (p, q)，如何使用后序遍历自底向上在 $O(n)$ 时间内找到它们的最近公共祖先？
3. **完全二叉树的节点个数统计**：对于一棵完全二叉树，如何利用其高度特性，在 $O(\log^2 n)$ 时间内（优于 $O(n)$ 暴力遍历）求出总节点个数？

<- 第 5 章 二叉树基础与遍历 | 课程目录 | 第 7 章 堆与优先队列 ->

所属模块：第四部分：树与二叉树
对应实现：binary_tree.cpp

本章导航

- 6.1 二叉搜索树 (BST) 的定义与全局不变量
- 6.2 BST 基础操作：查找、插入与前驱/后继定位
 - 6.2.1 递归与迭代查找算法
 - 6.2.2 插入操作与父子链接挂载不变量
 - 6.2.3 前驱 (Predecessor) 与后继 (Successor) 的定位原理
- 6.3 核心难点：BST 节点删除的三种情况与图解
- 6.4 顺序倾斜与退化危机：为什么需要平衡树？
- 6.5 BST 合法性验证的两种经典算法 (中序遍历 vs 区间收缩)
- 6.6 进阶实战：顺序统计树 (Order Statistic Tree) 查第 k 小
- 本章小结
- 思考题与经典大厂面试题

第 6 章 二叉搜索树

难度：★★☆☆☆ | 代码：binary_tree.cpp BST 部分

前置知识：第 5 章 二叉树遍历 (中序遍历在 BST 下为有序序列)，第 1 章 树高相关的渐进复杂度。

学习目标：1. 理解 BST 的全局不变量，彻底消除“只比对直接孩子”的局部性误区；2. 掌握查找、插入、前驱/后继查找的指针逻辑；3. 掌握删除节点的三大分支，透彻理解为什么用后继替换不会破坏 BST 性质；4. 掌握利用子树规模 (size) 增强二叉树实现 ($O(\log n)$) 的 Rank 查询。

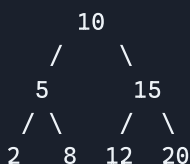
6.1 二叉搜索树 (BST) 的定义与全局不变量

二叉搜索树 (Binary Search Tree, BST)，又称二叉查找树或二叉排序树。其核心定义如下：

BST 全局不变量：

设 (x) 为树中的任意节点。

- 若 (y) 是 (x) **左子树** 中的任意节点，则必有 $(y.\text{key} < x.\text{key})$ ；
- 若 (z) 是 (x) **右子树** 中的任意节点，则必有 $(z.\text{key} > x.\text{key})$ 。



⚠️ 高频致命认知陷阱:

初学者常误以为性质只是“左孩子 < 根 < 右孩子”。

反例: 如果 15 的左孩子是 12, 而 12 的右孩子是 11 ((11 > 10) 成立, (11 < 12) 错误; 但如果 5 的右孩子是 8, 8 的右孩子是 11, 局部看 (8 < 11), 但 11 处于 10 的左子树中, 破坏了全局不变量!)。

数学推论:

对任何 BST 进行中序遍历 (LNR), 得到的必定是严格单调递增的序列!

6.2 BST 基础操作: 查找、插入与前驱/后继定位

6.2.1 递归与迭代查找算法

查找过程类似于在树形结构上执行“二分查找”: - 从根节点出发, 若目标值等于当前节点, 查找成功; - 若目标值小于当前节点, 转向左子树; - 若目标值大于当前节点, 转向右子树; - 遇到 `nullptr` 则说明目标不存在。

```
// 迭代版本: 空间 O(1), 无函数调用栈开销
TreeNode* searchBST(TreeNode* root, int val) {
    TreeNode* curr = root;
    while (curr != nullptr && curr->val != val) {
        if (val < curr->val) curr = curr->left;
        else curr = curr->right;
    }
    return curr;
}
```

6.2.2 插入操作与父子链接挂载不变量

新元素在 BST 中的插入位置永远是某个叶子节点的空指针域。

```
TreeNode* insertBST(TreeNode* node, int val) {
    if (node == nullptr) return new TreeNode(val);
    if (val < node->val) {
        node->left = insertBST(node->left, val);
    } else if (val > node->val) {
        node->right = insertBST(node->right, val);
    }
    return node; // 返回当前节点以维护父子链接
}
```

关键代码细节:

必须使用 `node->left = insertBST(...)` 接收递归返回值! 当到达 `nullptr` 实例化新节点时, 返回值会直接赋给其父节点的子指针。如果不接收返回值, 新节点虽被 `new` 出来, 但与整棵树处于断连状态!

6.2.3 前驱 (Predecessor) 与后继 (Successor) 的定位原理

在中序遍历序列中，排在节点 (u) 前面的称为其**直接前驱**，排在后面的称为其**直接后继**。

- **节点 (u) 拥有右子树：**
其后继必定是**右子树中的最小值**（从 `u→right` 出发，一路沿 `left` 指针走到尽头）。
- **节点 (u) 没有右子树：**
其后继必定是**某个祖先节点**——沿父指针向上回溯，直到找到第一个“以当前子树作为其左孩子”的祖先！

6.3 核心难点：BST 节点删除的三种情况与图解

删除是 BST 所有操作中逻辑最精妙的部分。设要删除的节点为 (Z)：

情况 1：(Z) 为叶子节点（无左右孩子）

直接释放 (Z) 的内存，并令其父节点指向它的指针置为 `nullptr`。

情况 2：(Z) 仅有一个孩子（只有左孩子或只有右孩子）

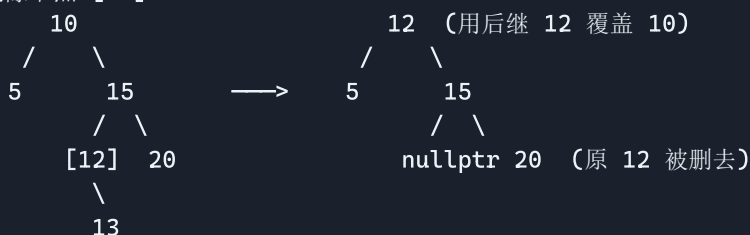
让 (Z) 的唯一孩子直接取代 (Z) 的位置（其父节点跨过 (Z)，直接与孙子相连），然后释放 (Z)。

情况 3：(Z) 同时拥有两个孩子（左右子树均非空）

不能直接删除 (Z)，否则整棵树将分裂为互不相连的碎片！

后继替换法： 1. 在 (Z) 的右子树中寻找其**直接后继 (Y)**（即右子树中的最小节点）； 2. 将 $(Y.\text{val})$ 拷贝覆盖到 $(Z.\text{val})$ ； 3. 转而**递归删除节点 (Y)**！

目标：删除节点 [10]



为什么后继 (Y) 一定容易删除？

因为 (Y) 是右子树中的最小值，(Y) **绝对不可能拥有左孩子**！因此，删除 (Y) 必定退化为【情况 1】((Y) 是叶子) 或【情况 2】((Y) 只有右孩子)，绝不会再度触发复杂分支！

```

TreeNode* deleteBST(TreeNode* root, int val) {
    if (!root) return nullptr;

    if (val < root->val) {
        root->left = deleteBST(root->left, val);
    } else if (val > root->val) {
        root->right = deleteBST(root->right, val);
    } else {
        // 找到了待删除节点 root
        if (!root->left) {
            TreeNode* temp = root->right;
            delete root;
            return temp; // 情况 1 & 2
        } else if (!root->right) {
            TreeNode* temp = root->left;
            delete root;
            return temp; // 情况 2
        }
        // 情况 3: 左右孩子俱在
        TreeNode* succ = root->right;
        while (succ->left) succ = succ->left; // 寻找后继
        root->val = succ->val; // 覆盖值
        root->right = deleteBST(root->right, succ->val); // 递归删除后继
    }
    return root;
}

```

6.4 顺序倾斜与退化危机：为什么需要平衡树？

BST 的操作复杂度受制于树的高度 (h)。- **最好情况**：节点分布均匀，树高 ($h = \lfloor \log_2 n \rfloor$)，所有操作均为 ($O(\log n)$)；- **最坏情况 (灾难)**：如果按照有序序列 `[1, 2, 3, 4, 5]` 依次插入，树将完全退化为单链表！树高 ($h = n$)，查找与插入退化为悲惨的 ($O(n)$)！

插入有序数据：1 → 2 → 3 → 4 → 5

```

1
 \
 2
  \
 3
  \
 4
  \
 5

```

← 逻辑上是树，物理上是单链表！

工程启示：基础 BST 无法抵御极端输入，必须引入**自动平衡机制**——这就引出了下一章的 AVL 树、红黑树与伸展树！

6.5 BST 合法性验证的两种经典算法（中序遍历 vs 区间收缩）

算法一：中序遍历递增法

维护一个指针 `long long* prevVal`，中序遍历整棵树，若当前节点值 $\leq$ `prevVal`，则判定非法。

算法二：区间收缩法（自顶向下传递约束）

每个节点的值必须严格落在开区间 $((low, high))$ 之内。- 进入左子树时，更新上界： $((low, root.val))$ ；- 进入右子树时，更新下界： $((root.val, high))$ 。

```
bool validate(TreeNode* node, long long low, long long high) {
    if (!node) return true;
    if (node->val <= low || node->val >= high) return false;
    return validate(node->left, low, node->val) &&
           validate(node->right, node->val, high);
}
```

6.6 进阶实战：顺序统计树（Order Statistic Tree）查第 k 小

普通的 BST 仅支持按键值查找。若想在 $O(\log n)$ 内支持：1. `select(k)`：查询树中第 k 小的元素；2. `rank(x)`：查询键为 x 的元素在全树中的排名。

增强设计：在每个节点中增加字段 `int size`，表示以该节点为根的子树所包含的总节点数。[`node.size = (node.left ? node.left.size : 0) + (node.right ? node.right.size : 0) + 1`]

查找第 k 小的核心分治逻辑：
设左子树大小为 `leftSize = node->left ? node->left->size : 0`；
- 若 `k == leftSize + 1`：当前节点恰好就是第 k 小！
- 若 `k <= leftSize`：第 k 小必定落在左子树中，递归进左子树查找第 k 小；
- 若 `k > leftSize + 1`：第 k 小落在右子树中，递归进右子树查找第 $(k - leftSize - 1)$ 小！

本章小结

1. **BST 的灵魂在于中序单调性**：任意节点的整棵左子树都小于它，整棵右子树都大于它。
2. **后继替换化繁为简**：双孩子删除通过借调后继节点，将原本棘手的结构重构巧妙降维成单孩子删除。
3. **树高的致命脆弱性**：普通 BST 无法自愈倾斜退化，催生了平衡树家族的繁荣。
4. **数据结构增强技术**：通过维护局部统计量（如子树 `size`），可以让树形结构以对数时间承载高阶统计功能。

思考题与经典大厂面试题

1. **BST 转双向链表**: 如何不创建任何新节点, 仅通过调整树中各节点的 `left` (作为 `prev`) 和 `right` (作为 `next`) 指针, 将一棵 BST 就地 (In-place) 转换为一个有序的双向循环链表?
2. **BST 中两数之和**: 给定一棵 BST 和一个目标和 (K), 如何利用类似双指针碰撞的思想, 以 $O(n)$ 时间复杂度、 $O(\log n)$ 空间复杂度判断树中是否存在两节点之和等于 (K)?
3. **退化树的重平衡**: 如果一棵 BST 已经不幸退化成了链表, 如何在 $O(n)$ 时间、 $O(n)$ 空间内将其重构为一棵完全平衡的 BST?

<- 第 6 章 二叉搜索树 | 课程目录 | 第 8 章 平衡二叉树 ->

所属模块: 第四部分: 树与二叉树
对应实现: heap.cpp

本章导航

- 7.1 优先队列 ADT 与二叉堆的物理映射
- 7.2 堆序性不变量与核心操作: 上浮 (SiftUp) 与下沉 (SiftDown)
- 7.3 经典证明: 为什么自底向上建堆只需 $O(n)$? (错位相减级数推导)
- 7.4 原地堆排序 (Heap Sort) 算法精要与不稳定性剖析
- 7.5 Top-K 问题经典模型: “阈值过滤器”与双堆求中位数
- 7.5.1 阈值模型: 海量数据 Top-K 为什么用相反性质的堆?
- 7.5.2 对顶堆 (双堆法) 动态维护数据流中位数
- 7.6 可并堆利器: 左式堆 (Leftist Heap) 与零路径长 (NPL) 证明
- 本章小结
- 思考题与经典大厂面试题

第 7 章 堆与优先队列

难度: ★★☆☆☆ | 代码: heap.cpp

前置知识: 第 5 章 完全二叉树的数组下标映射公式, 第 1 章 级数求和与复杂度分析。

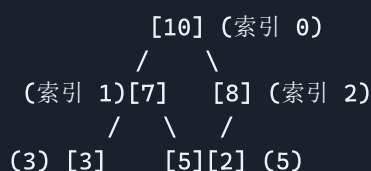
学习目标: 1. 掌握完全二叉堆的数组索引映射不变量与堆序性质; 2. 掌握 `siftUp` 与 `siftDown` 的执行轨迹与边界防御; 3. 掌握自底向上建堆 $O(n)$ 的严格数学推导; 4. 熟练运用“小顶堆求前 K 大 / 大顶堆求前 K 小”解决流式数据极值问题; 5. 理解左式堆通过牺牲右子树深度实现 $O(\log n)$ 快速合并的理论机制。

7.1 优先队列 ADT 与二叉堆的物理映射

在许多应用中，我们并不需要对全体数据进行全局排序，而仅仅关心**当前最具优先级的元素（最大或最小）**。这种抽象数据类型被称为**优先队列（Priority Queue）**。

二叉堆（Binary Heap）是优先队列最经典的实现：

1. **结构性性质（Structure Property）**：它是一棵**完全二叉树**。因此，无需指针，直接使用紧凑的数组连续存储！
2. **下标映射规则（0-based）**：
3. 节点 i 的父节点： $\lfloor (i - 1) / 2 \rfloor$
4. 节点 i 的左孩子： $2i + 1$
5. 节点 i 的右孩子： $2i + 2$
6. **堆序性质（Heap-Order Property）**：
7. **大顶堆（Max Heap）**：任意节点的值 $\geq$ 其任意孩子的值（根节点为全局最大值）；
8. **小顶堆（Min Heap）**：任意节点的值 $\leq$ 其任意孩子的值（根节点为全局最小值）。



数组映射存储：`[10, 7, 8, 3, 5, 2]` —— 100% 紧凑，无指针开销，缓存极度友好！

7.2 堆序性不变量与核心操作：上浮（SiftUp）与下沉（SiftDown）

1. 插入操作：尾部追加 + 上浮（SiftUp, $O(\log n)$ ）

- 将新元素放入数组末尾（维持完全二叉树结构）；
- 若该元素大于其父节点，则与父节点交换；
- 沿着到根的唯一路径反复上浮，直至满足堆序或到达根节点。

2. 删除堆顶：末尾替换 + 下沉（SiftDown, $O(\log n)$ ）

- 堆顶元素即为最值，将其弹出；
- 将数组**末尾最后一个元素移到堆顶**（维持完全二叉树结构）；
- 此时堆顶往往破坏了堆序性。将当前节点与其**两个孩子中较大者**进行比对；若当前节点小于较大孩子，则两者交换；
- 反复下沉，直至满足堆序或成为叶子。

为什么下沉时必须与“较大”的孩子交换？

设根为 (X) ，左孩子为 8，右孩子为 5。若 (X) 换到了右边（与较小的 5 交换），新根将变成 5，但 5 小于左孩子 8，依然破坏大顶堆性质！只有把较大的 8 换上去做新根，才能同时压制住 5 和下沉后的 (X) 。

7.3 经典证明：为什么自底向上建堆只需 $O(n)$ ？（错位相减级数推导）

将一个包含 (n) 个无序元素的数组构建为大顶堆，有两种思路：1. **连续插入建堆（自顶向下）**：从第 1 个元素开始依次执行 **push**（**siftUp**）。最坏复杂度为 $(\sum_{i=1}^n \log i = \Theta(n \log n))$ 。2. **自底向上建堆（Floyd 建堆法）**：直接把无序数组看作完全二叉树，从最后一个非叶节点开始，逆序向前执行 **siftDown**！

```
void buildHeap(std::vector<int>& arr) {
    int n = arr.size();
    // 最后一个非叶节点下标为 (n - 2) / 2
    for (int i = (n - 2) / 2; i >= 0; --i) {
        siftDown(arr, n, i);
    }
}
```

严格时间复杂度证明（错位相减法）：

- 设完全二叉树的高度为 $(H = \lfloor \log_2 n \rfloor)$ ；
- 处于高度 (h) 的节点（叶节点高度 $(h=0)$ ），最多需要下沉 (h) 层；
- 高度为 (h) 的节点数至多为 $(\lceil n / 2^{h+1} \rceil)$ ；
- 所有节点下沉操作的总步数上界为： $[S = \sum_{h=1}^H \lceil \frac{n}{2^{h+1}} \rceil \cdot h \leq \frac{n}{2} \sum_{h=1}^H \frac{h}{2^h}]$ 令级数 $(A = \sum_{h=1}^H \frac{h}{2^h} = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \dots + \frac{H}{2^H})$ 。

两边乘以 2： $[2A = 1 + \frac{2}{2} + \frac{3}{4} + \frac{4}{8} + \dots + \frac{H}{2^{H-1}}]$ 两式相减（错位相减）： $[2A - A = 1 + \left(\frac{2-1}{2}\right) + \left(\frac{3-2}{4}\right) + \dots + \left(\frac{H - (H-1)}{2^{H-1}}\right) - \frac{H}{2^H}]$ $[A = 1 + \left(\frac{1}{2}\right) + \left(\frac{1}{4}\right) + \dots + \left(\frac{1}{2^{H-1}}\right) - \frac{H}{2^H}]$ $= 1 + \left(1 - \frac{1}{2^{H-1}}\right) - \frac{H}{2^H} < 2$ 代回原式： $[S \leq \frac{n}{2} \cdot 2 = n = O(n) \quad \blacksquare]$

直观物理解释：

在树中，**绝大多数节点集中在最底层**（叶子占了近 $(n/2)$ ）。自底向上建堆时，海量的底层节点只需要下沉 0 步或 1 步；只有极少数的顶层节点（根节点只有 1 个）才需要下沉 (H) 步！“多劳者少，少劳者多”，总代价自然被收敛在线性 $(O(n))$ ！

7.4 原地堆排序（Heap Sort）算法精要与不稳定性剖析

堆排序由 J. W. J. Williams 于 1964 年发明，是唯一在最坏情况下兼具 $(O(n \log n))$ 时间与 $(O(1))$ 原地辅助空间的经典排序算法。

升序排序三部曲：

1. 将无序数组原地构建为【大顶堆】（耗时 $O(n)$ ）；
2. 将堆顶最大元素与当前堆末尾元素交换，堆有效大小减 1；
3. 对新堆顶执行 **siftDown** 恢复大顶堆性质；
4. 重复 2 和 3 直到堆大小为 1，整个数组便呈现严格升序！

为什么堆排序是不稳定的？

在将末尾元素换至堆顶，或在下沉过程中，相隔遥远的相同元素会被粗暴交换跨过彼此。

例：数组 `[5a, 5b, 3]` 构建大顶堆后仍为 `[5a, 5b, 3]`。第 1 轮将堆顶 `5a` 与末尾 `3` 交换，得到 `[3, 5b, 5a]`。此时 `5a` 已经排到了 `5b` 后面，原相对顺序彻底颠倒！

7.5 Top-K 问题经典模型：“阈值过滤器”与双堆求中位数

7.5.1 阈值模型：海量数据 Top-K 为什么用相反性质的堆？

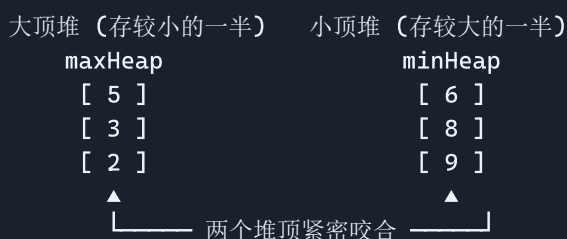
经典面试题：从 10 亿个浮点数中找出最大的 100 个数。内存仅能容纳数千个元素。

反直觉法则：- 求前 K 个最大值 $\implies$ 维护大小为 K 的小顶堆！ - 求前 K 个最小值 $\implies$ 维护大小为 K 的大顶堆！

原因解释：- 小顶堆的堆顶是这 (K) 个候选人中的最弱者（阈值）；- 遍历后续数据流中的每个新元素 (x)：- 若 $(x \leq \text{堆顶})$ ，说明它连候选人里最弱的都比不过，直接丢弃；- 若 $(x > \text{堆顶})$ ，说明它有资格晋级！淘汰当前堆顶，将 (x) 放入堆中并调整。- 若使用大顶堆，堆顶是最大值，当来了一个新元素时，我们无法判断它是否比这 (K) 个数中最小的那个大，阈值机制完全失效！

7.5.2 对顶堆（双堆法）动态维护数据流中位数

在流式数据中，数字不断流入，要求随时以 $O(1)$ 时间返回中位数。



平衡不变量：1. `maxHeap` 存放较小的一半元素，`minHeap` 存放较大的一半元素；2. 元素总数为偶数时，两堆大小相等；为奇数时，`maxHeap.size() == minHeap.size() + 1`；3. `maxHeap.top() ≤ minHeap.top()` 恒成立。

中位数查询：若总数为奇数，直接返回 `maxHeap.top()`；若为偶数，返回两堆顶的均值！

7.6 可并堆利器：左式堆（Leftist Heap）与零路径长（NPL）证明

普通二叉堆若要合并两个大小为 (n) 的堆，必须将两数组拼接后重新建堆，耗时 $O(n)$ 。为了以 $O(\log n)$ 支持高效合并（merge），产生了左式堆（Leftist Heap）。

零路径长 (Null Path Length, NPL) :

- 节点 x 的 $\text{npl}(x)$ 定义为从 x 出发到达一个**不具有两个孩子的节点 (外节点)** 的最短路径边数;
- 空节点的 $\text{npl}(\text{nullptr}) = -1$; 叶子节点的 $\text{npl} = 0$;
- $\text{npl}(x) = \min(\text{npl}(x.\text{left}), \text{npl}(x.\text{right})) + 1$ 。

左倾性质 (Leftist Property) :

对于树中任意节点 x , 必有 $\text{npl}(x.\text{left}) \geq \text{npl}(x.\text{right})$ 。

核心推论: 节点的 $\text{npl}(x)$ 完全由其**右孩子**决定: $\text{npl}(x) = \text{npl}(x.\text{right}) + 1$ 。

定理: 一棵包含 r 个右路径节点的左式堆, 至少包含 $(2^r - 1)$ 个节点。

也就是说, 包含 (n) 个节点的左式堆, 其右路径长度至多为 $(\lfloor \log_2(n + 1) \rfloor)$!

递归合并核心: 始终沿右路径深入!

```
LeftistNode* merge(LeftistNode* h1, LeftistNode* h2) {
    if (!h1) return h2;
    if (!h2) return h1;
    // 保证 h1 根节点较小 (以小顶堆为例)
    if (h1->val > h2->val) std::swap(h1, h2);

    // 递归将 h2 合并到 h1 的右子树上
    h1->right = merge(h1->right, h2);

    // 若违反左倾性质, 交换左右子树
    if (getNpl(h1->left) < getNpl(h1->right)) {
        std::swap(h1->left, h1->right);
    }
    // 更新当前节点的 npl
    h1->npl = getNpl(h1->right) + 1;
    return h1;
}
```

本章小结

1. **二叉堆是数组连续存储的巅峰之作:** 完全二叉树的物理特性消除了指针开销与内存碎片。
2. **Floyd 建堆的精妙:** 自底向上的下沉策略契约了“底层多、顶层少”的几何规律, 以 $O(n)$ 线性时间完成建堆。
3. **门槛模型是流式极值查询的屠龙刀:** 小顶堆控大、大顶堆控小, 对顶堆夹逼中位数。
4. **左式堆的非对称哲学:** 通过主动把树的深度“左倾”, 将右路径强行压缩至 $O(\log n)$, 化解了堆合并的线性诅咒。

思考题与经典大厂面试题

1. **d 叉堆 (d-ary Heap) 的时空折中**: 若将二叉堆推广为每个节点拥有 (d) 个孩子的 (d) 叉堆, 其 `siftUp` 和 `siftDown` 的时间复杂度分别是多少? 在什么应用场景 (例如 Dijkstra 稠密图) 下 ($d > 2$) 的堆表现优于普通二叉堆?
2. **二叉堆的原地索引查询瓶颈**: 标准二叉堆不支持以 ($O(\log n)$) 修改指定元素的值或删除指定元素 (因为在数组中寻找该元素需要 ($O(n)$))。如何通过引入“反向索引映射数组 (Inverted Index)”实现可索引优先队列 (Indexed Priority Queue) ?
3. **斜堆 (Skew Heap) 与左式堆的关系**: 斜堆不维护任何 `npL` 字段, 每次递归合并时无条件交换左右子树。请简述为什么斜堆能实现均摊 ($O(\log n)$) 的合并复杂度。

<- 第 7 章 堆与优先队列 | 课程目录 | 第 9 章 多路搜索树 ->

所属模块: 第四部分: 树与二叉树

对应实现: `avl_tree.cpp` / `redblacktree.cpp` / `splaytree.cpp`

本章导航

- 8.1 自平衡二叉搜索树的设计哲学与演进脉络
- 8.2 AVL 树: 严格高度平衡的开山之作
 - 8.2.1 平衡因子 (BF) 与最少节点数推导 (斐波那契序列)
 - 8.2.2 四大失衡形态与旋转原理 (LL / RR / LR / RL 纯 ASCII 图解)
 - 8.2.3 插入与删除的旋转代价差异剖析
- 8.3 红黑树 (Red-Black Tree): 工业界首选的工程艺术
 - 8.3.1 五大红黑性质的数学本质与黑高 (Black-Height) 定理
 - 8.3.2 红黑树与 2-3-4 树的完美同构投影 (顿悟之钥)
 - 8.3.3 插入修复的三大场景与变色旋转演进
 - 8.3.4 删除修复的四大分支彻底剖析 (“双重黑色”与借黑还红)
- 8.4 伸展树 (Splay Tree): 自适应访问与均摊分析典范
 - 8.4.1 核心哲学: 局部性原理与“访问即重构”
 - 8.4.2 三种 Splay 旋转: 为什么 Zig-Zig 必须先旋父再旋子?
- 8.5 平衡树家族横向技术选型雷达
- 本章小结
- 思考题与经典大厂面试题

第 8 章 平衡二叉树

难度: ★★★★★ | 代码: `avl_tree.cpp` / `redblacktree.cpp` / `splaytree.cpp`

前置知识：第 6 章 BST 基本操作与退化问题，第 1 章 均摊分析（势能法）。

学习目标：1. 掌握 AVL 树四种失衡旋转的精确几何演变与高度更新依赖；2. 深刻领悟红黑树五大性质的数学本质，借助 2-3-4 树同构模型彻底理解插入与删除修复；3. 掌握红黑树删除修复中“双重黑色”的传递机制与 Case 1~4 的全套变换；4. 理解伸展树通过双层旋转折叠树高、实现均摊 $O(\log n)$ 的精妙证明。

8.1 自平衡二叉搜索树的设计哲学与演进脉络

在上一章中我们看到，基础 BST 在最坏情况下会彻底退化为 $O(n)$ 的链表。为了驯服这种倾斜，计算机科学家们走过了三条不同的演进路径：



8.2 AVL 树：严格高度平衡的开山之作

8.2.1 平衡因子（BF）与最少节点数推导（斐波那契序列）

定义：节点 x 的**平衡因子**（Balance Factor, BF）为其左子树高度减去右子树高度： $BF(x) = height(x.left) - height(x.right)$ **AVL 不变量：**树中**每一个节点**的平衡因子必须满足： $BF(x) \in \{-1, 0, 1\}$ 。一旦任何节点的 $|BF| > 1$ ，立即触发旋转修复。

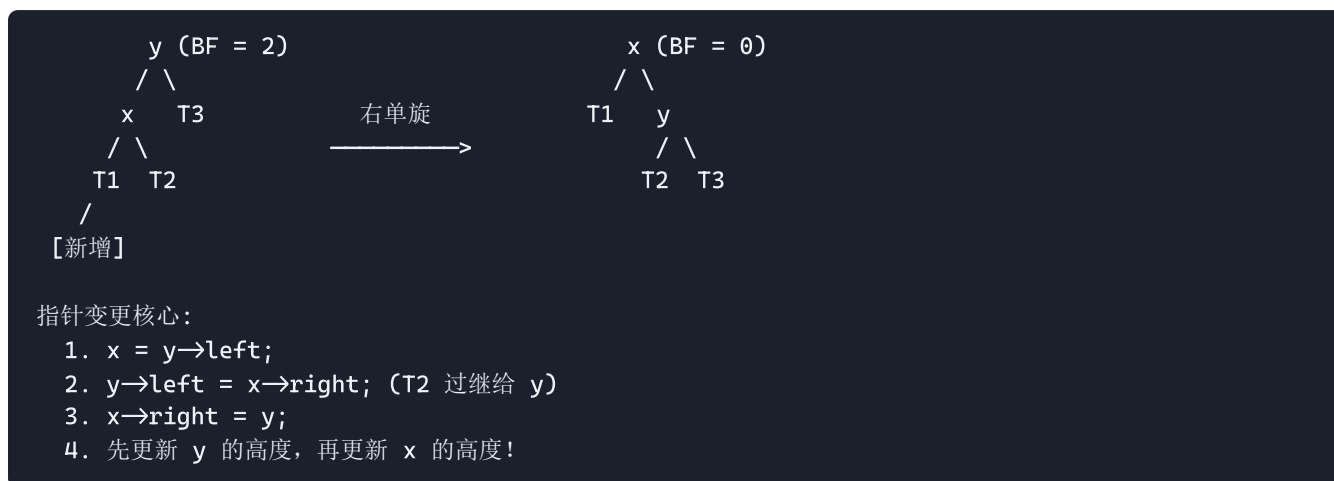
AVL 树的高度上界推导：

设高度为 h 的 AVL 树所需要的最少节点数为 $N(h)$ ：
- $N(0) = 1$ （只有根节点）；
- $N(1) = 2$ ；
- 要让高度为 h 且节点最少，其左右子树高度必须分别为 $h-1$ 和 $h-2$ ： $N(h) = N(h-1) + N(h-2) + 1$ 这与**斐波那契数列（Fibonacci Sequence）**高度同构！根据斐波那契数列的通项公式（黄金分割比 $\phi = \frac{1 + \sqrt{5}}{2} \approx 1.618$ ）： $N(h) \approx \frac{\phi^{h+2}}{\sqrt{5}} - 1 \implies h < 1.44 \log_2(n+2) - 0.328$ **结论：**即使在最坏情况下，AVL 树的高度也绝对不超过 $(1.44 \log_2 n)$ 。查找性能极度稳定。

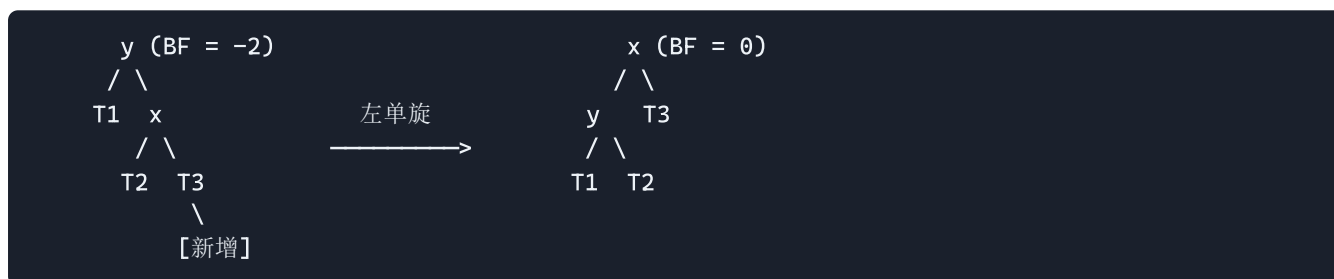
8.2.2 四大失衡形态与旋转原理（纯 ASCII 规范图解）

当在 AVL 树中插入一个新节点导致某个祖先节点失衡 ($|BF| = 2$) 时，根据插入位置与失衡节点的关系，分为四种情况：

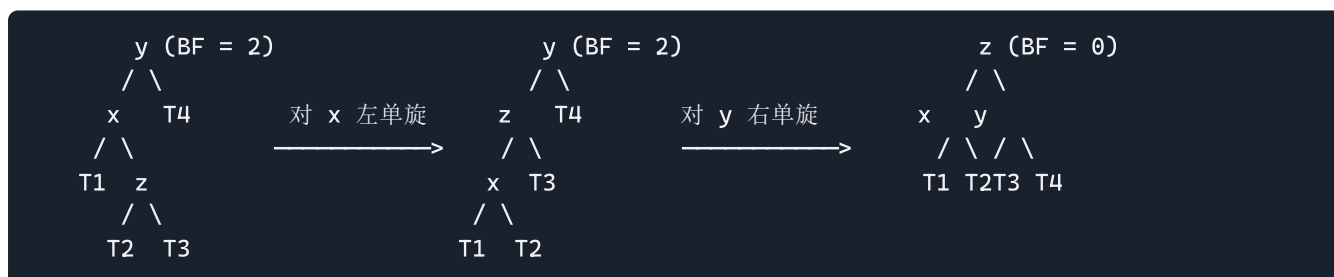
1. LL 型失衡（在左孩子的左子树插入） $\implies$ 执行【右单旋】



2. RR 型失衡（在右孩子的右子树插入） $\implies$ 执行【左单旋】（与 LL 完全镜像）



3. LR 型失衡（在左孩子的右子树插入） $\implies$ 先对左子【左旋】，再对根【右旋】



4. RL 型失衡（在右孩子的左子树插入） $\implies$ 先对右子【右旋】，再对根【左旋】（与 LR 完全镜像）

⚠️ 更新高度的依赖原则：

旋转完成后，节点的位置发生了高低颠倒。**必须先重新计算下层节点（原根节点 (y)）的高度，再计算上层新根节点 ((x) 或 (z)) 的高度！** 颠倒次序将导致祖先节点读到过期的脏高度！

8.2.3 插入与删除的旋转代价差异剖析

- **插入操作：**插入一个节点后，沿着回溯路径向上检查。**至多需要 1 次单旋或 1 次双旋（共 1~2 次局部旋转）**，子树高度就会恢复为插入前的高度，失衡不会继续向上传播，旋转立即终止！

- **删除操作**：删除一个节点并旋转修复后，该子树的高度可能会比删除前降低 1。这一高度变化可能导致其父节点乃至更高层的祖先**再度失衡**！因此，在最坏情况下，删除操作可能需要沿着整条到达根节点的路径进行 ($O(\log n)$) 次连续旋转。

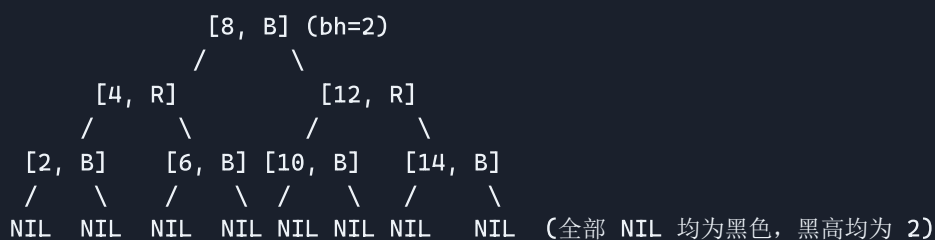
8.3 红黑树 (Red-Black Tree)：工业界首选的工程艺术

红黑树是自平衡 BST 领域的工程霸主。从 C++ STL 的 `std::map`、`std::set`，到 Linux CFS 完全公平调度器，再到 Java 的 `TreeMap` 和 `epoll` 事件就绪队列，背后全都是红黑树的身影。

8.3.1 五大红黑性质的数学本质与黑高 (Black-Height) 定理

红黑树是一棵在每个节点上增加了一个存储位表示节点颜色 (`RED` 或 `BLACK`) 的二叉搜索树。它必须严格满足以下五条性质：

1. 每个节点要么是红色的，要么是黑色的；
2. 根节点永远是黑色的；
3. 所有的叶子节点 (NIL 哨兵节点) 被视为黑色；
4. 如果一个节点是红色的，则它的两个子节点必须全都是黑色的 (即：绝对不允许出现连续两个红色节点)；
5. 从任意节点出发到其所有后代叶子节点的每条简单路径上，均包含相同数目的黑色节点 (这条性质被称为“黑高相同”)。



为什么这五条性质能保证树的高度对数受限？

- 根据性质 5，每条路径上的黑色节点数目必须完全相等 (设为 bh)；
- 根据性质 4，红色节点不能相邻，这意味着：在任意路径中，红色节点的数量至多等于黑色节点的数量；
- 因此：
- **最短可能路径**：全黑节点，长度为 bh ；
- **最长可能路径**：红黑交替相间，长度为 $(2 \cdot bh)$ ；
- **核心推论**：一棵包含 (n) 个内部节点的红黑树，其高度至多为 $(2 \log_2(n + 1))!$

深层不变量直观：

黑色节点才是红黑树的“真正主干”，而红色节点只是“寄居”在黑色节点周围、用于吸收动态变化的缓冲垫！

8.3.2 红黑树与 2-3-4 树的完美同构投影 (顿悟之钥)

许多人觉得红黑树的变色和旋转极其神秘。其实，**红黑树在本质上就是经典 B 树家族中 2-3-4 树的“二叉形式等价物”**！

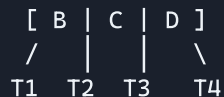
【2-3-4 树中的 3-节点】



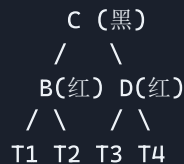
【等价的红黑树子结构（红色节点向黑色父节点贴靠）】



【2-3-4 树中的 4-节点】



【等价的红黑树子结构（两红夹一黑）】



- **红黑树的变色** $\iff$ 2-3-4 树中 4-节点满载时的**裂变 (Split)**；
- **红黑树的旋转** $\iff$ 2-3-4 树中节点内部子树形态的**重平衡 (Rebalance)**！
只要脑海中有这幅同构图，红黑树的全部修复逻辑将瞬间洞若观火！

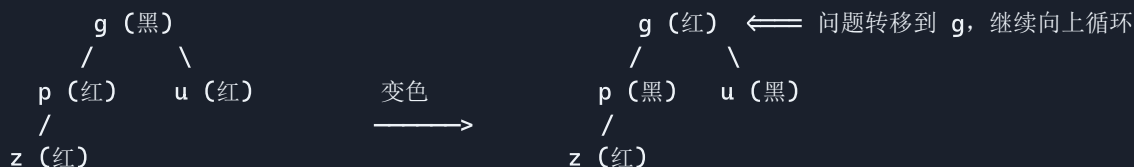
8.3.3 插入修复的三大场景与变色旋转演进

插入准则：新插入的节点 (z) 默认染成**红色**。

为什么是红色？因为插入红色节点绝不会破坏性质 5（黑高平衡），它唯一可能破坏的是性质 2（若为根）或性质 4（若父节点恰好也是红色，产生“红红冲突”）。

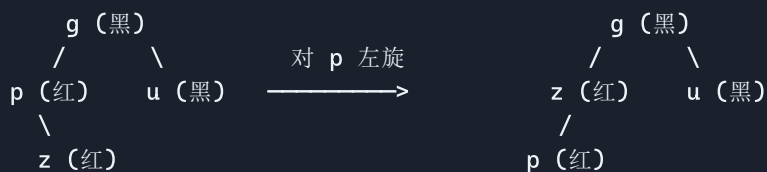
设新节点为 (z)，父节点为 (p)，祖父节点为 (g)，叔叔节点为 (u)（设 (p) 为 (g) 的左孩子，右侧对称）：

【Case 1】叔叔节点 (u) 是红色 $\implies$ 变色并向上传递



操作：令 (p) 和 (u) 染为黑色，(g) 染为红色；将当前考察节点指针移到 (g)，继续向上检查。

【Case 2】叔叔节点 (u) 是黑色，且 (z) 是右孩子（折线型） $\implies$ 对 (p) 左旋转化为 Case 3



操作：令当前节点 (z = p)，对 (p) 执行左单旋，形态瞬间转化为直线型（Case 3）。

【Case 3】叔叔节点 (u) 是黑色，且 (z) 是左孩子 (直线型) $\implies$ 变色 + 对 (g) 右旋 (终结!)



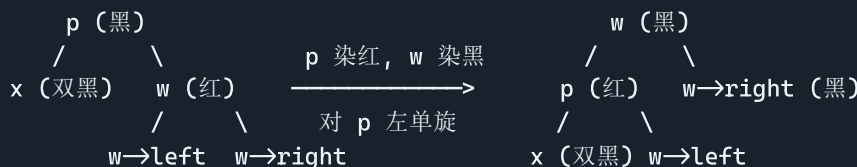
操作：令 (p) 染黑，(g) 染红，对 (g) 执行右单旋。红红冲突彻底化解，全树恢复平衡，直接终止！

8.3.4 删除修复的四大分支彻底剖析 (“双重黑色”与借黑还红)

删除是红黑树中最艰深的部分。- 若删除的是红色节点，黑高未变，无需任何修复；- 若删除（或移走）的是黑色节点，经过该路径的黑色节点数减少了 1，性质 5 被破坏！- **核心抽象**：设取代被删位置的节点为 (x)，我们认为 (x) 额外背负了一层黑色，即成为**双重黑色 (Double-Black)** 节点。删除修复的使命，就是**将这个多余的黑色推向根部消解，或者通过兄弟节点借调黑色来消除**！

设 (x) 是其父节点 (p) 的左孩子，兄弟节点为 (w)：

【Case 1】兄弟节点 (w) 是红色



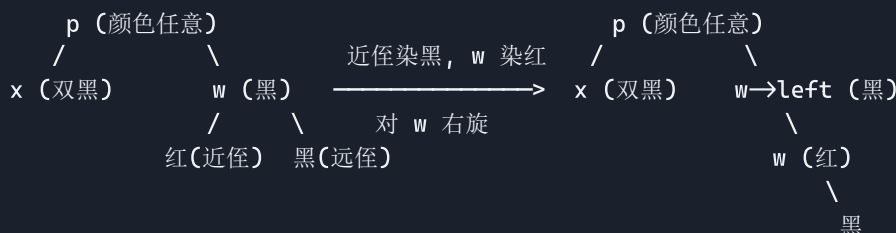
本质：通过旋转和变色，将红兄弟转化为黑兄弟，从而顺利套用后续的 Case 2/3/4！

【Case 2】兄弟 (w) 是黑色，且 (w) 的两个孩子全为黑色



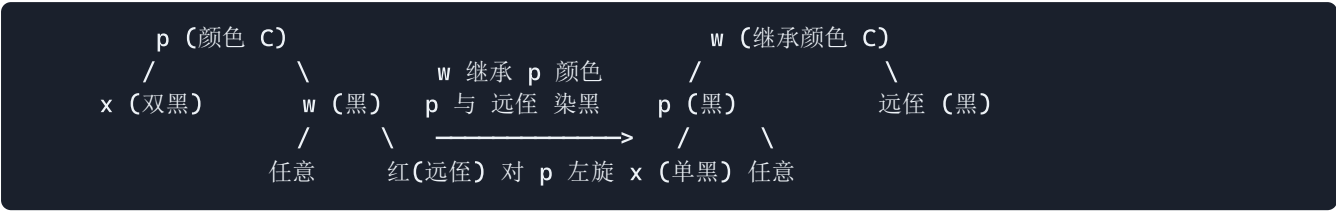
本质：兄弟 (w) 也很穷，拿不出多余的黑色。于是 (w) 变红（让右侧也少一个黑），将两边共同缺失的黑色上推给父节点 (p)。若 (p) 原本为红，直接染黑即可终止；若 (p) 为黑，则 (p) 成为新的双黑节点向上回溯。

【Case 3】兄弟 (w) 为黑色，近侧侄子 (左孩子) 为红，远侧侄子 (右孩子) 为黑



本质：近侧红侄子无法在左旋后支撑黑高，先通过右单旋将红色“转运”到远侧，转换为 Case 4。

【Case 4】兄弟 (w) 为黑色，远侧侄子 (右孩子) 为红色 (绝杀！)



本质：远侧侄子富得流油！通过一次对 (p) 的左旋，把兄弟 (w) 提到父节点位置，让它继承 (p) 的颜色；同时将 (p) 和远侄染黑。此时左侧双黑被彻底化解，右侧黑高完全平衡，**算法完美终止！**

8.4 伸展树 (Splay Tree)：自适应访问与均摊分析典范

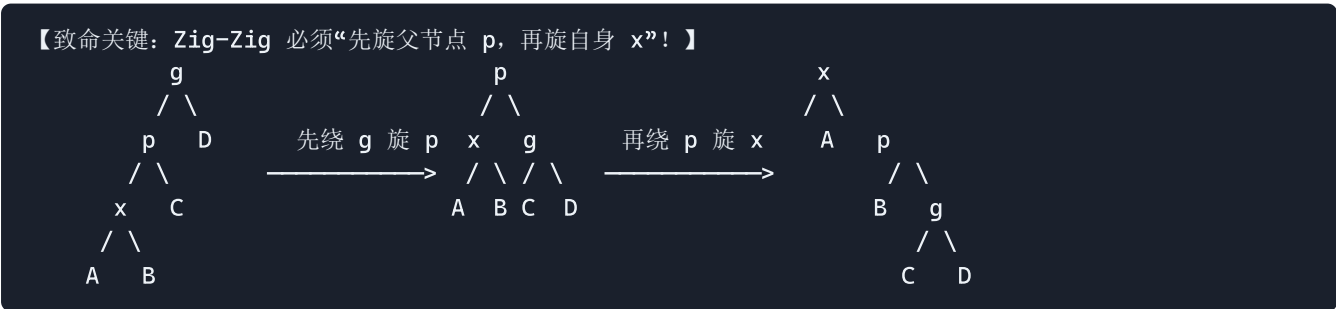
8.4.1 核心哲学：局部性原理与“访问即重构”

在实际计算中，**数据的访问极不均衡**——80% 的查询往往集中在 20% 的热点数据上（80/20 法则与时间局部性）。

由图灵奖得主 Robert Tarjan 和 Daniel Sleator 发明的**伸展树 (Splay Tree)** 贯彻了极简主义：- 节点中**不存储任何平衡因子或颜色标记**；- 每次访问（查找、插入、删除）任何一个节点 (x) 后，立即执行一系列旋转，**将节点 (x) 强行旋转到树根 (Splay 操作)**；- 从而让刚刚访问过的热点数据在接下来的访问中只需 $O(1)$ 即可直接命中！

8.4.2 三种 Splay 旋转：为什么 Zig-Zig 必须先旋父再旋子？

1. Zig 操作（单旋）：当父节点 (p) 已经是根节点时，单旋一次直接到根。
2. Zig-Zag 操作（双旋，异侧）：(x) 与 (p) 处于异侧折线，与 AVL 的 LR/RL 类似，连续旋转 (x) 两次。
3. Zig-Zig 操作（双旋，同侧）：(x) 与 (p) 处于同一直线。



为什么不能先旋 x 再旋 x?
如果像朴素单旋那样连续对 (x) 旋转两次，整棵子树的结构只是被粗暴地拉扯，原先很深的树高依然很深！而**先旋父再旋子**能够将从 (g) 到 (x) 的整条瘦长路径**沿途折叠对半砍 (Halving Effect)**！正是这种折叠效应，才在数学上严格保证了 Tarjan 的势能函数分析成立，从而赋予了伸展树令人惊叹的**均摊 $O(\log n)$ 复杂度**！

8.5 平衡树家族横向技术选型雷达

维度	AVL 树	红黑树	伸展树 (Splay)
平衡度	极严格 (高度差 ≤ 1)	适度 (最长 $\leq 2 \times$ 最短)	动态自适应 (无显式平衡界)
最大树高	$\approx 1.44 \log_2 n$	$\leq 2 \log_2(n + 1)$	最坏单次 $O(n)$, 均摊 $O(\log n)$
额外空间开销	每个节点存一个 <code>int height</code>	每个节点存 1 个 bit (<code>Color</code>)	0 额外空间 (纯粹的指针)
插入旋转次数	至多 2 次旋转	至多 2 次旋转, 若干次变色	伴随路径重构
删除旋转次数	最坏需 $O(\log n)$ 次	至多 3 次旋转!	伴随路径重构
最适合场景	读多写极少 (如静态只读字典)	读写均衡的高频增删 (如 STL map)	局部性极强、热点集中 (如缓存代理)

本章小结

- AVL 是严谨的数学对称之美**：以严格的高度差换取了极致的查询速度，插入只需常数次旋转，但删除可能沿途多次调整。
- 红黑树是工业妥协的最高境界**：通过引入红色的缓冲机制，将插入和删除的旋转次数全部压制在常数步 (≤ 3) 次，是写密集型业务的不二之选。
- 2-3-4 树是红黑树的解密钥匙**：变色即分裂，旋转即重排，看透这一点便不再畏惧红黑树代码。
- 伸展树的折叠智慧**：放弃静态规则，拥抱时间局部性；Zig-Zig 先父后子的旋转次序是均摊对数复杂度的数学灵魂。

思考题与经典大厂面试题

- 为什么红黑树删除最多只需 3 次旋转?**：对照 Case 1~4 的状态转移图，证明从任何一个失衡状态开始，在执行至多 3 次旋转后必定进入终止状态。
- 左倾红黑树 (LLRB, Left-Leaning Red-Black Tree)**：Robert Sedgewick 提出的 LLRB 规定“所有红色链接必须向左倾斜”。相比标准红黑树，这种约束如何简化了插入和删除的代码实现？
- Splay 树的势能证明简述**：设节点 x 的秩 $r(x) = \lfloor \log_2(\text{size}(x)) \rfloor$ ，整棵树的势能 $(\Phi = \sum r(x))$ 。试简述为什么先旋父后子的 Zig-Zig 操作能够产生负的势能变化量。

所属模块：第四部分：树与二叉树

对应实现：multiwaytree.cpp / bplustree.cpp

本章导航

- 9.1 存储墙与磁盘 I/O：为什么二叉树在海量数据面前失效？
- 9.2 B 树 (B-Tree)：经典多路平衡搜索树
 - 9.2.1 m 阶 B 树的严格定义与容量平衡不变量
 - 9.2.2 插入操作：节点满溢与中间键上提分裂 (Split)
 - 9.2.3 删除操作：借位 (Borrow) 与下溢合并 (Merge)
- 9.3 B+ 树 (B+ Tree)：数据库与文件系统的索引霸主
 - 9.3.1 B+ 树相对 B 树的架构革新
 - 9.3.2 为什么 MySQL InnoDB 坚定选择 B+ 树？(四大决定性理由)
 - 9.3.3 叶节点分裂与中间键“保留副本”的关键机制
- 9.4 工业级数据库实战：聚集索引与二级索引的物理组织
- 本章小结
- 思考题与经典大厂面试题

第 9 章 多路搜索树

难度：★★★★☆ | 代码：multiwaytree.cpp / bplustree.cpp

前置知识：第 6 章 BST 搜索逻辑，第 8 章 红黑树与 2-3-4 树的等价关系。

学习目标：1. 理解“外存储器（磁盘/SSD）I/O 成为主瓶颈”的系统背景；2. 掌握 (m) 阶 B 树的严格定义、节点容量上下界不变量；3. 熟练演练 B 树的分裂、向兄弟借位与合并过程；4. 透彻掌握 B+ 树在范围查询、缓存命中与树高控制上对 B 树的全面超越。

9.1 存储墙与磁盘 I/O：为什么二叉树在海量数据面前失效？

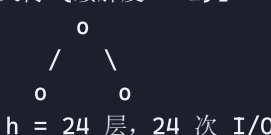
在内存中，CPU 访问指针引用的耗时仅为几十纳秒；但在企业级数据库中，数据量通常高达数十 GB 到数十 TB，远超物理内存容量，必须持久化在磁盘或 SSD 上。

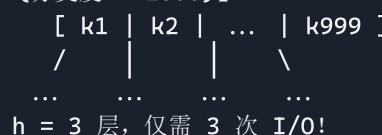
磁盘与内存的“物理时间鸿沟”：

- **内存读取：** $\approx 50 \sim 100 \text{ ns}$
- **机械磁盘随机寻道：** $\approx 5 \sim 10 \text{ ms}$ (**相差 10 万倍！**)
- **SSD 随机读取：** $\approx 50 \sim 100 \mu\text{s}$ (依然相差千倍！)

若使用红黑树或 AVL 树管理 1000 万条记录： - 树高 ($h \approx \log_2(10^7) \approx 24$) 层； - 每次查找目标键，指针散落在不同磁盘页中，最坏需要执行 **24 次磁盘 I/O!** - 耗时 ($24 \times 10 \text{ ms} = 0.24 \text{ 秒}$)! 单次查询耗费近 1/4 秒，系统吞吐量彻底归零!

破局之道：增大分支因子 (Branching Factor / Fan-out)，将“瘦高的二叉树”压扁为“极矮胖的多路树”!

【二叉树 (矮胖度 = 2)】

h = 24 层, 24 次 I/O

【多路树 (分支度 = 1000)】

h = 3 层, 仅需 3 次 I/O!

9.2 B 树 (B-Tree)：经典多路平衡搜索树

1970 年，Rudolf Bayer 和 Edward M. McCreight 发明了 B 树 (B 意为 Balanced 或 Boeing)。

9.2.1 m 阶 B 树的严格定义与容量平衡不变量

一棵 (m) 阶 ($m \geq 3$) B 树必须满足以下不变量： 1. **每个节点至多拥有 (m) 棵子树，至多包含 (m - 1) 个关键字 (Keys)**； 2. **根节点若非叶子，则至少拥有 2 棵子树 (至少 1 个关键字)**； 3. **除根节点外的所有非叶子节点，至少拥有 ($\lceil m/2 \rceil$) 棵子树，至少包含 ($\lceil m/2 \rceil - 1$) 个关键字**； 4. **所有叶子节点必须位于绝对相同的深度 (完美高度平衡!)**； 5. **关键字有序排列**：节点内的关键字满足 ($K_1 < K_2 < \dots < K_k$)，且各子树中的所有关键字均落在对应关键字所夹的开区间之内。

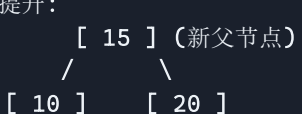
一个 5 阶 B 树节点的标准物理布局 (最多 4 个 Key, 5 个子指针):

P0	K1	P1	K2	P2	K3	P3	K4	P4
----	----	----	----	----	----	----	----	----

其中：P0 树中所有值 < K1；P1 树中所有值介于 (K1, K2)； ...

9.2.2 插入操作：节点满溢与中间键上提分裂 (Split)

新关键字永远首先插入到**底层叶子节点**中： 1. 若插入后该节点的关键字数 ($\leq m - 1$)，插入成功； 2. 若插入后该节点关键字数达到 (m) (**发生溢出 Overflow**)，则必须执行**中间键上提分裂**： - 提取该节点正中间的关键字 (位置在 ($\lfloor m/2 \rfloor$))； - 将中间键**推入其父节点**中； - 原节点裂变为左右两个独立的子节点，中间键成为连接它们的父级枢纽； - 若父节点也因吸收新键而溢出，则递归向上分裂，直至根节点；若根节点也分裂，则树高增 1!

示例：在 3 阶 B 树 (2-3 树) 中插入 15
原节点：[10 | 20] (已满)
插入 15 后临时状态：[10 | 15 | 20] (溢出!)
中间键 15 向上提升：


9.2.3 删除操作：借位 (Borrow) 与下溢合并 (Merge)

若被删除关键字在非叶子节点中，**必须用其前驱或后继关键字覆盖**，将删除操作转化为底层叶子节点的删除。

若删除后叶子节点的关键字数 ($\lceil m/2 \rceil - 1$) (**发生下溢 Underflow**)：

- **分支 A (向富裕兄弟借位)**：若左或右相邻兄弟拥有多于下限的关键字，不能直接借过来，必须**通过父节点进行“三方轮换借位”**（父节点的键下沉，兄弟节点的键上浮到父节点）。
- **分支 B (向贫困兄弟合并)**：

若左右兄弟均无富余，则将当前节点、相邻兄弟节点以及父节点中介于它们之间的关键字**三者合并为一个新节点**。

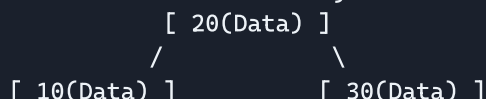
父节点因此失去一个键，若父节点也下溢，递归向上调整！

9.3 B+ 树 (B+ Tree)：数据库与文件系统的索引霸主

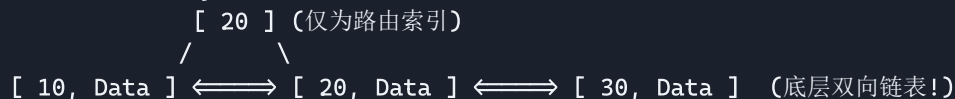
尽管 B 树大幅降低了树高，但它在数据库查询中依然面临一个致命短板：**范围查询性能低下**。B+ 树正是为了解决这一痛点而诞生的终极形态。

9.3.1 B+ 树相对 B 树的架构革新

【B 树】所有节点均存储完整的 Key + Data



【B+ 树】非叶子仅存 Key (做路由导航)，所有 Data 集中于叶子，叶子由双向链表相连



9.3.2 为什么 MySQL InnoDB 坚定选择 B+ 树？（四大决定性理由）

1. 磁盘 I/O 效率达到物理极限 (树极度矮胖)：

InnoDB 的默认物理页大小为 16 KB。若为主键索引 (`bigint` 占 8B，子指针占 6B，单条路由占 14B)，一个 16KB 的内部节点可容纳 $(16384 / 14 \approx 1170)$ 个分支指针！

2. 深度为 3 的 B+ 树可容纳：(1170 $\times$ 1170 $\times$ 16 $\approx$ 2000 万行数据)！

3. 千万级的数据表，只需 3 次磁盘 I/O 即可精确定位！根节点常驻内存，实际仅需 1~2 次 I/O！

4. 范围查询极速吞吐 (Range Query)：

执行 `WHERE age BETWEEN 18 AND 30` 时，B 树必须执行复杂的中序遍历跨页跳转；而 B+ 树**只需通过索引找到 18，然后沿着底层双向链表顺序线性扫描至 30**，顺序 I/O 吞吐量极高！

5. 查询性能高度稳定：

任何查询都必须一路走到叶子节点，查询耗时高度一致（无抖动）。

6. 叶子节点可直接存放整行数据 (聚集索引)：

减少一次回表查询开销。

9.3.3 叶节点分裂与中间键“保留副本”的关键机制

⚠ **B+ 树与 B 树最核心的代码区别：**

- B 树分裂时，中间键被**抽走并推入父节点**（原节点不再包含该键）；
- B+ 树分裂时，中间键被**推入父节点作为路由副本**，**但该键同时必须完好保留在右侧叶子节点中**！因为所有真实数据必须在底层叶子中全集呈现！

9.4 工业级数据库实战：聚集索引与二级索引的物理组织

在 MySQL InnoDB 存储引擎中： - **聚集索引（Clustered Index）**：以主键构建的 B+ 树，其**叶子节点直接存储整行数据记录**； - **二级索引（Secondary Index / 辅助索引）**：以非主键列（如 `email`）构建的 B+ 树，其**叶子节点存储的是该列的值以及对应的主键值**； - **回表（Bookmark Lookup）**：通过二级索引查到主键后，再次进入聚集索引 B+ 树查找完整行数据的过程。

本章小结

1. **外存瓶颈重塑算法体系**：算法分析从“比较次数最少”演进为“磁盘页读取次数最少”。
2. **B 树的分支哲学**：通过多路平衡保证完美树高，中间键上提与向兄弟借位是维持容量不变量的核心。
3. **B+ 树的解耦艺术**：将索引路由（非叶子）与数据存储（叶子）彻底分离，底层双向链表化解了范围查询的世纪难题。
4. **数据库索引的基石**：B+ 树以 3~4 层的极浅深度统领千万级数据，是当代所有关系型数据库的底层核心支柱。

思考题与经典大厂面试题

1. **InnoDB 为什么不用哈希表或红黑树做索引？**：分别从范围查询、磁盘页局部性、树高与内存利用率角度给出系统性对比。
2. **B+ 树并发控制（B-Tree Latching）**：在多线程高并发读写环境下，B+ 树分裂可能向上层传播引起连锁加锁。工业界如何利用“蟹行锁（Crabbing / Lock Coupling）”或“乐观锁并发”来保证高效读写？
3. **手算演练**：给定一棵空的 3 阶 B+ 树，依次插入数据 `[5, 15, 25, 35, 45]`，请画出每次分裂后的整棵树形态，特别标明叶子链表的链接关系。

<- 第 9 章 多路搜索树 | 课程目录 | 第 11 章 图论算法 ->

所属模块：第四部分：树与二叉树
对应实现：kdtree.cpp

本章导航

- 10.1 多维空间索引的挑战：为什么 1 维 BST 失效？
- 10.2 KD 树的几何本质：正交超平面的空间剖分
- 10.3 KD 树的构建算法与维度切分策略
 - 10.3.1 轮换切分 vs 最大方差优先切分
 - 10.3.2 中位数选取与 `std::nth_element` 线性建树优化 ($O(n \log n)$)
 - 10.3.3 二维点集建树分步追踪 (Dry-Run)
- 10.4 核心算法一：范围查询 (Range Query) 与包围盒相交剪枝
- 10.5 核心算法二：最近邻搜索 (NNS) 与 KNN 检索
 - 10.5.1 超球与超矩形相交判定的几何原理
 - 10.5.2 剪枝不变量：垂直距离即为全局下界
 - 10.5.3 优先队列维护 K 近邻状态机
- 10.6 维度灾难 (Curse of Dimensionality) 与工业界高维检索演进
- 本章小结
- 思考题与经典大厂面试题

第 10 章 多维搜索树 (KD 树)

难度：★★★★☆ | 代码：kdtree.cpp

前置知识：第 5 章 二叉树遍历，第 6 章 BST 的比较与分治，第 7 章 优先队列维护 Top-K。

学习目标：1. 掌握从一维标量搜索跨越到 (k) 维多维向量空间的拓扑思维；2. 理解 KD 树利用正交超平面逐层剖分空间的几何本质；3. 熟练掌握使用 `std::nth_element` 以 $O(n \log n)$ 构建平衡 KD 树；4. 掌握基于“超球与超矩形相交判定”的最近邻搜索 (NNS) 与 KNN 剪枝机制；5. 理解高维空间下的“维度灾难”以及现代向量数据库 (如 HNSW) 的演进方向。

10.1 多维空间索引的挑战：为什么 1 维 BST 失效？

在传统的业务场景中，我们检索的键通常是一维标量（如 `user_id = 10086`，或 `age BETWEEN 20 AND 30`）。BST 依赖一维数轴上的全序关系 (Total Order)：对任意两个数 (a) 和 (b)，必定有 $(a < b)$ 或 $(a > b)$ 或 $(a = b)$ 。

然而，现实世界充满了**多维空间数据**：
- **地理信息系统 (GIS)**：查找经纬度 $((x, y))$ 范围内距离当前位置最近的 5 家加油站；
- **游戏物理引擎**：三维空间 $((x, y, z))$ 中判定角色与障碍物的碰撞检测；
- **机器学习 / 推荐系统**：在 128 维人脸特征向量中查找与目标人脸欧氏距离最近的 Top-K 候选者。

为什么不能简单建立多个 1 维 BST？

若对 (x) 坐标建立 `BST_x`，对 (y) 坐标建立 `BST_y`：
- 在矩形范围 $[[x_1, x_2] \times [y_1, y_2]]$ 搜索时，`BST_x` 只能过滤 (x) 维，返回所有满足 (x) 范围的点，然后再逐个检验 (y)；
- 若点集在一条对角线上，`BST_x` 返回的点可能多达

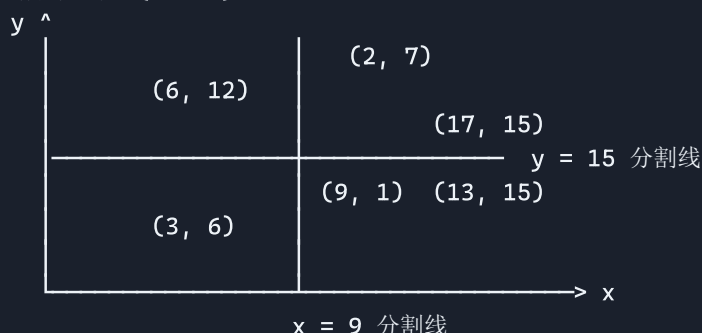
($O(n)$), 导致索引完全失效!

我们需要一种能够“同时在多个维度上交替切割空间”的全新数据结构——KD 树 (k-Dimensional Tree) !

10.2 KD 树的几何本质：正交超平面的空间剖分

1975 年, Jon Bentley 发明了 KD 树。其核心思想是: - 它是对标准 BST 的高维推广; - 树的每一个节点不仅存储一个 (k) 维坐标点, 还隐式对应着一个正交于某个坐标轴的超平面 (Hyperplane); - 该超平面将当前空间区域一分为二: 处于该超平面“左侧”的点归入左子树, 处于“右侧”的点归入右子树。

2D 平面剖分示意图 ($k = 2$):



- **树根 (第 0 层)**: 选择 (x) 轴, 用垂直于 (x) 轴的直线 ($x = 9$) 将平面划分为左右半区;
- **第 1 层节点**: 选择 (y) 轴, 在左右半区分别用水平直线 ($y = \dots$) 将空间再次横向剖分;
- **几何含义**: 随着深度增加, 整个多维空间被逐级细分为相互嵌套、互不重叠的轴对齐超矩形包围盒 (Axis-Aligned Bounding Box, AABB) !

10.3 KD 树的构建算法与维度切分策略

10.3.1 轮换切分 vs 最大方差优先切分

1. 经典策略：维度周期轮换 (Round-Robin)

深度为 (d) 的节点, 其分割维度直接定义为: $\text{split_dim} = d \bmod k$ 在 2D 空间中: 第 0 层切 (x), 第 1 层切 (y), 第 2 层切 (x), 第 3 层切 (y), 循环往复。

2. 进阶策略：最大方差优先 (Maximum Spread / Variance)

计算当前候选点集在各个维度上的坐标方差。优先选择数据离散程度最大 (方差最大) 的维度进行切分。这种策略能有效避免因为某个维度数据扎堆而导致的细长畸形空间剖分。

10.3.2 中位数选取与 `std::nth_element` 线性建树优化 ($O(n \log n)$)

为了让 KD 树尽可能平衡 (深度为 ($O(\log n)$)), 分割点必须选取当前维度坐标的中位数 (Median)。

- 若使用快速排序对点集全排序, 建树时间为 ($O(k \cdot n \log^2 n)$);
- **现代 C++ 优化**: 使用 `std::nth_element` ! 它基于快速选择算法 (QuickSelect), 仅需 ($O(n)$) 线性时间即可将中位数放置在正中间, 并将较小者置于左侧、较大者置于右侧 (无需完全有序)。
- **建树总时间复杂度**: 根据主定理 ($T(n) = 2T(n/2) + O(n) \implies O(n \log n)$)!

```

struct Point {
    std::vector<double> coords;
};

struct KNode {
    Point point;
    int axis; // 本节点切分的维度
    KNode *left, *right;
    KNode(Point p, int a) : point(p), axis(a), left(nullptr), right(nullptr) {}
};

KNode* buildKDTree(std::vector<Point>& points, int depth, int k) {
    if (points.empty()) return nullptr;

    int axis = depth % k;
    int mid = points.size() / 2;

    // O(n) 线性选择中位数
    std::nth_element(points.begin(), points.begin() + mid, points.end(),
        [axis](const Point& a, const Point& b) {
            return a.coords[axis] < b.coords[axis];
        });

    KNode* node = new KNode(points[mid], axis);

    std::vector<Point> leftPoints(points.begin(), points.begin() + mid);
    std::vector<Point> rightPoints(points.begin() + mid + 1, points.end());

    node->left = buildKDTree(leftPoints, depth + 1, k);
    node->right = buildKDTree(rightPoints, depth + 1, k);

    return node;
}

```

10.4 核心算法一：范围查询（Range Query）与包围盒相交剪枝

问题：给定一个超矩形范围（如 $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ ），求落在其中的所有点。

递归搜索规则：

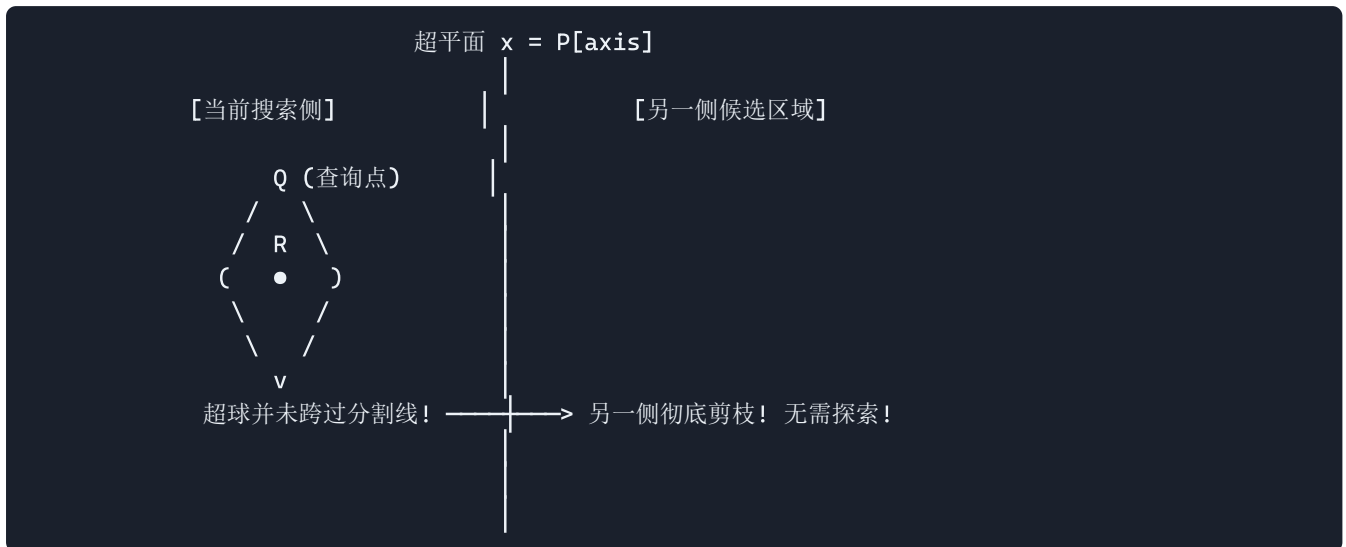
1. 若当前节点所在点落在目标矩形内，加入结果集；
2. 检查左子树代表的包围盒是否与目标矩形相交：
若相交，递归搜索左子树；否则整棵左子树直接【剪枝】跳过！
3. 检查右子树代表的包围盒是否与目标矩形相交：
若相交，递归搜索右子树；否则整棵右子树直接【剪枝】跳过！

10.5 核心算法二：最近邻搜索（NNS）与 KNN 检索

最近邻搜索（Nearest Neighbor Search, NNS） 是 KD 树最惊艳的应用：给定查询点 (Q)，在集合中找出欧几里得距离最近的点。

10.5.1 超球与超矩形相交判定的几何原理

设当前找到的全局最短距离为 (R) (即当前以 (Q) 为球心、半径为 (R) 的搜索超球内暂未发现更近的点)。



10.5.2 剪枝不变量：垂直距离即为全局下界

定理：

查询点 (Q) 到分割超平面的垂直距离 (即一维坐标差绝对值 $|Q[\text{axis}] - P[\text{axis}]|$)，是 (Q) 到超平面另一侧区域内任何点的距离的严格数学下界！ $[\text{Distance}(Q, \text{AnyPointInOtherSide}) \geq |Q[\text{axis}] - P[\text{axis}]|]$ 剪枝决策： - 若 $|Q[\text{axis}] - P[\text{axis}]| \geq R$ ：说明即使走到分割线边缘，距离都至少是垂直距离，已经大于等于当前最优解 (R)！因此另一侧的子树中绝对不可能存在更近的点，整棵子树立即剪掉！ - 仅当 $|Q[\text{axis}] - P[\text{axis}]| < R$ 时，超球才“跨越”了分割线，必须进入另一侧子树递归搜寻。

10.5.3 优先队列维护 K 近邻状态机

在求前 (K) 个最近邻 (KNN) 时，只需维护一个大小为 (K) 的大顶堆 (存放当前找到的最近 (K) 个点)： - 堆顶即为当前 Top-K 中的“最远距离” (门槛值 (R))； - 剪枝判断的标准就是看垂直距离是否小于堆顶距离； - 一旦找到更近的点，淘汰堆顶，压入新点并下沉！

10.6 维度灾难 (Curse of Dimensionality) 与工业界高维检索演进

KD 树在低维空间 ($k \leq 10$) 表现极其优异，平均时间复杂度为 $O(\log n)$ 。然而在现代 AI 应用中，特征维度往往高达 128、512 甚至 1536 维。

为什么高维下 KD 树会退化？

- 在 (k) 维超空间中，一个超立方体有 (2^k) 个“角落”；
- 随着维度 (k) 的增大，高维超球的体积占外接超立方体体积的比例指数级趋近于 0！
- 几何效应导致：查询超球几乎必然与几乎所有的分割超平面相交，剪枝条件彻底失效，算法退化为全树遍历的 $O(n)!$

工业界高维近似最近邻 (ANN) 演进:

1. **球树 (Ball Tree)** : 以超球而非超矩形分割空间, 对高维流形分布更具鲁棒性;
2. **局部敏感哈希 (LSH)** : 通过哈希函数将高维相近点映射到同一桶;
3. **基于图的近邻索引 (HNSW / Hierarchical Navigable Small World)** : 当前所有主流向量数据库 (Milvus、Pinecone、Faiss) 的核心引擎, 以可导航小世界图网络在毫秒级内实现高维向量检索。

本章小结

1. **正交剖分是多维索引的基石**: KD 树将 1 维二分查找升维为多维空间的正交超平面切分。
2. **中位数保障树高平衡**: 借助 `std::nth_element`, 以 $O(n \log n)$ 线性对数时间完成整树构建。
3. **垂直距离下界是剪枝灵魂**: 超球与超平面的几何距离比对, 赋予了 NNS 与 KNN 惊人的分支剪枝能力。
4. **理性认识维度灾难**: KD 树是低维空间检索 (GIS、游戏物理引擎) 的神器, 而高维 AI 向量检索应积极拥抱 HNSW 与图索引。

思考题与经典大厂面试题

1. **2D 范围查询的时间复杂度界限**: 在包含 (n) 个点的 2D KD 树中, 证明任意正交矩形范围查询的最坏时间复杂度为 $O(\sqrt{n} + k)$, 其中 (k) 为返回的结果点数。
2. **点插入与删除导致的失衡**: 如果在构建好的 KD 树上频繁执行动态插入和删除, 树形结构可能会严重倾斜。工业界通常如何借鉴替罪羊树 (Scapegoat) 思想对 KD 树进行局部重平衡?
3. **KNN 搜索中进入分支的先后顺序**: 在最近邻搜索中, 应当先进入包含查询点 (Q) 的同侧子树, 还是先进入异侧子树? 为什么这一顺序对于提升剪枝效率至关重要?

<- 第 10 章 多维搜索树 (KD 树) | 课程目录 | 第 12 章 哈希表 ->

所属模块: 第五部分: 图
对应实现: graph.cpp

本章导航

- 11.1 图论基础: 从树到多对多网络
- 11.2 图的三大经典存储范式: 矩阵、邻接表与链式前向星
- 11.3 深度与广度优先遍历 (DFS vs BFS) 及四类边拓扑剖析
- 11.4 拓扑排序 (Topological Sort) : DAG 的线性化调度
- 11.5 最小生成树 (MST) : 切割定理与 Prim vs Kruskal
- 11.6 最短路径四大算法全家桶与松弛技术
 - 11.6.1 Dijkstra 算法: 贪心正确性与非负权限制剖析
 - 11.6.2 Bellman-Ford 与 SPFA: 负权边支持与负环检测

- 11.6.3 Floyd-Warshall：动态规划求全源最短路
- 11.7 工程项目管理之魂：AOE 网与关键路径（CPM）大题手算演练
- 11.8 图算法工程技术选型雷达
- 本章小结
- 思考题与经典大厂面试题

第 11 章 图论算法

难度：★★★★★ | 代码：graph.cpp

前置知识：第 3 章 队列（BFS）与栈（DFS），第 7 章 堆（Dijkstra / Prim 优先队列优化），第 13 章 并查集（Kruskal 环检测）。

学习目标：1. 掌握邻接矩阵、邻接表以及高性能“链式前向星”的内存布局与访问开销；2. 掌握 DFS 树中四类边（树边、回边、前向边、横叉边）的有向图拓扑含义与环检测；3. 深刻理解贪心切割定理（Cut Property）在最小生成树中的指导作用；4. 透彻理解 Dijkstra 贪心松弛界，掌握负权边导致其失效的底层原因；5. 熟练掌握 AOE 网关键路径中 `etv`，`ltv`，`ete`，`lte` 四大数组的手算推导。

11.1 图论基础：从树到多对多网络

如果说树是“层级分明、无环连接”的特殊图（ n 个节点恰好 $(n-1)$ 条边），那么图（**Graph, $(G = (V, E))$** ）则是表达客观世界实体间“多对多”复杂关系的通用数学语言。- **顶点集（Vertex Set, (V)** ）：实体的集合， $(|V| = n)$ ；- **边集（Edge Set, (E)** ）：关系的集合， $(|E| = m)$ ；- **有向图 vs 无向图**：边是否存在方向性；- **权（Weight）**：边上的标量值（距离、费用、时延、容量）；- **简单路径 vs 环（Cycle）**：顶点不重复出现的路径称为简单路径；起点终点重合且边数 (≥ 3) 的简单路径构成环。

11.2 图的三大经典存储范式：矩阵、邻接表与链式前向星

选择何种数据结构存储图，直接决定了图算法的常数因子与渐进复杂度：

【1. 邻接矩阵（Adjacency Matrix）】

空间： $O(V^2)$
判两点相邻： $O(1)$
遍历某点所有出边： $O(V)$
适用：稠密图 $(E \approx V^2)$

```

0  1  2  3
0 [0, 5, 2, 0]
1 [0, 0, 0, 1]
2 [0, 0, 0, 6]
3 [0, 0, 0, 0]
```

【2. 邻接表（Adjacency List）】

空间： $O(V + E)$
判两点相邻： $O(\text{degree}(u))$
遍历某点所有出边： $O(\text{degree}(u))$
适用：稀疏图 $(E \ll V^2)$

```

0 —> [ 1, w=5 ] —> [ 2, w=2 ]
1 —> [ 3, w=1 ]
2 —> [ 3, w=6 ]
3 —> nullptr
```

3. 高性能存储王牌：链式前向星 (Forward Star)

在算法竞赛和追求极限性能的工业引擎中，`std::vector<Edge> adj[V]` 的频繁小对象分配会产生严重的缓存未命中 (Cache Miss)。链式前向星利用三个扁平的一维静态数组，实现了对邻接表的零指针、缓存紧凑化压缩：

```
const int MAXV = 10000;
const int MAXE = 100000;

int head[MAXV]; // head[u] 存储顶点 u 的第一条出边在 edges 数组中的下标
int to[MAXE];   // to[e] 存储第 e 条边的终点
int weight[MAXE];
int nxt[MAXE];  // nxt[e] 存储与第 e 条边同起点的下一条边下标
int edgeCount = 0;

void addEdge(int u, int v, int w) {
    to[edgeCount] = v;
    weight[edgeCount] = w;
    nxt[edgeCount] = head[u]; // 头插法串联同起点的边
    head[u] = edgeCount++;
}

// 遍历顶点 u 的所有出边：极其迅速！
for (int e = head[u]; e != -1; e = nxt[e]) {
    int v = to[e];
    int w = weight[e];
    // 处理邻接点 v
}
```

11.3 深度与广度优先遍历 (DFS vs BFS) 及四类边拓扑剖析

算法	核心数据结构	空间复杂度	核心特性	经典适用场景
BFS (广度优先)	队列 (Queue)	$O(V)$ (最宽层节点数)	逐层扩散，具有最短步数保证	无权图单源最短路径、连通块染色、二分图判定
DFS (深度优先)	递归调用栈 / 显式栈	$O(V)$ (最深递归深度)	一路深入到底，撞墙回溯	连通性分析、拓扑排序、强连通分量 (Tarjan)、回溯搜索

DFS 生成森林中的四类边 (有向图深层拓扑)：

当对有向图运行 DFS 时，每个顶点被打上两个时间戳：发现时间 `d[u]` 和完成时间 `f[u]`。此时图中的每条边 $((u, v))$ 被严格归入四类之一：

- 1. 树边 (Tree Edge)：DFS 搜索树的实际分支边，直接通往未访问的白色节点。
- 2. 回边 (Back Edge)：由子孙节点指向 DFS 树中的祖先节点 $\longrightarrow$ 【存在回边 $\iff$ 图中必有有向环！】
- 3. 前向边 (Forward Edge)：由祖先节点直接指向非直接后代的树下部节点。
- 4. 横叉边 (Cross Edge)：连接两个互无祖先关系的平行分支，或跨越不同连通分量。

无向图的重要定理：在无向图的 DFS 生成树中，**绝不会出现前向边和横叉边**，所有的边非此即彼，要么是树边，要么是回边！

11.4 拓扑排序 (Topological Sort) : DAG 的线性化调度

有向无环图 (Directed Acyclic Graph, DAG) 是任务依赖调度的通用数学模型 (例如：大学课程选修依赖、Makefile 编译依赖、任务流调度)。

拓扑排序：将 DAG 中所有顶点排成一个线性序列，使得对于任意边 $(u \rightarrow v)$ ， (u) 在序列中必须排在 (v) 的前面。

Kahn 算法 (基于入度的 BFS 队列法)：

1. 统计每个顶点的入度 (`in_degree`)；
2. 将所有入度为 0 的顶点压入队列 (入度为 0 意味着无前置依赖，可立即执行)；
3. 当队列非空时：
4. 弹出队头 (u) ，加入拓扑序列；
5. 遍历 (u) 的所有邻接出边 $(u \rightarrow v)$ ，令 (v) 的入度减 1 (`--in_degree[v]`)；
6. 若 (v) 的入度衰减为 0，将其压入队列；
7. **判环不变量：**若最终拓扑序列中的顶点数 $(< |V|)$ ，说明图中存在有向环，拓扑排序不存在！

11.5 最小生成树 (MST) : 切割定理与 Prim vs Kruskal

问题：给定连通无向带权图 $(G = (V, E))$ ，选出 $(V - 1)$ 条边，在保证图依然连通的前提下，使得选出的边权之和最小。

贪心选择的数学基石：切割定理 (Cut Property)

- 将顶点集 (V) 任意划分为两个互不相交的子集 (S) 和 $(V - S)$ ，连接这两个集合的所有边构成的集合称为一个**割 (Cut)**；
- **切割定理：**跨越该割的所有边中，**权值最小的那条边 (轻边, Light Edge)** 必然属于某棵最小生成树！

【Prim 算法 (加点法)】

- 思想：维护一个不断长大的树点集 S
- 策略：每次寻找跨越 S 与 $V-S$ 的最短边
- 复杂度： $O(E \log V)$ (堆优化)
- 最佳战场：稠密图 ($E \approx V^2$)

【Kruskal 算法 (加边法)】

- 思想：全局边权升序排序，贪心选边
- 策略：用并查集判环，若两端不在同一连通块则加入
 - 复杂度： $O(E \log E)$
 - 最佳战场：稀疏图 ($E \ll V^2$)

11.6 最短路径四大算法全家桶与松弛技术

松弛操作 (Relaxation) 是所有最短路径算法的共同原子操作：

```
void relax(int u, int v, int weight) {
    if (dist[v] > dist[u] + weight) {
        dist[v] = dist[u] + weight; // 经由 u 走向 v 能够缩短总路程
    }
}
```

11.6.1 Dijkstra 算法：贪心正确性与非负权限制剖析

Dijkstra 算法维护一个已经确定最短距离的集合 (S)：1. 初始令 `dist[src] = 0`，其余置为 (∞)；2. 每次从未确定集合 ($V - S$) 中选取当前 `dist` 最小的顶点 (u) 加入 (S) (利用小顶堆优化为 ($O(\log V)$))；3. 用 (u) 的所有出边对邻接点进行松弛；4. 重复步骤 2 和 3 直到所有顶点都被处理。

【为什么 Dijkstra 面对负权边必然崩溃？】

考虑网络：A $\xrightarrow{(5)}$ B, A $\xrightarrow{(1)}$ C $\xrightarrow{(-10)}$ B

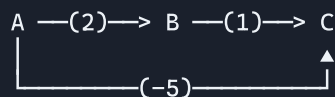
第 1 轮：从 A 出发，`dist[B]=5`，`dist[C]=1`

第 2 轮：贪心地选取未锁定的最小者 C (`dist=1`) 锁定！

松弛 C→B: `dist[B] = min(5, 1 + (-10)) = -9`

第 3 轮：锁定 B (`dist=-9`)

看似正常？但如果网络是：



Dijkstra 在第 1 轮会把 B 锁定为 2 (因为 $2 < \infty$)，随后 C 被松弛为 3。

但实际存在 A 直接向 C 走 -5 的捷径！一旦节点被出堆锁定，Dijkstra 永不复查，彻底失误！

11.6.2 Bellman-Ford 与 SPFA：负权边支持与负环检测

- **核心逻辑**：对图中的全部 (E) 条边执行整整 ($V - 1$) 轮松弛循环。
- **为什么是 ($V - 1$) 轮？**
在一个不含负环的图中，任何单源最短简单路径至多包含 ($V - 1$) 条边。第 (k) 轮松弛保证了所有至多包含 (k) 条边的最短路径均被正确求出。
- **负环检测**：若在第 (V) 轮松弛中，依然有边能够被成功松弛，说明图中必然存在权值和为负数的环路 (Negative Cycle)！因为只有负环才能无限次缩短距离！
- **SPFA 算法**：利用队列仅将“上一轮 `dist` 发生减小的节点”入队去松弛邻居。平均复杂度 ($O(kE)$)，但面对网格图等最坏数据会退化为 ($O(VE)$)。

11.6.3 Floyd-Warshall：动态规划求全源最短路

求任意两点 (i) 到 (j) 的最短路径。- **状态定义**：(`dp[k][i][j]`) 表示只允许经过前 (k) 个顶点作为中间过渡节点时，从 (i) 到 (j) 的最短路径。- **状态转移方程**：[`dp[k][i][j] = min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])`] - **空间压缩**：由于 (`dp[k]`) 仅依赖 (`dp[k-1]`)，最外层 (k) 的维度可以直接抹去：

```

for (int k = 0; k < V; ++k) {
    for (int i = 0; i < V; ++i) {
        for (int j = 0; j < V; ++j) {
            if (dist[i][k] != INF && dist[k][j] != INF) {
                dist[i][j] = std::min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}
}

```

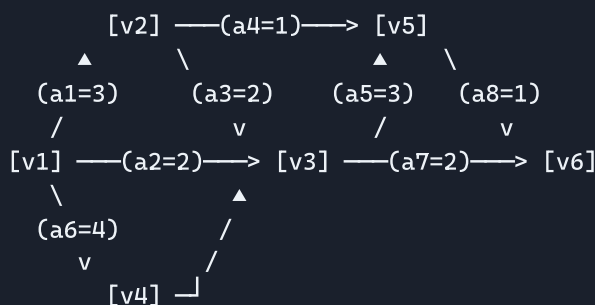
⚠️ **最外层循环必须是 (k)!** 若把 (k) 放在内层，动态规划的无后效性将被彻底打破!

11.7 工程项目管理之魂：AOE 网与关键路径（CPM）大题手算演练

在现代软件工程、建筑工程与芯片设计中，**关键路径法（Critical Path Method, CPM）** 是计算总工期和确定瓶颈工序的核心工具。

- **AOE 网（Activity On Edge Network）**：
- **顶点表示事件（Event）**（某阶段完成的里程碑时刻）；
- **有向边表示活动（Activity）**，边权表示持续时间。
- **四大关键数组（考研与大厂手算标准）**：
- **etv[k]**（Earliest Time of Vertex）：事件 (k) 的**最早发生时间**；
- **ltv[k]**（Latest Time of Vertex）：事件 (k) 的**最晚允许发生时间**；
- **ete[r]**（Earliest Time of Edge）：活动 $(a_r = \langle u, v \rangle)$ 的**最早开始时间** ($ete[r] = etv[u]$)；
- **lte[r]**（Latest Time of Edge）：活动 $(a_r = \langle u, v \rangle)$ 的**最晚允许开始时间** ($lte[r] = ltv[v] - \text{weight}$)。
- **关键活动**：满足 **$lte[r] - ete[r] == 0$** （即松弛时间 / 时差为 0）的活动！所有关键活动串联起来的路径即为**关键路径（工期瓶颈）**！

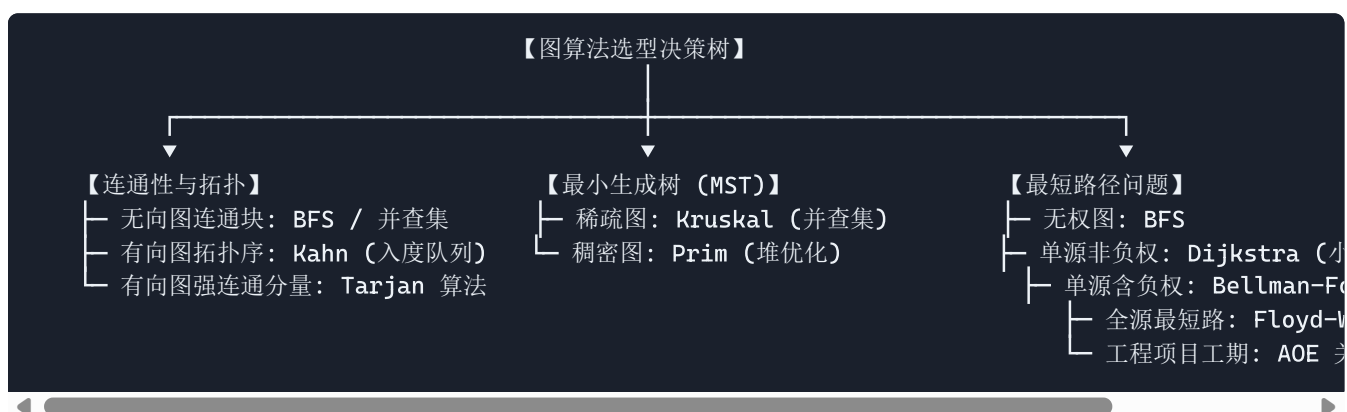
完整手算追踪示例：



1. **正向拓扑推进求 etv**（取各前驱的最大值）：
2. **$etv[v1] = 0$**
3. **$etv[v2] = etv[v1] + 3 = 3$**
4. **$etv[v4] = etv[v1] + 4 = 4$**

5. $etv[v3] = \max(etv[v1]+2, etv[v2]+2, etv[v4]+0) = \max(2, 5, 4) = 5$
6. $etv[v5] = \max(etv[v2]+1, etv[v3]+3) = \max(4, 8) = 8$
7. $etv[v6] = \max(etv[v5]+1, etv[v3]+2) = \max(9, 7) = 9$ $\implies$ 全工程总工期为 9!
8. 逆向拓扑倒推求 **ltv** (取各后继的最小值) :
9. $ltv[v6] = 9$
10. $ltv[v5] = ltv[v6] - 1 = 8$
11. $ltv[v3] = \min(ltv[v5] - 3, ltv[v6] - 2) = \min(5, 7) = 5$
12. $ltv[v2] = \min(ltv[v5] - 1, ltv[v3] - 2) = \min(7, 3) = 3$
13. $ltv[v1] = \min(ltv[v2]-3, ltv[v3]-2, ltv[v4]-4) = 0$
14. 活动时差判定:
15. 活动 $(a_1 = \langle v1, v2 \rangle)$: $ete = 0, lte = 3 - 3 = 0$ $\implies$ 关键活动!
16. 活动 $(a_3 = \langle v2, v3 \rangle)$: $ete = 3, lte = 5 - 2 = 3$ $\implies$ 关键活动!
17. 活动 $(a_5 = \langle v3, v5 \rangle)$: $ete = 5, lte = 8 - 3 = 5$ $\implies$ 关键活动!
18. 活动 $(a_8 = \langle v5, v6 \rangle)$: $ete = 8, lte = 9 - 1 = 8$ $\implies$ 关键活动!
19. 关键路径为: $(v1 \rightarrow v2 \rightarrow v3 \rightarrow v5 \rightarrow v6)$!

11.8 图算法工程技术选型雷达



本章小结

1. 链式前向星是性能利器: 以一维数组模拟邻接链表, 最大化 CPU 缓存命中率。
2. 生成树与回边: DFS 生成树不仅是搜索路径, 回边的存在是有向图判环的充要条件。
3. 松弛操作统领最短路: Dijkstra 用贪心锁死非负权, Bellman-Ford 用 $(V-1)$ 轮全覆盖化解负权, Floyd 以外层阶段 (k) 贯通全源。
4. 关键路径是时差的消解: 最早与最迟时间的碰撞锁定了工程进度的绝对瓶颈。

思考题与经典大厂面试题

1. 差分约束系统 (Difference Constraints) : 给定一组不等式 $(x_i - x_j \leq c_k)$, 如何将其转化为有向图的最短路径问题? 系统无解对应于图论中的什么特征?

2. **关键路径的唯一性与优化**：一个 AOE 网中是否存在多条关键路径？若想缩短整个项目的总工期，仅仅压缩某一条关键路径上的某一个活动是否一定奏效？
3. **二分图的最大匹配 (Bipartite Matching)**：如何将二分图的最大匹配问题转化为网络流 (Network Flow) 的最大流问题？

[<- 第 11 章 图论算法](#) | [课程目录](#) | [第 13 章 并查集](#) ->

所属模块：第六部分：查找
对应实现：hashing.cpp

本章导航

- 12.1 散列映射的本质：从直接寻址到概率性 $O(1)$ 检索
- 12.2 鸽巢原理与生日悖论：为什么冲突不可避免？
- 12.3 优秀哈希函数的设计准则与经典算法
 - 12.3.1 除留余数法：为什么表大小必须取素数？
 - 12.3.2 乘法哈希法与黄金分割常数
 - 12.3.3 字符串哈希：BKDR 与雪崩效应 (Avalanche Effect)
- 12.4 冲突消解两大流派的深度较量
 - 12.4.1 分离链接法（拉链法）与 Java 8 红黑树化阈值
 - 12.4.2 开放寻址法：线性探测、二次探测与双重散列
 - 12.4.3 开放寻址的墓碑机制 (Tombstone) 与查找链不变量
- 12.5 负载因子 (Load Factor) 与渐进式扩容 (Incremental Rehashing)
- 12.6 完美哈希 (FKS 方案)：最坏 $O(1)$ 查询的数学构造
- 12.7 分布式系统基石：一致性哈希 (Consistent Hashing) 与虚拟节点
- 本章小结
- 思考题与经典大厂面试题

第 12 章 哈希表

难度：★★☆☆☆ | 代码：hashing.cpp

前置知识：第 1 章 均摊分析（动态扩容重哈希），第 2 章 链表与数组。

学习目标：1. 理解哈希表“用计算时间换取寻址空间”的设计哲学；2. 掌握除留余数法、乘法哈希及字符串哈希的数论原理；3. 掌握拉链法与开放寻址法（一次聚集、二次探测、双重散列）的利弊；4. 熟练运用墓碑机制保证开放寻址删除时的查找链不中断；5. 理解 FKS 完美哈希与一致性哈希在现代工程中的应用。

12.1 散列映射的本质：从直接寻址到概率性 $O(1)$ 检索

如果所有可能的键集合 (U) 规模很小 (如 ASCII 字符, ($|U| = 256$)), 我们可以直接建立一个大小为 256 的数组, 以字符的 ASCII 码为下标直接存取, 称为**直接寻址表 (Direct Addressing Table)**, 查询时间为纯粹的 ($O(1)$)。

但在真实业务中: - 身份证号、IPv6 地址、URL 字符串等键的理论空间 ($|U|$) 极为庞大 (如 18 位身份证号有 (10^{18}) 种可能); - 而我们实际存储的数据量 (n) 往往只有几十万或几百万 ($(n \ll |U|)$)。 - 直接开辟 (10^{18}) 大小的数组需要数万 PB 内存, 物理上完全不可行!

哈希表 (Hash Table / 散列表) 应运而生: - 建立一个容量仅为 ($m \approx O(n)$) 的紧凑桶数组; - 通过**哈希函数 ($h(\text{key})$)**, 将庞大宇宙中的键映射为 ($[0, m-1]$) 内的整数下标; - **核心代价**: 由于空间被大幅压缩, 不同的键可能被映射到相同的槽位, 这就是**哈希冲突 (Hash Collision)**!

12.2 鸽巢原理与生日悖论：为什么冲突不可避免？

- **鸽巢原理 (Pigeonhole Principle)**: 若有 (n) 只鸽子飞入 (m) 个巢, 且 ($n > m$), 则至少有一个巢里有 2 只及以上的鸽子。由于 ($|U| > m$), 冲突在数学上不可避免。
- **生日悖论 (Birthday Paradox)**: 在一个只有 23 人的房间里, 至少有两个人同一天生日的概率竟然超过了 50%!

数学推导: 23 个人生日全不相同的概率为: $[P(\text{全不同}) = \frac{365}{365} \times \frac{364}{365} \times \dots \times \frac{343}{365} \approx 0.4927 \implies P(\text{至少两人相同}) = 1 - 0.4927 = 50.73\%]$

这表明: **即使哈希表还非常空旷, 发生冲突的概率也远比直觉预想的高得多!** 因此, 设计优良的哈希函数与冲突消解机制是哈希表的立足之本。

12.3 优秀哈希函数的设计准则与经典算法

一个理想的哈希函数必须满足**简单均匀散列假设 (Simple Uniform Hashing)**: 任何给定的键, 均等可能地映射到 (m) 个槽位中的任意一个, 且与其他键的映射相互独立。

12.3.1 除留余数法：为什么表大小必须取素数？

公式: ($h(k) = k \bmod m$)

表大小 (m) 必须选素数, 且不能接近 2 的幂次!

【致命反例：若 m 选合数 (如 $m = 12$)】

假设输入的键有某种规律 (如大量键是 4 或 6 的倍数):

$keys = [6, 12, 18, 24, 30, 36]$

$h(k) = k \% 12$ 后的结果: $[6, 0, 6, 0, 6, 0] \longrightarrow$ 所有的键被严重压缩在 0 和 6 两个槽中!

其余 10 个槽位全部空置, 冲突率高达 100%!

【若 m 选素数 (如 $m = 13$)】

$h(k) = k \% 13$ 后的结果: $[6, 12, 5, 11, 4, 10] \longrightarrow$ 6 个键均匀散布在 6 个完全不同的槽位中!

素数能够最大限度打破输入数据中潜在的公约数周期性, 消除周期聚集。

12.3.2 乘法哈希法与黄金分割常数

由 Donald Knuth 提出：1. 选取一个常数 ($A \in (0, 1)$)；2. 计算 $(k \cdot A)$ 并提取其小数部分 $((k \cdot A) \bmod 1)$ ；3. 乘以 (m) 并向下取整： $h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$ Knuth 证明，当选取黄金分割比的共轭数 ($A = \frac{\sqrt{5} - 1}{2} \approx 0.6180339887\dots$) 时，散列分布最为均匀。该方法对 (m) 的选取不敏感，通常令 $(m = 2^p)$ 以便通过位运算加速。

12.3.3 字符串哈希：BKDR 与雪崩效应 (Avalanche Effect)

字符串作为键在现代开发中最为普遍。著名的 BKDR Hash 使用多项式滚动累加：

```
uint64_t BKDRHash(const std::string& str) {
    uint64_t seed = 131; // 也可以是 31, 1313, 13131 等奇素数
    uint64_t hash = 0;
    for (char ch : str) {
        hash = hash * seed + static_cast<unsigned char>(ch);
    }
    return hash;
}
```

雪崩效应：输入的字符串即使只变动一个比特（例如 "test1" 变为 "test2"），输出的哈希值也应当有大约 50% 的比特位发生剧烈翻转，避免相邻字符串产生相近的哈希值。

12.4 冲突消解两大流派的深度较量

12.4.1 分离链接法（拉链法）与 Java 8 红黑树化阈值

每个槽位 (Bucket) 维护一个链表。当产生冲突时，直接将新键值对挂接在链表头部或尾部。 - **优点：**结构简单，对负载因子 (α) 容忍度高 (α 可以超过 1.0)； - **缺点：**链表节点离散分配，缺乏 CPU 缓存局部性；每个节点额外付出指针开销。 - **Java 8 HashMap 的工程演进：**

当某一个桶内的链表长度达到 8 且表总容量 (≥ 64) 时，该链表将**自动蜕变为**一棵**红黑树**！将最坏情况下的查找时间从 $(O(n))$ 强制截断为 $(O(\log n))$ ，有效抵御了针对哈希表的拒绝服务攻击 (Hash DoS)。

12.4.2 开放寻址法：线性探测、二次探测与双重散列

所有元素直接存储在桶数组内（无指针）。发生冲突时，通过某种规则在表中探测下一个可用槽位： $h_i(k) = (h(k) + d_i) \bmod m, \quad i = 0, 1, 2, \dots$

探测方式	探测增量序列 (d _i)	核心特性与缺陷
线性探测 (Linear Probing)	(0, 1, 2, 3, \dots)	一次聚集 (Primary Clustering) ：连续占用的槽位会迅速滚雪球般扩大，导致后续插入探测链越来越长。
二次探测 (Quadratic Probing)	(0, 1^2, -1^2, 2^2, -2^2, \dots)	跳跃探测，化解了一次聚集；但具有相同初始哈希值的键仍会走相同路径（二次聚集）。
双重散列 (Double Hashing)	(i \cdot h_2(k))	增量由第二个独立哈希函数 (h ₂ (k)) 决定，不同键探测路径完全不同。 要求 (h₂(k)) 与 (m) 互质。

12.4.3 开放寻址的墓碑机制 (Tombstone) 与查找链不变量

在开放寻址表中，**绝对不能直接把被删除的槽位清空为 EMPTY**！

查找链不变量破裂反例：
假设 key1 与 key2 发生冲突，key1 落在槽 3，key2 被线性探测推到了槽 4。
此时若直接把槽 3 的 key1 清空（设为 EMPTY）：
接下来查找 key2：计算 h(key2) = 3，检查槽 3 发现是 EMPTY！
算法立刻误判：“该键不存在”，提前退出！槽 4 里的 key2 永远失联！

解决方案：墓碑机制 (Lazy Deletion / Tombstone)：

- 节点状态三值化：EMPTY（从未占用）、OCCUPIED（有效占用）、DELETED（已删除/墓碑）；
- 查找时：遇到 DELETED 视同有占位，继续向后探测；只有遇到 EMPTY 才能断定键不存在；
- 插入时：遇到 DELETED 或 EMPTY 均可作为空闲位置填入新元素。

12.5 负载因子 (Load Factor) 与渐进式扩容 (Incremental Rehashing)

负载因子 (Load Factor)：[$\alpha = \frac{n}{m} = \frac{\text{表中元素总数}}{\text{散列表总容量}}$] 在开放寻址法中，当 ($\alpha > 0.75$) 后，查找失败的平均探测次数呈指数级暴增（**膝盖效应**）。因此，工业级哈希表通常在 ($\alpha \geq 0.7 \sim 0.75$) 时触发扩容（容量翻倍）。

Redis dict 的渐进式 Rehash 艺术：

若哈希表存有 1000 万个键，单次重分配内存并拷贝所有键需要耗费数百毫秒，导致单线程的 Redis 服务严重卡顿（Stop-the-world）。

Redis 解决方案：

1. 维护两张哈希表：ht[0]（旧表）和 ht[1]（新表），以及游标 rehashidx = 0；
2. 在每次接收到客户端的增删改查请求时，顺手将 ht[0] 在 rehashidx 索引上的单个槽位数据搬运到 ht[1]，然后 ++rehashidx；
3. 将一次性的大开销均摊到成千上万次日常请求中，做到纳秒级平滑过渡！

12.6 完美哈希 (FKS 方案)：最坏 O(1) 查询的数学构造

由 Fredman, Komlós, Szemerédi 于 1984 年提出（FKS 完美哈希）。在键集合静态已知（如编程语言保留字、编译器关键字字典）的场景下，可以实现**最坏情况下严格 O(1) 查询**，且**总体空间严格为 O(n)**！

第一层哈希（槽数 $m = n$ ）:



第二层哈希表（容量 $S_1 = n_1^2 = 4$ ）
[保证 0 冲突!]

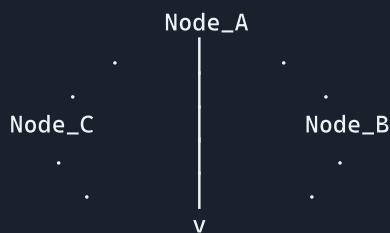
第二层哈希表（容量 $S_3 = n_3^2 = 9$ ）
[保证 0 冲突!]

- **核心数学定理**：若将 (k) 个元素散列到大小为 (k^2) 的表中，根据生日悖论，**不发生任何冲突的概率** $(> 1/2)!$
- 若发生冲突，只需重新随机挑选第二层哈希函数再次尝试，期望常数次即可找到零冲突哈希表。

12.7 分布式系统基石：一致性哈希（Consistent Hashing）与虚拟节点

在分布式缓存集群（如 Memcached、Redis Cluster）中，若使用简单取模 $h(key) \% N$ ： - 一旦某台服务器宕机（机器数从 (N) 变为 $(N-1)$ ），**几乎 99% 的键的映射位置全部失效**，引发毁灭性的缓存雪崩！

一致性哈希环 $(0 \sim 2^{32} - 1)$:



每个 **Key** 沿环顺时针寻找距离最近的物理节点。

节点宕机或扩容时，仅影响该节点逆时针方向的相邻子集，其余节点完全不受波及！

- **虚拟节点（Virtual Nodes）**：为每个物理节点分配数十个虚拟副本节点（如 **NodeA#1**，**NodeA#2**）均匀撒在环上，彻底消除数据分布倾斜。

本章小结

1. **散列是用计算换空间**：哈希函数将无限键空间投射到有限槽位，冲突消解是哈希表的灵魂。
2. **数论保驾护航**：除留余数选素数，双重散列求互质，黄金分割乘法消除周期分布。
3. **墓碑维持链式连续**：开放寻址不能直接赋空，惰性删除确保探测链不被腰斩。
4. **工业工程演进**：Java 8 树化防 DoS，Redis 渐进 Rehash 防卡顿，一致性哈希统领分布式集群。

思考题与经典大厂面试题

1. **二次探测的覆盖率证明**：在二次探测中，若表大小 (m) 为 $(4k+3)$ 型素数，证明增量序列 $(\pm 1^2, \pm 2^2, \dots)$ 能够恰好探测到全表中至少一半的槽位。

2. **布隆过滤器 (Bloom Filter)**：哈希表不仅能存储键值，还能只判断“是否存在”。请简述布隆过滤器如何利用多个哈希函数与位图 (BitMap) 实现空间极小、允许假阳性但不允许假阴性的集合判重。
3. **哈希碰撞 DoS 攻击防范**：攻击者若通过逆向工程构造出上万个具有完全相同哈希值的字符串向 Web 服务器发送 POST 请求，服务器将遭受什么后果？工业界编程语言（如 Python、Go、Rust）如何通过 SipHash 随机种子抵御此类攻击？

[<- 第 12 章 哈希表](#) | [课程目录](#) | [第 14 章 排序算法](#) ->

所属模块：第六部分：查找
对应实现：disjointset.cpp

本章导航

- [13.1 动态连通性问题与等价关系](#)
- [13.2 并查集的演进之路：从 Quick-Find 到 Quick-Union](#)
- [13.3 极简工程美学：单数组双重语义设计 \(parent 数组编码\)](#)
- [13.4 两大核心优化与双剑合璧](#)
 - [13.4.1 启发式合并：按大小 \(Size\) 或按秩 \(Rank\) 合并的树高对数证明](#)
 - [13.4.2 路径压缩 \(Path Compression\)：读操作就地扁平化](#)
- [13.5 复杂度奇迹：阿克曼函数与反阿克曼函数 \$\alpha\(n\)\$](#)
- [13.6 高阶拓展：带权并查集与可撤销并查集 \(Rollback DSU\)](#)
- [本章小结](#)
- [思考题与经典大厂面试题](#)

第 13 章 并查集

难度：★★☆☆☆ | 代码：disjointset.cpp

前置知识：[第 1 章 均摊分析](#)，[第 5 章 树与双亲表示法](#)。

学习目标： 1. 掌握动态连通性问题的模型抽象； 2. 理解单数组同时编码“父节点”与“集合大小”的极简工程设计； 3. 掌握按大小合并保证树高不超过 ($O(\log n)$) 的数学证明； 4. 掌握递归路径压缩的代码实现，理解读操作改善数据结构的惰性优化思想； 5. 了解带权并查集在多集合相对关系维护中的应用。

13.1 动态连通性问题与等价关系

在许多现实场景中，我们需要维护一个动态的“朋友圈”或“连通网络”： - **操作 1**: `union(p, q)` —— 在网络中连接两个实体 (p) 和 (q) (建立关系)； - **操作 2**: `connected(p, q)` / `find(p)` —— 查询实体 (p) 和 (q) 是否处于同一连通分支中。

在数学上，连通性是一种**等价关系 (Equivalence Relation)**，满足： 1. **自反性**：(p) 与自身连通； 2. **对称性**：若 (p) 与 (q) 连通，则 (q) 与 (p) 连通； 3. **传递性**：若 (p) 与 (q) 连通，且 (q) 与 (r) 连通，则 (p) 与 (r) 必然连通。

13.2 并查集的演进之路：从 Quick-Find 到 Quick-Union

【演进 1: Quick-Find 方案】

- 物理存储：数组 `id[n]`，每个位置直接存所属集合的唯一 ID (如帮派编号)
- `Find(p)`: `return id[p]`; $\rightarrow O(1)$ 极速
- `Union(p, q)`: 必须遍历整个数组，将所有等于 `id[p]` 的元素改为 `id[q]` $\rightarrow O(n)$ 缓慢!
- 瓶颈：连续 N 次合并需要 $O(N^2)$ ，面对海量数据直接瘫痪。

【演进 2: Quick-Union 方案】

- 物理存储：树形森林，每个节点只记录自己的父节点 `parent[p]`
- `Find(p)`: 顺着 `parent` 指针一路向上爬到根节点 $\rightarrow O(\text{树高})$
- `Union(p, q)`: 找到 `p` 和 `q` 的根节点 `rootP` 和 `rootQ`，直接令 `parent[rootP] = rootQ` $\rightarrow O(\text{树高})$
- 瓶颈：若无控制，树可能退化为瘦长的单链表，单次操作依然退化为 $O(n)$!

13.3 极简工程美学：单数组双重语义设计 (parent 数组编码)

在通用实现中，很多人会开两个数组：`int parent[N]` 和 `int size[N]`。然而，著名算法学家 Mark Allen Weiss 展示了一种**令人拍案叫绝的单数组双重编码方案**：

parent 数组的单一物理存储

↓
【若 `parent[i] ≥ 0`】
表示节点 `i` 不是根，其值为【父节点下标】

↓
【若 `parent[i] < 0`】
表示节点 `i` 本身就是【树根】!
且 `|parent[i]|` 恰好是该集合的【元素总数】!

```

class DisjointSet {
private:
    std::vector<int> parent; // 初始时所有元素均为 -1（每个节点自成一棵大小为 1 的树）

public:
    DisjointSet(int n) : parent(n, -1) {}

    // 判断是否为根
    bool isRoot(int x) const { return parent[x] < 0; }

    // 获取根所在集合的大小
    int getSetSize(int root) const { return -parent[root]; }
};

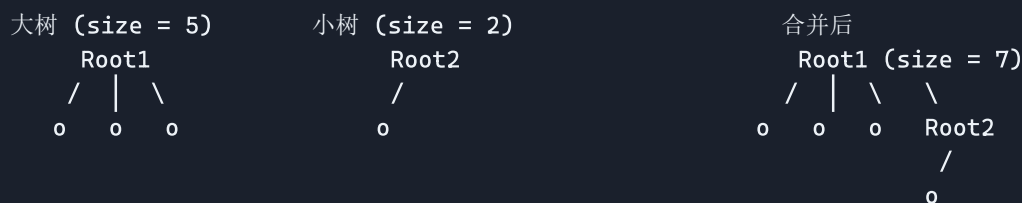
```

工程之美：利用负数符号位消除冗余数组，内存开销直接减半，且绝无两数组不同步的隐患！

13.4 两大核心优化与双剑合璧

13.4.1 启发式合并：按大小（Size）或按秩（Rank）合并的树高对数证明

核心准则：合并两棵树时，永远将较小的树挂到较大的树的根节点之下（矮树挂在大树下）。



（小树挂在大树下，小树中所有节点的深度仅增加 1，而大树所有节点的深度完全不受影响！）

树高上界定理证明：

定理：采用按大小（或按秩）合并，包含 (n) 个节点的树，其高度 $(h \leq \lfloor \log_2 n \rfloor + 1)$ 。

证明：

考虑任意一个节点 (x) 。只有当包含 (x) 的树被合并到一棵规模至少与它自身一样大的树之下时， (x) 的深度才会增加 1。

因此， (x) 的深度每增加 1，它所属树的节点总规模至少翻倍 $(\times 2)$ ！

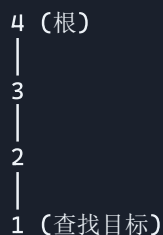
因为全树总节点数至多为 (n) ，规模至多翻倍 $(\log_2 n)$ 次。

故任何节点的深度绝对不会超过 $(\lfloor \log_2 n \rfloor + 1)$ ！查找与合并在最坏情况下稳稳锁定在 $(O(\log n))$ ！

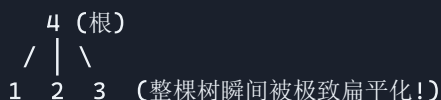
13.4.2 路径压缩（Path Compression）：读操作就地扁平化

按大小合并已经将树高限制在 $(O(\log n))$ ，但计算机科学家们更进一步：**为什么每次 `find` 爬树之后，不顺手把沿途的所有节点直接挂到根节点上？**

【路径压缩前】



【执行 find(1) 路径压缩后】



极简的一行递归实现:

```
int find(int x) {  
    if (parent[x] < 0) return x; // 找到了根  
    return parent[x] = find(parent[x]); // 递归寻根，并就地更新父指针!  
}
```

13.5 复杂度奇迹：阿克曼函数与反阿克曼函数 $\alpha(n)$

当按大小合并与路径压缩同时使用时，Robert Tarjan 于 1975 年证明了并查集的单次操作均摊时间复杂度为：[$T(n) = O(\alpha(n))$] 其中 $(\alpha(n))$ 为反阿克曼函数 (Inverse Ackermann Function) 。

阿克曼函数的增长有多恐怖？

阿克曼函数 $A(m, n)$ 的定义如下：[$A(0, n) = n + 1$] [$A(m, 0) = A(m - 1, 1)$] [$A(m, n) = A(m - 1, A(m, n - 1))$] - $A(1, n) = n + 2$ - $A(2, n) = 2n + 3$ - $A(3, n) = 2^{\{n+3\}} - 3$ - $A(4, 1) = 2^{\{16\}} - 3 = 65,533$ - $A(4, 2) = 2^{\{65536\}} - 3$ —— 这个数字的十进制位数超过 19,700 位，远超全宇宙原子总数 ($\approx 10^{\{80\}}$) ！

反阿克曼函数的物理直观：

$(\alpha(n))$ 定义为满足 $A(k, k) \geq n$ 的最小整数 (k) ： - 当 $(n \leq 65,536)$ 时， $(\alpha(n) \leq 3)$ ； - 当 $(n \leq 2^{\{65536\}})$ 时， $(\alpha(n) \leq 4)$ ； - 对于人类现实世界中能接触到的任何宇宙级天文数字， $(\alpha(n) \leq 4)$ ！
因此，在一切工程实践中，我们完全可以自信地将并查集的操作视为 $(O(1))$ 常数时间！

13.6 高阶拓展：带权并查集与可撤销并查集 (Rollback DSU)

1. 带权并查集 (Weighted DSU)

在节点与父节点之间的边上赋予权值（代表两者间的相对关系，如相对距离、模关系）。

在路径压缩时，根据群论或向量加法同步更新权值：`weight[x] += weight[oldParent]`。

经典应用：POJ 1182 《食物链》（A 吃 B、B 吃 C、C 吃 A 的模 3 关系判定）。

2. 可撤销并查集 (Rollback DSU)

在某些分治算法（如动态图连通性、线段树分治）中，需要支持“撤销上一次 union 操作”。

- **关键设计**：放弃路径压缩（因为路径压缩会破坏原有的树形拓扑，无法回退），仅使用按大小合并（维持树高 $O(\log n)$ ）； - 维护一个历史变更栈 `std::stack<Operation>`，每次回退只需恢复修改过的两个数组槽位即可。

本章小结

1. **等价关系的极简抽象**：并查集将动态连通性简化为双亲指针森林。
2. **负数复用编码**：用负数一石二鸟兼顾根标志与集合大小，是 C++ 内存布局的精妙范式。
3. **双重优化实现理论极限**：按大小合并控制树高上界，路径压缩就地摊平分支。
4. **反阿克曼函数的工程意义**：这是理论计算机科学中极其罕见但确定性的“非严格常数的实用常数”。

思考题与经典大厂面试题

1. **若仅使用路径压缩**：若只保留路径压缩而放弃按大小/按秩合并，最坏单次操作复杂度是多少？请构造一个退化序列。
2. **岛屿数量与动态连通**：在二维网格地图中，如何利用并查集实时计算水域变为陆地后连通岛屿的数量变化？
3. **二分图判定的并查集解法**：给定一个无向图，如何使用“扩展域并查集”（将每个点拆分为 (x) 和 $(x + n)$ 两个虚拟节点）在线性时间内判定图是否为二分图？

<- 第 13 章 并查集 | 课程目录 | 第 15 章 后缀树与后缀数组 ->

所属模块：第七部分：排序
对应实现：sort.cpp

本章导航

- 14.1 排序问题的理论基石：稳定性与决策树下界证明
 - 14.1.1 稳定性的工程价值（多关键字排序）
 - 14.1.2 比较排序下界的决策树严格数学证明（Stirling 公式）
- 14.2 $O(n^2)$ 基础排序的优雅与局限
 - 14.2.1 插入排序：小数据量与近乎有序数据的无敌王者
 - 14.2.2 希尔排序（Shell Sort）：打破平方屏障的步长跃迁
- 14.3 $O(n \log n)$ 三大巨头的分治与堆化
 - 14.3.1 归并排序（Merge Sort）与逆序对统计
 - 14.3.2 快速排序（Quick Sort）：三大划分方法深度剖析
 - 14.3.3 堆排序（Heap Sort）：极致的 $O(1)$ 空间权衡

- 14.4 工业级混合排序之王：TimSort 深度解密
 - 14.4.1 真实世界数据的 Run 模式与 minrun 选择
 - 14.4.2 栈式归并约束与飞奔模式（Galloping Mode）
- 14.5 超越比较下界：非比较排序的数论哲学
 - 14.5.1 计数排序：前缀累加和的稳定定位法
 - 14.5.2 基数排序：为什么 LSD 必须从低位到高位？
- 14.6 海量数据与外存挑战：外部排序与败者树（Loser Tree）
 - 14.6.1 K 路平衡归并的两阶段流程
 - 14.6.2 败者树原理：为什么比较次数比堆减少一半？
 - 14.6.3 置换-选择排序：生成 2M 长度初始归并段的黑魔法
- 14.7 十大排序全维度横向雷达与选型指南
- 本章小结
- 思考题与经典大厂面试题

第 14 章 排序算法

难度：★★☆☆☆ | 代码：sort.cpp

前置知识：第 1 章 渐进复杂度与主定理，第 5 章 二叉树性质，第 7 章 二叉堆原理。

学习目标：1. 理解稳定性的工程实质与比较排序理论下界 $\Omega(n \log n)$ 的数学证明；2. 熟练掌握快速排序 Lomuto、Hoare 与三向切分划分法；3. 掌握 Python/Java 默认排序器 TimSort 的核心运作机制；4. 理解计数与基数排序突破下界的先决条件；5. 掌握外部排序中的 K 路归并、败者树重赛优化与置换-选择排序。

14.1 排序问题的理论基石：稳定性与决策树下界证明

14.1.1 稳定性的工程价值（多关键字排序）

稳定性定义：若待排序序列中存在两个相等的关键字 ($K_i = K_j$)，且在排序前 (K_i) 领先于 (K_j)；若排序后 (K_i) 依然严格领先于 (K_j)，则称该算法是**稳定的 (Stable)**。

原始学生成绩表（已预先按学号 1, 2, 3 升序排列）：

[（张三，90分，学号1），（李四，90分，学号2），（王五，80分，学号3）]

【若使用稳定排序按分数降序】：

[（张三，90分，学号1），（李四，90分，学号2），（王五，80分，学号3）]

（同分的张三依然排在李四前面，保持了先前的学号顺序！）

【若使用不稳定排序（如快排/堆排）】：

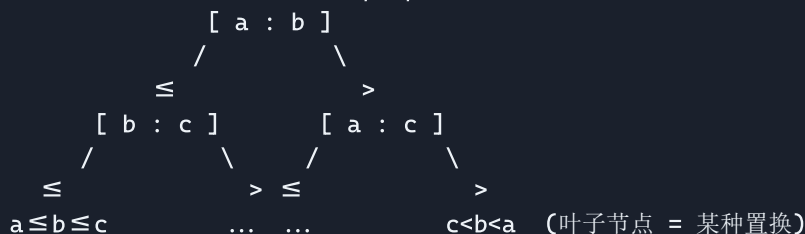
[（李四，90分，学号2），（张三，90分，学号1），（王五，80分，学号3）]

（学号顺序被强行打乱，之前排序的成果前功尽弃！）

14.1.2 比较排序下界的决策树严格数学证明 (Stirling 公式)

在**基于比较的排序模型 (Comparison Sort)** 中，算法只能通过两两比对元素 $(a_i \leq a_j)$ 来决定相对次序。

决策树模型 (对包含 3 个元素的数组 $[a, b, c]$ 进行比较):



证明过程:

1. (n) 个互异元素共有 $(n!)$ 种不同的排列置换;
2. 排序算法必须能够区分出这 $(n!)$ 种可能, 这意味着决策树的**叶子节点总数** $(L \geq n!)$;
3. 一棵高度为 (h) 的二叉树, 其叶子节点最多为 (2^h) 个。因此必有: $[2^h \geq L \geq n! \implies h \geq \log_2(n!)]$
4. 根据**斯特林公式 (Stirling's Approximation)**: $[n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n]$ $[\log_2(n!) = \sum_{i=1}^n \log_2 i \geq \sum_{i=\lceil n/2 \rceil}^n \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \Omega(n \log_2 n)]$ **结论: 在最坏情况下, 任何基于比较的排序算法至少需要执行 $(\lceil \log_2(n!) \rceil \approx \Omega(n \log n))$ 次比较!** 任何号称“比较排序做到 $(O(n))$ ”的说法在数学上都是荒谬的。

14.2 $O(n^2)$ 基础排序的优雅与局限

14.2.1 插入排序: 小数据量与近乎有序数据的无敌王者

模拟摸牌理牌的过程: 已排序区间在左, 未排序区间在右。每次取一张新牌, 在左侧已排序区间中从右向左扫描并后移大于它的牌, 找到合适位置插入。

- **最好情况 (完全有序)**: 每张牌只需比对 1 次, 总时间 $(O(n))$;
- **小数据性能极速**: 当 $(n \leq 32 \sim 64)$ 时, 由于无递归开销且完全是连续内存比较, 其常数因子比快排小得多! 现代高性能混合排序 (如 `std::sort` 的 Introsort 和 TimSort) **在底层均无一例外地退回使用插入排序**。

14.2.2 希尔排序 (Shell Sort): 打破平方屏障的步长跃迁

1959 年 Donald Shell 提出了第一个突破 $(O(n^2))$ 的排序算法。

核心思想: 通过步长序列 (Gap) 将数组分组, 对每组分别执行插入排序。步长逐渐缩小, 最终 `gap = 1` 时数组已高度有序, 插入排序近乎以 $(O(n))$ 极速收尾。 - **Hibbard 增量序列** $((2^k - 1))$: 最坏时间复杂度为 $(O(n^{1.5}))$; - **Sedgewick 增量序列**: 最坏时间复杂度可优化至 $(O(n^{4/3}) \approx O(n^{1.33}))$ 。

14.3 $O(n \log n)$ 三大巨头的分治与堆化

14.3.1 归并排序 (Merge Sort) 与逆序对统计

- **分治法 (Divide and Conquer)** : 将数组一分为二, 递归排序左右两半, 再通过两个指针以 $O(n)$ 线性合并;
- **严格稳定**: 合并时只要左半部分元素 $\leq$ 右半部分元素, 优先放入左侧元素;
- **空间代价**: 合并时需要 $O(n)$ 的辅助数组;
- **逆序对统计应用**: 当右侧指针元素 $arr[j]$ 被优先取走时, 说明左侧区间内从当前位置到中点的所有剩余元素 (共 $mid - i + 1$ 个) 均大于 $arr[j]$, 借此可在线性对数时间内统计出数组中的逆序对总量。

14.3.2 快速排序 (Quick Sort) : 三大划分方法深度剖析

由 Tony Hoare 于 1959 年发明。其平均性能是同级别比较排序中常数因子最小的, 被誉为 20 世纪最伟大的算法之一。

1. Lomuto 划分 (单向推进, 逻辑直观)

选取末尾元素为 Pivot, 维护指针 i 。所有比 Pivot 小的元素被连续排在 $[0, i)$, 指针 j 向前探路:

```
[ 小于 Pivot 区域 (0..i) ] [ 大于等于区域 (i+1..j-1) ] [ 未知探索区 (j..n-2) ] [ Pivot ]
```

2. Hoare 划分 (双向对撞, 极其高效)

左右双指针同时向中间扫描。左指针寻找大于等于 Pivot 的元素, 右指针寻找小于等于 Pivot 的元素, 两者发生逆序则交换并继续推进。交换次数远少于 Lomuto。

3. 三向切分 (3-Way Partition / 荷兰国旗问题)

面对包含海量重复元素的数组 (如只有 0 和 1), 普通快排会因为无法均分元素而退化为 $O(n^2)$! 三向切分将数组严格划分为三部分:

```
< Pivot 区域 | == Pivot 区域 | 未处理区域 | > Pivot 区域
```

只需对 **< Pivot** 和 **> Pivot** 的两侧递归, 相等区间直接跳过, 排序速度接近线性 $O(n)$!

14.3.3 堆排序 (Heap Sort) : 极致的 $O(1)$ 空间权衡

先用 $O(n)$ 建大顶堆, 再将堆顶与末尾交换并下沉, 原地完成排序。不占用额外空间, 但由于在数组中跨步下沉寻址, **CPU 缓存局部性较差**, 实际常数慢于快排。

14.4 工业级混合排序之王：TimSort 深度解密

2002 年 Tim Peters 为 Python 语言设计了 TimSort，随后被 Java SE 7、Android 和 Chrome V8 引擎采纳为标准排序器。

14.4.1 真实世界数据的 Run 模式与 minrun 选择

真实世界的的数据极少是完全纯随机的，往往包含大量的“局部连续单调递增”或“局部单调递减”的片段，称为 **Run**。

- **识别 Run**：扫描输入数据，若发现连续递增，扩展该 Run；若发现连续严格递减，就地将其翻转为递增； - **minrun 哲学**：为了保证归并平衡，TimSort 规定每个 Run 的长度至少为 **minrun**（通常在 32 到 64 之间）。若识别出的自然 Run 长度不足 **minrun**，使用**二分插入排序（Binary Insertion Sort）**强行补齐！

14.4.2 栈式归并约束与飞奔模式（Galloping Mode）

- **归并栈不变量**：维护一个存放已就绪 Run 的栈，栈顶三个 Run 的长度 (A, B, C) 必须时刻满足：
 - $(A > B + C)$
 - $(B > C)$这保证了归并过程高度贴合斐波那契数列，使得归并层数严格控制在 $(O(\log n))!$
- **飞奔模式（Galloping Mode）**：当归并两个 Run 时，若发现连续多次都是其中一个 Run 的元素胜出（阈值通常为 7 次），算法认为该 Run 存在大块优势区间，立即切换为**指数步长搜索（1, 2, 4, 8...）**配合**二分查找**，一次性批量拷贝整片数据，大幅削减比较次数！

14.5 超越比较下界：非比较排序的数论哲学

非比较排序不进行元素间的大小比对，而是利用键的数字特征，实现 $(O(n))$ 级别的突破。

14.5.1 计数排序：前缀累加和的稳定定位法

当待排元素均为 $([0, k])$ 内的整数且 $(k = O(n))$ 时： 1. 统计每个元素出现的频次 **count[val]**； 2. **核心技巧：计算前缀和！** **count[i] = count[i] + count[i-1]**。此时 **count[val]** 的值恰好表示“小于等于 **val** 的元素总个数”，即该元素在输出数组中的**最终下标上限**； 3. **逆序扫描原数组**：将元素放入对应位置并递减 **count**，完美保证稳定性！

14.5.2 基数排序：为什么 LSD 必须从低位到高位？

最低位优先（LSD, Least Significant Digit first）： - 依次按个位、十位、百位进行稳定的计数排序； - **为什么必须从低位到高位？**

因为高位对大小起决定性支配作用！根据稳定性定理，在完成了第 (k) 位的排序后，若两数第 (k) 位相同，它们在第 $(k-1)$ 位形成的相对顺序将被稳定保持下来！

14.6 海量数据与外存挑战：外部排序与败者树（Loser Tree）

当数据量高达 1TB，而物理内存仅有 1GB 时，内排序无法一次性载入全部数据。

14.6.1 K 路平衡归并的两阶段流程

1. 阶段一：生成初始归并段 (Runs)

分批次将 1GB 数据读入内存，用快排排好序后写回磁盘，生成 (M) 个初始有序段；

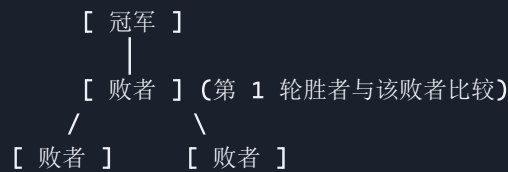
2. 阶段二：K 路多路归并

同时打开 (K) 个有序段文件的输入流，每次从这 (K) 个流中选出最小者输出到最终文件。

14.6.2 败者树原理：为什么比较次数比堆减少一半？

在 (K) 路归并中，若使用常规二叉堆选最小：- 堆顶弹出后，推入一个新元素，该元素需要与左右两个孩子各比较一次，在每层需要 2 次比较，总比较次数为 $(2 \log_2 K)$ 。

败者树结构：内部节点记录的是【败者（输者）的归并段编号】，胜者（赢者）继续向上比



- **败者树重赛 (Replay)**：新元素进入后，沿着到根节点的单一路径向上，**每层只需与该节点上记录的旧败者进行 1 次比较**！输者留在节点中，赢者继续向上！
- **比较次数由 $(2 \log_2 K)$ 减半为 $(\log_2 K)$ ！**在万级路数归并中，极大降低了 CPU 比较指令开销！

14.6.3 置换-选择排序：生成 2M 长度初始归并段的黑魔法

普通分块只能生成长度为内存大小 (M) 的归并段。严蔚敏《数据结构》中经典的**置换-选择排序 (Replacement Selection)** 利用小顶堆：- 只要输入的新元素比刚刚输出的元素大，就可继续纳入当前归并段；- **数学期望推导**：平均可生成长度高达 **(2M) 的超长归并段**！初始段数量直接减半，外部 I/O 归并轮数减少整整一轮！

14.7 十大排序全维度横向雷达与选型指南

算法	平均时间	最坏时间	最好时间	空间复杂度	稳定性	核心工业场景
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	小数组 ($n < 32$)、近乎有序数据
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定	嵌入式硬件、无递归栈开销场景
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	教学演示 (工业界几乎不用)
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	极力减少写入交换次数的场景
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	不稳定	通用内存计算、基本类型数组排序
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	链表排序、外部排序、要求稳定排序
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定	内存受限且绝对不能承受 $O(n^2)$ 最坏情况
TimSort	$O(n \log n)$	$O(n \log n)$	$O(n)$	$O(n)$	稳定	Python / Java / V8 语言级标准排序
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	稳定	键值范围极小的离散整数排序
基数排序	$O(d \cdot (n + k))$	$O(d \cdot (n + k))$	$O(d \cdot (n + k))$	$O(n + k)$	稳定	定长数字、手机号、定长字符串

本章小结

- 1. **比较下界是数学律令：** $O(n \log n)$ 设立了不可逾越的理论鸿沟，非比较排序通过利用数字位权巧妙突围。
- 2. **快排的常数王者地位：**缓存局部性与划分策略让快速排序在通用内排序中独占鳌头。
- 3. **TimSort 敬畏现实数据：**识别自然有序的 Run 与飞奔模式，是算法理论向工程现实妥协并超越的典范。
- 4. **外部排序化解空间绝境：**败者树重赛减半与置换选择扩长段，彻底征服了海量磁盘数据的吞吐瓶颈。

思考题与经典大厂面试题

- 1. **三数取中法的防退化机制：**快速排序中，三数取中 (Median-of-Three) 是如何选定 Pivot 的？为什么它能有效抵御针对已排序数组的 $O(n^2)$ 恶化？

2. **寻找未排序数组中的第 K 大元素 (QuickSelect)**：利用快速排序的划分思想，如何以 $O(n)$ 的平均时间复杂度（优于堆排序的 $O(n \log k)$ ）直接定位第 (K) 大元素？
3. **外部排序的 I/O 次数精确计算**：设待排数据有 128 个初始归并段，采用 4 路平衡归并，需要进行多少轮归并？若采用 8 路归并，I/O 次数减少多少？

所属模块：第八部分：高级字符串

对应实现：suffixtree.cpp / suffixarray.cpp

本章导航

- 15.1 全文本索引的核心公理：子串即后缀的前缀
- 15.2 后缀树 (Suffix Tree)：压缩 Trie 的几何进化
 - 15.2.1 从 $O(n^2)$ 后缀 Trie 到 $O(n)$ 后缀树的边压缩
 - 15.2.2 终止符 \$ 的关键作用与节点数上界证明
- 15.3 Ukkonen 算法 $O(n)$ 在线构建核心秘籍
 - 15.3.1 活动点 (Active Point) 三元状态机
 - 15.3.2 后缀链接 (Suffix Link) 的虫洞跳转
 - 15.3.3 扩展三规则 (Rule 1 / 2 / 3) 与提前终止
- 15.4 后缀数组 (Suffix Array, SA)：紧凑内存的工程典范
 - 15.4.1 SA 数组与 Rank 数组的互逆映射
 - 15.4.2 倍增算法 (Doubling Algorithm) 构建原理
- 15.5 最长公共前缀 (LCP) 与 Kasai 算法 $O(n)$ 线性推导
 - 15.5.1 Kasai 不变量： $\text{height}[\text{rank}[i]] \geq \text{height}[\text{rank}[i-1]] - 1$ 的严格证明
 - 15.5.2 线性扫描代码实现
- 15.6 四大经典字符串难题的 SA + LCP 妙解
 - 15.6.1 单/多模式串快速匹配 (二分查找)
 - 15.6.2 最长重复子串 (LRS)
 - 15.6.3 两串最长公共子串 (LCS, 分隔符拼接法)
 - 15.6.4 本质不同子串计数公式推导
- 15.7 后缀树 vs 后缀数组：理论巅峰与工程实用的对决
- 本章小结
- 思考题与经典大厂面试题

第 15 章 后缀树与后缀数组

难度：★★★★★ | 代码：suffixtree.cpp / suffixarray.cpp

前置知识：第 4 章 字符串匹配思想，第 5 章 树的遍历，第 14 章 基数排序与快速排序。

学习目标： 1. 理解“子串等于某个后缀的前缀”这一全文索引第一性原理； 2. 掌握后缀树通过边压缩将空间从 $(O(n^2))$ 降维至 $(O(n))$ 的机制； 3. 掌握 Ukkonen 算法中“活动点、后缀链接、三类扩展规则”的在线更新机制； 4. 熟练掌握后缀数组 (SA) 的倍增构造法与二分匹配算法； 5. 掌握 Kasai 算法的核心不等式证明，并在 $(O(n))$ 线性时间内解决最长重复子串与本质不同子串计数问题。

15.1 全文索引的核心公理：子串即后缀的前缀

在文本检索和生物信息学（如人类基因组 30 亿对碱基的比对）中，我们面临一个核心挑战：**主文本 (T) 极其庞大且固定，而查询模式串 (P) 会到来成千上万次。**

若每次都从 KMP 从头扫描文本，单次查询需 $(O(n))$ ，成本高昂。我们必须对主文本 (T) 进行预处理，建立**全文索引 (Full-Text Index)**。

全文索引的核心公理：
文本串 (T) 的任意一个子串 (Substring)，必然是 (T) 的某个后缀 (Suffix) 的前缀 (Prefix) ！

以 $T = \text{"banana"}$ 为例：
所有的 6 个后缀为：
 后缀 0: "banana"
 后缀 1: "anana"
 后缀 2: "nana"
 后缀 3: "ana"
 后缀 4: "na"
 后缀 5: "a"

考察任意子串：
 子串 "nan" → 正好是 后缀 2 ("nana") 的前 3 个字符！
 子串 "ana" → 正好是 后缀 1 ("anana") 的前 3 个字符，也是后缀 3 ("ana") 的前 3 个字符！

数学结论： 只要我们构建了一种能够高效检索所有后缀前缀的数据结构，**就等于在同一瞬间解决了该文本上所有可能的子串查询问题！**

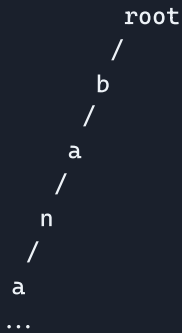
15.2 后缀树 (Suffix Tree)：压缩 Trie 的几何进化

15.2.1 从 $O(n^2)$ 后缀 Trie 到 $O(n)$ 后缀树的边压缩

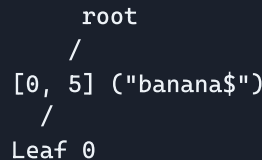
最直观的方法是将 (T) 的所有后缀全部插入到一棵字典树 (Trie) 中，称为**后缀 Trie**： - **致命缺陷**：若 $(T = \text{"a"}^n \text{"b"}^n \text{"c"}^n)$ ，后缀 Trie 的节点总数将高达 $(O(n^2))$ ！对于 100KB 的文本，节点数突破 100 亿，内存直接崩溃！

1973 年 Weiner 与 1976 年 McCreight 提出了**后缀树 (Suffix Tree)**，引入了**路径压缩 (Path Compression)**： - **压缩单分支路径**：将所有没有分支的单链内部节点全部抹除，将一整串连续字符合并为一条复合边； - **边上的区间表示**：边上不存储真实的字符串拷贝，而是仅存储一个下标二元组 $[start, end]$ ，代表子串 $(T[start .. end])$ ，边开销恒定为 $(O(1))$ ！

【后缀 Trie (节点数爆棚 $O(n^2)$)】



【后缀树 (压缩单分支, 节点数 $\leq 2n$)】



15.2.2 终止符 \$ 的关键作用与节点数上界证明

在构建后缀树时, 必须在文本串末尾追加一个从未在文本中出现过的独特字符 (通常为 **\$**) : - 作用: 防止某个后缀恰好是另一个较长后缀的前缀 (例如 **"an"** 与 **"banana"**), 导致该较短后缀停留在内部节点而无法到达叶子; 追加 **\$** 后, 所有后缀两两互不为前缀, 每个后缀必然唯一对应树中的一个叶子节点!

节点数上界定理:

- 包含 (n) 个字符的字符串 (含 **\$**), 恰好有 (n) 个叶子节点;
- 在压缩树中, 每个内部节点至少拥有 2 个孩子 (度 ≥ 2);
- 根据二叉/多叉树性质, 拥有 (n) 个叶子且每个分支至少 2 度的树, 其内部节点数至多为 $(n - 1)$;
- 结论: 整棵后缀树的节点总数绝对不会超过 $(2n - 1)$, 空间复杂度严格为 $O(n)$!

15.3 Ukkonen 算法 $O(n)$ 在线构建核心秘籍

1995 年 Esko Ukkonen 发表了著名的在线线性时间后缀树构建算法。算法从左到右逐个字符推进, 维护一个不变量: 在第 (i) 步结束时, 成功构建出前缀 $T[0..i]$ 的隐式后缀树。

15.3.1 活动点 (Active Point) 三元状态机

算法绝不从根节点重复扫描, 而是维护一个活动点 (Active Point) 三元组: $[\text{active_node}, \text{active_edge}, \text{active_length}]$ - **active_node**: 当前插入操作发生的基准节点; - **active_edge**: 当前正沿着哪一条以该字符开头的出边向下走; - **active_length**: 在当前出边上已经深入了几个字符的距离。

15.3.2 后缀链接 (Suffix Link) 的虫洞跳转

若某个内部节点 (v) 代表子串 $(x\alpha)$ (x 为单个字符, α 为子串), 则 (v) 的后缀链接 (Suffix Link) 指向代表子串 (α) 的内部节点。 - 作用: 当在节点 (v) 分裂创建新叶子后, 无需回到树根重新下潜, 直接顺着后缀链接像“穿越虫洞”一样以 $O(1)$ 跳转到对应节点, 继续执行下一个后缀的插入!

15.3.3 扩展三规则 (Rule 1 / 2 / 3) 与提前终止

在处理新字符 $T[i]$ 时: - **Rule 1 (叶子延伸)**: 当前路径到达了某个叶子, 直接将边末尾的 **end** 推进 (在代码中通过全局指针变量实现 $O(1)$ 批量隐式延伸); - **Rule 2 (分裂建点)**: 当前路径上没有新字符 $T[i]$, 在此处

把边切断，分裂出一个新的内部节点，并分出一条通往新叶子的分支； - Rule 3 (字符已存在 / 提前终止!) : 当前路径上已经存在新字符 (T[i])! 这意味着后续所有更短的后缀此前也必然已经存在，本次阶段直接提前终止! 将未插入的计数累积到 remainder，立即进入下一个字符! 正是 Rule 3 赋予了 Ukkonen 算法惊人的 (O(n)) 线性性能!

15.4 后缀数组 (Suffix Array, SA) : 紧凑内存的工程典范

尽管后缀树理论完美，但其每个节点包含多个子指针、区间边界和后缀链接，指针开销巨大 (在 64 位机器上每个字符通常占用 20~40 字节内存)。

1990 年 Manber 和 Myers 提出了后缀数组 (Suffix Array, SA)，将树形结构彻底拍扁为一个纯粹的 32 位整型数组!

15.4.1 SA 数组与 Rank 数组的互逆映射

- sa[i] : 将字符串 (S) 的所有 (n) 个后缀按字典序升序排列后，排在第 (i) 名的后缀在原字符串中的起始下标;
- rank[k] : 起始于下标 (k) 的后缀在字典序中的排名 (名次)。
- 互逆不变量: $[\text{sa}][\text{rank}[i]] = i \quad \text{且} \quad \text{rank}[\text{sa}[k]] = k$

字符串 S = "banana" (长度 6):

下标与对应后缀:

0: banana
1: anana
2: nana
3: ana
4: na
5: a

字典序升序排列:

名次 0: "a" (原下标 5)
名次 1: "ana" (原下标 3)
名次 2: "anana" (原下标 1)
名次 3: "banana" (原下标 0)
名次 4: "na" (原下标 4)
名次 5: "nana" (原下标 2)

故后缀数组 sa = [5, 3, 1, 0, 4, 2]

名次数组 rank = [3, 2, 5, 1, 4, 0] (例如原下标 0 的 "banana" 排在第 3 名)

15.4.2 倍增算法 (Doubling Algorithm) 构建原理

若直接对 (n) 个后缀进行快排，每次字符串比较耗时 (O(n))，总体需 (O(n^2 log n))。

倍增思想 (Manber-Myers) : - 第 1 轮: 按长度为 1 的前缀排序; - 第 2 轮: 利用第 1 轮排名的结果，两两拼装成长度为 2 的关键字进行双关键字排序; - 第 (k) 轮: 已知长度为 (2^{k-1}) 的排名，将后缀 (i) 的前 (2^k) 个字符拆分为: - 第一关键字: 前半段长度 (2^{k-1}) 的排名 (rank[i]); - 第二关键字: 后半段长度 (2^{k-1}) 的排名 (rank[i + 2^{k-1}])。 - 借助基数排序，每轮排序只需 (O(n)); 总共倍增 (ceil(log_2 n)) 轮，整体以极高常数因子在 (O(n log n)) 时间内完成构建!

15.5 最长公共前缀 (LCP) 与 Kasai 算法 $O(n)$ 线性推导

仅有 `sa` 数组只能做模式二分查找。要让后缀数组具备与后缀树完全等价的能力，必须引入**高度数组 (Height / LCP 数组)**！

定义：

- $\text{lcp}(A, B)$ 表示字符串 (A) 和 (B) 的最长公共前缀长度； - `height[i]` 表示在排序后的后缀数组中，**第 (i) 个后缀与第 (i-1) 个后缀的最长公共前缀长度**：
$$\text{height}[i] = \text{lcp}(S[\text{sa}[i]..n-1], S[\text{sa}[i-1]..n-1])$$

15.5.1 Kasai 不变量： $\text{height}[\text{rank}[i]] \geq \text{height}[\text{rank}[i-1]] - 1$ 的严格证明

2001 年 Kasai 等人提出了一个震惊算法界的定理：

Kasai 核心定理：

设 $h[i] = \text{height}[\text{rank}[i]]$ ，即从原串下标 (i) 开始的后缀与其在字典序中前一名后缀的 LCP 长度。则必有：
$$h[i] \geq h[i-1] - 1$$

严格推导证明：

1. 设排在后缀 (i - 1) 前一名后缀为 (k) (即 $\text{rank}[k] = \text{rank}[i-1] - 1$) ；
2. 它们的公共前缀长度为 $(h[i-1])$ ；
3. 若 $(h[i-1] \leq 1)$ ，不等式 $(h[i] \geq 0)$ 自然恒成立；
4. 若 $(h[i-1] > 1)$ ，说明后缀 (i - 1) 和后缀 (k) 具有至少 2 个相同的公共前缀字符：
5. 后缀 (i - 1) 的形式为： $(c \ \alpha \ \dots)$
6. 后缀 (k) 的形式为： $(c \ \alpha \ \dots)$ (其中 (c) 是单个字符， (α) 长度为 $(h[i-1] - 1)$) ；
7. 同时抹去首字符 (c)：
8. 后缀 (i - 1) 变为后缀 $(i = \alpha \ \dots)$
9. 后缀 (k) 变为后缀 $(k + 1 = \alpha \ \dots)$
10. 因为在字典序中 $(k < i - 1)$ ，且首字符相同，抹去首字符后依然有： $(\text{后缀 } k + 1 < \text{后缀 } i)$ ；
11. 此时后缀 (k + 1) 与后缀 (i) 的公共前缀长度至少为 $(h[i-1] - 1)$ ；
12. 而在字典序中排在后缀 (i) 正好紧挨着的前一名后缀，其与后缀 (i) 的 LCP **只会比后缀 (k+1) 更长，绝不可能变短！**
13. 故必有： $(h[i] \geq h[i-1] - 1)$ ！证毕！ ■

15.5.2 线性扫描代码实现

```
std::vector<int> buildHeight(const std::string& s, const std::vector<int>& sa, const std::vec
    int n = s.size();
    std::vector<int> height(n, 0);
    int h = 0; // 当前 lcp 长度

    for (int i = 0; i < n; ++i) {
        if (rank[i] == 0) {
            h = 0;
            continue;
        }
        int j = sa[rank[i] - 1]; // 字典序中紧邻的前一名后缀
        if (h > 0) --h;          // 利用 Kasai 定理：下限至少为 h - 1

        while (i + h < n && j + h < n && s[i + h] == s[j + h]) {
            ++h;
        }
        height[rank[i]] = h;
    }
    return height;
}
```

复杂度分析： `h` 最多增加 (n) 次，每次循环最多减少 1（最多减 (n) 次）。故 `while` 内的字符比对总次数绝对不超过 $(2n)$ ，**计算全套 Height 数组严格为 $O(n)$ ！**

15.6 四大经典字符串难题的 SA + LCP 妙解

15.6.1 单/多模式串快速匹配（二分查找）

在 `sa` 数组上对模式串 (P) 进行二分查找。由于所有后缀在 `sa` 中是严格字典序排列的，只需 $O(|P| \log n)$ 即可找到匹配区间的左右边界。配合 RMQ（区间最值查询）加速 LCP，可进一步优化至 $O(|P| + \log n)$ ！

15.6.2 最长重复子串（LRS）

问题：求字符串中至少出现两次的最长子串。

解法：出现在两处的子串，必然是两个不同后缀的公共前缀。最长公共前缀必定出现在字典序相邻的两项之间！因此， $(\max(\text{height}[i]))$ 即为全串最长重复子串的长度！

15.6.3 两串最长公共子串（LCS，分隔符拼接法）

问题：求字符串 (A) 和 (B) 的最长公共子串。

解法：用未出现过的符号 `#` 拼接两串： $(S = A + \text{"#"} + B)$ 。计算 (S) 的 `sa` 和 `height`。遍历 `height` 数组，寻找满足 $(\text{sa}[i])$ 与 $(\text{sa}[i-1])$ 分别来自不同原串（一个小于 $(|A|)$ ，一个大于 $(|A|)$ ）的最大 $(\text{height}[i])$ ！

15.6.4 本质不同子串计数公式推导

字符串 (S) 的全部非空子串总数上限为 $\frac{n(n+1)}{2}$ 。每个后缀 $\text{sa}[i]$ 贡献 $(n - \text{sa}[i])$ 个前缀子串。其中，与前一个后缀重复的前缀个数恰好是 $\text{height}[i]!$

去重后的本质不同子串总数公式：
$$\text{TotalUniqueSubstrings} = \frac{n(n + 1)}{2} - \sum_{i=1}^{n-1} \text{height}[i]$$

15.7 后缀树 vs 后缀数组：理论巅峰与工程实用的对决

比较维度	后缀树 (Suffix Tree)	后缀数组 (Suffix Array)
内存开销	$O(n)$ ，每个字符约占 20~40 字节（指针繁多）	$O(n)$ ，每个字符仅需 4 字节（极度紧凑）
CPU 缓存表现	差（指针跳转引发高频 Cache Miss）	极优（纯一维数组连续存储，预取友好）
构建复杂度	$O(n)$ （Ukkonen 算法）	$O(n \log n)$ （倍增）或 $O(n)$ （SA-IS 算法）
代码实现难度	极高（状态机与后缀链接极其繁复）	中等（倍增基数排序易于调试与模板化）
模式查询时间	$O(m)$ 沿树下潜	$O(m \log n)$ 二分查找（工程中常数极小）
工业界现状	偏向理论研究	工业界与算法竞赛绝对事实标准（如 BWT 算法、BWA 基因比对工具）

本章小结

- 1. **第一性原理的威力**：子串即后缀之前缀，统摄了全文本子串检索的万千变式。
- 2. **边压缩的巧夺天工**：后缀树通过区间二元组替代显式字符串拷贝，将爆炸的平方级空间降维为严格线性。
- 3. **Ukkonen 的状态转移**：活动点记忆上下文，后缀链接实现跨越式转移，Rule 3 赋予了提前剪枝的生命力。
- 4. **Kasai 不等式的优雅**： $(h \geq h-1)$ 巧妙利用了相邻后缀去掉首字符后的单调有序性，以最朴素的双指针实现了 $O(n)$ 线性计算。

思考题与经典大厂面试题

- 1. **后缀自动机 (SAM, Suffix Automaton)**：后缀自动机是能够接受字符串所有后缀的最小确定有限状态自动机 (DFA)。相比后缀树，SAM 在状态转移和时空开销上有何独到优势？
- 2. **最长回文子串的后缀数组解法**：如何将原串 (S) 与其翻转串 (S^R) 用特殊字符 `#` 拼接，通过后缀数组的 LCP 快速在 $O(n \log n)$ 时间内找出最长回文子串？
- 3. **手算演练**：对于字符串 ($S = \text{"ababa"}$)，请手工写出其所有后缀、`sa` 数组、`rank` 数组和 `height` 数组，并据此计算其本质不同的子串个数。

补充材料：review, selection, and self-check material.

本章导航

- Reference page

附录 A：数据结构选择速查

需求	推荐结构	复杂度
快速增删，不需频繁查找	链表	$O(1)$ 插入/删除
快速查找 + 有序遍历	平衡 BST	$O(\log n)$
最快查找（无序）	哈希表	$O(1)$ 平均
优先级队列	堆	$O(\log n)$ push/pop
局部性访问	伸展树	均摊 $O(\log n)$
大量数据存磁盘	B 树 / B+ 树	$O(\log_m n)$
判断连通性	并查集	$\approx O(1)$
多维搜索	KD 树	平均 $O(\log n)$
模式匹配（单次）	KMP	$O(n+m)$
模式匹配（多次）	后缀树 / 后缀数组	$O(m)$ / $O(m \log n)$
范围查询（数据库）	B+ 树	$O(\log_m n + k)$
Top-K 问题	最小堆	$O(n \log k)$
数据流中位数	双堆	插入 $O(\log n)$
滑动窗口最大值	单调队列	$O(n)$
区间最值查询	稀疏表 / 线段树	$O(1)$ / $O(\log n)$
大文件排序	外部归并	$O(N \log N)$ I/O

补充材料：review, selection, and self-check material.

本章导航

- 线性结构
 - 树
 - 堆与排序
 - 哈希与查找
 - 图
 - 字符串
-

附录 B：常见面试题与练习

线性结构

1. 反转链表（迭代 + 递归）
2. 检测链表是否有环 → 找环入口
3. 合并两个有序链表 → 合并 K 个有序链表
4. 删除链表倒数第 N 个节点
5. 用两个栈实现队列 / 用两个队列实现栈
6. 最小栈 ($O(1)$ 获取最小值)
7. 有效的括号匹配
8. 单调栈 — 下一个更大元素 / 每日温度 / 柱状图中最大矩形
9. 循环队列实现

树

1. 二叉树的最大深度 / 最小深度
2. 判断是否为平衡二叉树
3. 验证 BST 合法性
4. 二叉树的最近公共祖先 (LCA)
5. 二叉树的层序遍历 / 锯齿形遍历
6. 路径总和（是否存在 + 所有路径）
7. AVL 树手动构建（给定插入序列，画图）

堆与排序

1. 数组中的第 K 大元素 (Top-K)
2. 数据流中的中位数（双堆）
3. 归并排序 / 快速排序手写实现
4. 求逆序对数量
5. 外部 K 路归并设计
6. 手写堆排序（建堆 + 依次弹出）
7. 快速选择 (QuickSelect)

哈希与查找

1. 两数之和 (Two Sum)
2. 最长连续序列
3. 字母异位词分组
4. LRU 缓存 (哈希表 + 双向链表)

图

1. 课程表 (拓扑排序判环)
2. 岛屿数量 (DFS/BFS 连通分量)
3. 网络延迟时间 (Dijkstra)
4. 最小生成树 (Kruskal / Prim)
5. 关键路径上的活动识别

字符串

1. 实现 strStr() (KMP)
2. 最长回文子串 (中心扩展 / Manacher)
3. 最长公共前缀
4. 无重复字符的最长子串 (滑动窗口)

补充材料: review, selection, and self-check material.

本章导航

- [指针陷阱](#)
- [递归陷阱](#)
- [边界陷阱](#)
- [算法选择陷阱](#)
- [性能陷阱](#)

附录 C: 常见陷阱与注意事项

指针陷阱

1. `delete` 后未置 `nullptr` → 野指针，后续使用崩溃
2. `new` 后忘记 `delete` → 内存泄漏
3. 默认拷贝构造只复制指针 → 两个对象共享一棵树 → `double free`
→ 自定义深拷贝 `or = delete`
4. 递归删除树 → 必须后序遍历（先子后父）

递归陷阱

1. 忘记 `base case` → 无限递归爆栈
2. 递归返回值不接收 → `BST` 删除/插入时丢失父子链接
3. 深度过大 → 栈溢出（改用迭代 `or` 尾递归）
4. 重复计算 → 使用备忘录（记忆化搜索）

边界陷阱

1. 取 $(i-1)/2$ 前确保 $i>0$
2. `pop()/top()` 前先检查 `!IsEmpty()`
3. 循环队列: $(rear+1)\%cap == front$ 判满（牺牲一个位置）
4. `int` vs `size_t`: 有符号和无符号混用导致意外行为
5. 链表操作时注意处理空链表和单节点链表的特殊情况

算法选择陷阱

1. 稠密图上用 `Kruskal` 不用 `Prim` $O(V^2)$ → 忽略 `Prim` 在稠密图上的优势
2. 需要范围查询时用哈希表 → 哈希表不支持有序遍历
3. 小数据 ($n<50$) 上用快速排序 → 插入排序常数更小
4. 数据已排好序时 `BST` 逐个插入 → 退化为链表 $O(n^2)$
5. 对流式数据用归并排序 → 归并排序需要看到全部数据

性能陷阱

1. 哈希表最坏 $O(n)$: 选择好的哈希函数 + 控制负载因子
2. `BST` 退化成链表: 使用自平衡变体 (`AVL`/红黑树)
3. 递归建树 $O(n^2)$: 每次取中位数确保 $O(n \log n)$
4. 大对象传值: 传引用避免 $O(n)$ 拷贝

本文档基于 18 个可编译运行的 C++ 文件编写，按经典教材体系编排，从绪论到高级字符串，递进讲解。每个章节配备前置知识、内容详解、本章小结和思考题，适合自学和教学参考。

学习建议：从头到尾按顺序阅读，每学完一章尝试回答思考题。代码实现细节请参阅各章标注的 .cpp 源文件。